UNIVERSIDADE PAULISTA

CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

HENDRYO BATISTA SARMENTO – RA R661946

LUCAS ALCÂNTARA LEAL – RA H32CIA9

MATHEUS TERCI – RA T319595

VINNICIUS GABRIEL MATOS DOS SANTOS – RA H76DJJ6

PIM ESTOQUE:

solução tecnológica integrada para o controle de almoxarifado da Fixar Manutenção

SOROCABA

2026

HENDRYO BATISTA SARMENTO – RA R661946
LUCAS ALCÂNTARA LEAL – RA H32CIA9
MATHEUS TERCI – RA T319595
VINNICIUS GABRIEL MATOS DOS SANTOS – RA H76DJJ6

PIM ESTOQUE:

solução tecnológica integrada para o controle de almoxarifado da Fixar Manutenção

Projeto Integrado Multidisciplinar (PIM IV) apresentado à Universidade Paulista – UNIP como requisito parcial para aprovação no 4º/3º período do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas.

SOROCABA
2026

HENDRYO BATISTA SARMENTO – RA R661946

LUCAS ALCÂNTARA LEAL – RA H32CIA9

MATHEUS TERCI – RA T319595

VINNICIUS GABRIEL MATOS DOS SANTOS – RA H76DJJ6

PIM ESTOQUE:

solução tecnológica integrada para o controle de almoxarifado da Fixar Manutenção

Projeto Integrado Multidisciplinar (PIM IV) apresentado à Universidade Paulista – UNIP como requisito parcial para aprovação no 4º/3º período do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas.

Aprovado em: ____/____/_____

BANCA EXAMINADORA

_____ ____/____/_____

Prof.(ª) _____

Universidade Paulista – UNIP

_____ ____/____/_____

Prof.(ª) _____

Universidade Paulista – UNIP

RESUMO

Este trabalho apresenta o PIM Estoque, solução tecnológica integrada desenvolvida para modernizar o controle do almoxarifado da Fixar Manutenção e Instalações Ltda., pequena empresa fictícia de manutenção predial sediada em Sorocaba/SP, dando continuidade ao estudo iniciado no PIM III. O problema central era a gestão de materiais em planilhas e anotações em papel, que provocava divergência de saldo, falta de itens nas obras, compras emergenciais e ausência de registro sobre quem retirou cada material. O objetivo foi planejar, desenvolver e documentar uma solução completa, composta por uma API REST em ASP.NET Core 8 com autenticação JWT e perfis de acesso, um painel web administrativo, um aplicativo Flutter para Android com fila de sincronização para uso sem conexão e um banco de dados SQL Server com procedures, triggers e view. O trabalho percorre as nove etapas propostas: caracterização da organização; modelo de negócio em formato de assinatura (SaaS), com estimativa de custos de nuvem e ponto de equilíbrio; ações de responsabilidade social, diversidade e acessibilidade alinhadas à Lei Brasileira de Inclusão, ao Estatuto da Igualdade Racial e às diretrizes WCAG; arquitetura em camadas orientada a objetos; projeto lógico e físico do banco; plano de infraestrutura no Microsoft Azure com containers e pipeline de integração e entrega contínuas; e gerenciamento ágil com Scrum e Kanban em seis sprints de duas semanas. A metodologia combinou pesquisa bibliográfica nos livros-texto e materiais das disciplinas do semestre com desenvolvimento incremental versionado em Git. Como resultado, obteve-se uma solução funcional em que web e mobile compartilham as mesmas regras de negócio, o saldo nunca fica negativo, o histórico de movimentações é imutável e cada operação identifica o responsável. Conclui-se que a integração entre as disciplinas permitiu construir um produto viável para pequenas empresas, com caminho claro de evolução para relatórios, inventário físico e operação multiempresa.

Palavras-chave: Controle de estoque; ASP.NET Core; Flutter; Computação em nuvem; Scrum.

ABSTRACT

This work presents PIM Estoque, an integrated technological solution developed to modernize the storeroom control of Fixar Manutenção e Instalações Ltda., a fictitious small building maintenance company based in Sorocaba, Brazil, continuing the study started in PIM III. The core problem was the management of materials through spreadsheets and paper notes, which caused inconsistent balances, missing items at job sites, emergency purchases and no record of who withdrew each material. The goal was to plan, develop and document a complete solution, made up of an ASP.NET Core 8 REST API with JWT authentication and access profiles, an administrative web panel, a Flutter Android application with a synchronization queue for offline use, and a SQL Server database with stored procedures, triggers and a view. The work covers the nine proposed stages: characterization of the organization; a subscription (SaaS) business model, with cloud cost estimates and break-even analysis; social responsibility, diversity and accessibility actions aligned with Brazilian inclusion and racial equality laws and the WCAG guidelines; a layered object-oriented architecture; logical and physical database design; a Microsoft Azure infrastructure plan with containers and a continuous integration and delivery pipeline; and agile management with Scrum and Kanban over six two-week sprints. The methodology combined bibliographic research in the textbooks and materials of the semester courses with incremental development versioned in Git. As a result, a working solution was obtained in which web and mobile share the same business rules, the balance never becomes negative, the movement history is immutable and every operation identifies the person responsible. It is concluded that integrating the courses made it possible to build a viable product for small companies, with a clear path toward reports, physical inventory and multi-company operation.

Keywords: Inventory control; ASP.NET Core; Flutter; Cloud computing; Scrum.

LISTA DE FIGURAS

Figura 1 – Organograma da Fixar Manutenção 15
Figura 2 – Ponto de equilíbrio do PIM Estoque 18
Figura 3 – Arquitetura da aplicação web 22
Figura 4 – Fluxo de navegação do aplicativo 28
Figura 5 – Diagrama de classes do back-end 31
Figura 6 – Modelo entidade-relacionamento do PIM Estoque 33
Figura 7 – Arquitetura de implantação no Microsoft Azure 37
Figura 8 – Pipeline de integração e entrega contínuas 39
Figura 9 – Cronograma das sprints 42
Figura 10 – Quadro Kanban da equipe em 10/11/2026 43

LISTA DE QUADROS

Quadro 1 – Processos do almoxarifado e apoio do PIM Estoque 15
Quadro 2 – Business Model Canvas do PIM Estoque 17
Quadro 3 – Critérios WCAG 2.1 e recursos implementados 20
Quadro 4 – Endpoints da API do PIM Estoque 23
Quadro 5 – Configuração da autenticação JWT (Program.cs) 24
Quadro 6 – Permissões por perfil de acesso 26
Quadro 7 – Registro de movimentação no serviço (MovimentacaoService.cs) 26
Quadro 8 – Telas principais do aplicativo 27
Quadro 9 – Envio da fila de pendências (product_catalog.dart) 29
Quadro 10 – Ciclo de vida da Activity e comportamento do aplicativo 29
Quadro 11 – Modelo lógico: tabelas, chaves e relacionamentos 34
Quadro 12 – Trecho da procedure sp_RegistrarMovimentacao (003_procedures.sql) 35
Quadro 13 – Trigger de imutabilidade do histórico (002_triggers.sql) 35
Quadro 14 – Serviços de nuvem do plano de implantação 38
Quadro 15 – Papéis do Scrum na equipe 41
Quadro 16 – Product Backlog 41
Quadro 17 – Sprint Backlog da Sprint 6 42

LISTA DE TABELAS

Tabela 1 – Custos mensais estimados de nuvem 17
Tabela 2 – Planos de assinatura propostos 18
Tabela 3 – Custos fixos mensais estimados 18

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
ACR	Azure Container Registry
API	Application Programming Interface (interface de programação de aplicações)
BMC	Business Model Canvas (quadro de modelo de negócios)
CI/CD	Continuous Integration / Continuous Delivery (integração e entrega contínuas)
CNPJ	Cadastro Nacional da Pessoa Jurídica
CSP	Content Security Policy (política de segurança de conteúdo)
DI	Dependency Injection (injeção de dependência)
DoD	Definition of Done (definição de pronto)
DTO	Data Transfer Object (objeto de transferência de dados)
EF Core	Entity Framework Core
EPI	Equipamento de Proteção Individual
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
LBI	Lei Brasileira de Inclusão da Pessoa com Deficiência
LGPD	Lei Geral de Proteção de Dados Pessoais
MER	Modelo Entidade-Relacionamento
ORM	Object-Relational Mapper (mapeador objeto-relacional)
OWASP	Open Worldwide Application Security Project
PaaS	Platform as a Service (plataforma como serviço)
PBKDF2	Password-Based Key Derivation Function 2
PIM	Projeto Integrado Multidisciplinar
PMBOK	Project Management Body of Knowledge
PO	Product Owner
REST	Representational State Transfer
SaaS	Software as a Service (software como serviço)
SQL	Structured Query Language

TDE	Transparent Data Encryption
TI	Tecnologia da Informação
UNIP	Universidade Paulista
WCAG	Web Content Accessibility Guidelines
WIP	Work in Progress (trabalho em andamento)

SUMÁRIO

1 INTRODUÇÃO 12
2 CARACTERIZAÇÃO DA ORGANIZAÇÃO 14
2.1 Segmento de atuação 14
2.2 Produtos e serviços 14
2.3 Estrutura organizacional 14
2.4 Principais processos 15
2.5 Problemas identificados 15
2.6 Oportunidades de melhoria por meio da tecnologia 15
3 PLANEJAMENTO DA SOLUÇÃO E EMPREENDEDORISMO EM TI 16
3.1 Problema, objetivos e público-alvo 16
3.2 Proposta de valor, benefícios e diferenciais 16
3.3 Modelo de negócio 16
3.4 Análise de viabilidade 17
4 RESPONSABILIDADE SOCIAL, DIVERSIDADE E RELAÇÕES ÉTNICO-RACIAIS 19
4.1 Combate à discriminação 19
4.2 Valorização da diversidade 20
4.3 Recursos tecnológicos inclusivos e acessibilidade 20
4.4 Tecnologia para a promoção da cidadania 21
5 APLICAÇÃO WEB (.NET) 22
5.1 Arquitetura da aplicação 22
5.2 APIs REST 23
5.3 Autenticação 24
5.4 Segurança 25
5.5 Estrutura administrativa 25
5.6 Integração entre módulos 26
6 APLICAÇÃO MOBILE 27
6.1 Telas principais 27
6.2 Fluxo de navegação 27
6.3 Autenticação 28
6.4 Sincronização de dados 28
6.5 Integração com as APIs 29
6.6 Ciclo de vida no Android 29
7 ARQUITETURA DE SOFTWARE 30
7.1 Organização em camadas 30
7.2 Orientação a objetos 30
7.3 Modularização 32

8 BANCO DE DADOS 33
8.1 Modelo entidade-relacionamento 33
8.2 Modelo lógico 33
8.3 Modelo físico 34
8.4 Procedures e view 34
8.5 Triggers 35
8.6 Script completo do banco 35
9 INFRAESTRUTURA EM NUVEM E DEVOPS 37
9.1 Arquitetura em nuvem 37
9.2 Serviços utilizados 37
9.3 Containers 38
9.4 Pipeline CI/CD 38
9.5 Monitoramento 39
9.6 Escalabilidade 39
9.7 Segurança da infraestrutura 39
9.8 Implantação realizada 40
10 GERENCIAMENTO ÁGIL DO PROJETO 41
10.1 Papéis e eventos 41
10.2 Product Backlog 41
10.3 Sprint Backlog 42
10.4 Cronograma das sprints 42
10.5 Quadro Kanban 43
10.6 Definição das entregas 43
11 CONCLUSÃO 44
REFERÊNCIAS 45
APÊNDICE A – SCRIPT SQL COMPLETO DO BANCO DE DADOS 48
APÊNDICE B – CÓDIGO-FONTE DA API (ASP.NET CORE) 54
APÊNDICE C – CÓDIGO-FONTE DO PAINEL WEB 66
APÊNDICE D – CÓDIGO-FONTE DO APLICATIVO MOBILE (FLUTTER) 75
ANEXO A – FICHA DE CONTROLE DO PIM 103

1 INTRODUÇÃO

A transformação digital deixou de ser exclusividade das grandes corporações. Pequenas empresas de serviços também dependem de informação confiável para atender clientes no prazo, controlar custos e planejar compras. No setor de manutenção predial, em que cada chamado exige materiais específicos, a falta de um parafuso, de um disjuntor ou de um EPI pode interromper uma obra inteira, gerar deslocamentos extras e comprometer a reputação da empresa diante do cliente.

Este Projeto Integrado Multidisciplinar (PIM IV) dá continuidade ao estudo realizado no PIM III e tem como tema o desenvolvimento e a implantação de uma solução tecnológica corporativa integrada para apoiar a transformação digital de uma organização. A empresa escolhida é a Fixar Manutenção e Instalações Ltda., organização fictícia de Sorocaba/SP cujo almoxarifado é controlado por planilhas e anotações em papel. A proposta do grupo é o PIM Estoque, sistema composto por uma API REST, um painel web administrativo, um aplicativo móvel e um banco de dados relacional, planejado para operar em nuvem.

O objetivo geral é desenvolver uma solução completa, que vai do planejamento do negócio à definição da infraestrutura, considerando inovação, inclusão, desempenho, escalabilidade, segurança e qualidade. Os objetivos específicos acompanham as nove etapas da proposta: caracterizar a organização; planejar a solução e avaliar sua viabilidade; incorporar ações de responsabilidade social e diversidade; desenvolver as aplicações web e mobile integradas por APIs; definir a arquitetura orientada a objetos; projetar o banco de dados; planejar a infraestrutura em nuvem com práticas de DevOps; e gerenciar o projeto com métodos ágeis.

A justificativa está no impacto direto do controle de materiais sobre o resultado de uma prestadora de serviços: estoque desatualizado leva a compras emergenciais, mais caras, e a perdas sem responsável identificado. Do ponto de vista acadêmico, o projeto integra os conteúdos de Empreendedorismo em TI, Relações Étnico-raciais e Afrodescendência, Desenvolvimento Web com .NET, Desenvolvimento Mobile, Programação Aplicada em .NET, Programação de Banco de Dados, Cloud Computing e DevOps e Gerenciamento de Projetos Ágil.

A metodologia combinou pesquisa bibliográfica, com base nos livros-texto e nas aulas das disciplinas, e desenvolvimento incremental do software em sprints de

duas semanas, com código versionado em Git e verificado por testes automatizados. O documento foi normalizado conforme o guia da Biblioteca da Universidade Paulista - UNIP (2025).

O trabalho está organizado da seguinte forma: a seção 2 caracteriza a organização; a seção 3 apresenta o planejamento da solução e o modelo de negócio; a seção 4 trata de responsabilidade social, diversidade e acessibilidade; as seções 5 e 6 descrevem as aplicações web e mobile; a seção 7 detalha a arquitetura de software; a seção 8, o banco de dados; a seção 9, a infraestrutura em nuvem e DevOps; e a seção 10, o gerenciamento ágil. Por fim, a conclusão retoma os objetivos e aponta trabalhos futuros.

2 CARACTERIZAÇÃO DA ORGANIZAÇÃO

Esta seção apresenta a organização estudada desde o PIM III. Como aquele estudo não havia atribuído nome à empresa, o grupo adotou uma razão social fictícia, preservando as características levantadas.

2.1 Segmento de atuação

A Fixar Manutenção e Instalações Ltda. é uma pequena empresa de Sorocaba/SP, com cerca de 25 colaboradores, que atua em manutenção predial e instalações elétricas e hidráulicas para condomínios, lojas e pequenos prédios comerciais, por contratos mensais e chamados avulsos.

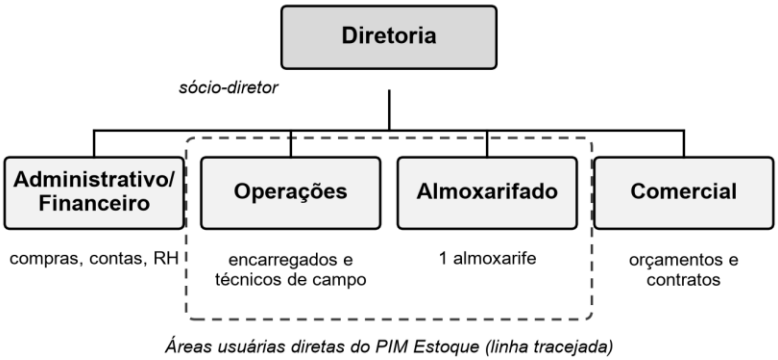
2.2 Produtos e serviços

A empresa não fabrica produtos: vende serviços cuja execução consome materiais, como manutenção preventiva de áreas comuns, atendimento corretivo a chamados com prazo contratual, pequenas instalações e reformas e inspeção de itens de segurança. Para isso, o almoxarifado guarda consumíveis, fixadores, EPIs, ferramentas e materiais elétricos, hidráulicos e de limpeza, cada um com unidade própria (unidade, peça, par, caixa, pacote ou rolo).

2.3 Estrutura organizacional

A estrutura é funcional e enxuta: diretoria (sócio-proprietário), administrativo e financeiro, que também faz as compras, operações, com encarregados e técnicos, comercial e almoxarifado, com um almoxarife. A Figura 1 destaca as áreas usuárias do PIM Estoque.

Figura 1 – Organograma da Fixar Manutenção



Fonte: elaborado pelos autores (2026).

2.4 Principais processos

O Quadro 1 resume os processos do almoxarifado mapeados no PIM III e o apoio do sistema a cada um.

Quadro 1 – Processos do almoxarifado e apoio do PIM Estoque

Processo	Situação anterior	Com o PIM Estoque
Recebimento	Papel; planilha no fim do dia	Entrada com fornecedor e usuário
Retirada para obra	Anotação nem sempre feita	Saída recusada se faltar saldo
Reposição	Percepção do almoxarife	Lista automática pelo mínimo
Consulta de saldo	Ligação ao almoxarifado	Pesquisa no celular
Prestação de contas	Sem histórico confiável	Histórico imutável e auditoria

Fonte: elaborado pelos autores (2026).

2.5 Problemas identificados

O levantamento apontou cinco problemas: saldo da planilha diferente do físico; falta de material na obra, obrigando o técnico a voltar à base; compras emergenciais mais caras; ausência de registro de quem retirou cada item; e dependência do almoxarife, único a saber onde cada material estava.

2.6 Oportunidades de melhoria por meio da tecnologia

Esses problemas têm causa comum: a informação é registrada tarde demais. A tecnologia permite registrar a movimentação na hora, pelo celular, centralizar as regras de saldo, identificar o responsável e alertar antes da falta.

3 PLANEJAMENTO DA SOLUÇÃO E EMPREENDEDORISMO EM TI

Esta seção define a solução proposta para a Fixar e analisa sua viabilidade como produto, uma vez que o mesmo problema se repete em muitas pequenas prestadoras de serviço.

3.1 Problema, objetivos e público-alvo

O problema a ser resolvido é a perda de controle sobre os materiais do almoxarifado, que gera custo e atraso nos serviços. Os objetivos da solução são: manter o saldo correto e sempre não negativo; registrar toda entrada e saída com responsável e motivo; avisar quando um item atingir o estoque mínimo; permitir consulta e registro pelo celular, mesmo sem conexão; e restringir as operações conforme o perfil do usuário.

O público-alvo imediato são almoxarife, técnicos, comprador e diretoria da Fixar; como produto, são pequenas empresas de serviços técnicos, com 10 a 80 colaboradores, que mantêm almoxarifado próprio e ainda usam planilhas.

3.2 Proposta de valor, benefícios e diferenciais

A proposta de valor é “saber o que há no almoxarifado, onde está e quem retirou, em tempo real e pelo celular”. Os benefícios esperados são menos compras emergenciais e idas à base, responsabilização nas retiradas e compras planejadas pelo estoque mínimo. Os diferenciais competitivos são a implantação simples, o foco no almoxarifado de serviços, o uso no celular sem internet, a acessibilidade desde o projeto e a assinatura mensal, sem investimento em servidores.

3.3 Modelo de negócio

Para organizar o modelo de negócio foi utilizado o Business Model Canvas, ferramenta que descreve em nove blocos como uma organização cria, entrega e captura valor (Osterwalder; Pigneur, 2011). O Quadro 2 resume o canvas do PIM Estoque como software como serviço (SaaS).

Quadro 2 – Business Model Canvas do PIM Estoque

Bloco	Definição
Segmentos de clientes	Pequenas empresas de manutenção e serviços técnicos com almoxarifado próprio; cliente-piloto: Fixar Manutenção
Proposta de valor	Saldo confiável em tempo real, rastreio de quem retirou, alerta de reposição e uso no celular sem internet
Canais	Site com teste gratuito de 30 dias, indicação de clientes, associações comerciais e parceiros contábeis
Relacionamento	Autoatendimento, base de ajuda, suporte por chat e implantação assistida no primeiro mês
Fontes de receita	Assinatura mensal por empresa em três planos; serviço opcional de importação de planilhas
Recursos principais	Software (API, painel e app), equipe de desenvolvimento e infraestrutura em nuvem
Atividades principais	Desenvolvimento contínuo, operação da nuvem, suporte e marketing digital
Parcerias principais	Provedor de nuvem (Microsoft Azure), escritórios de contabilidade e distribuidores de materiais
Estrutura de custos	Nuvem, pessoal de suporte e desenvolvimento, impostos, taxas de pagamento e marketing

Fonte: elaborado pelos autores (2026).

3.4 Análise de viabilidade

Os valores a seguir são estimativas do grupo, em reais arredondados, feitas com a calculadora de preços do Azure (Microsoft, 2026b), e variam com câmbio e consumo. A Tabela 1 compara a fase piloto com uma fase de crescimento.

Tabela 1 – Custos mensais estimados de nuvem

Serviço	Configuração (piloto → crescimento)	Piloto (R$/mês)	Crescimento (R$/mês)
App Service (Linux)	B1, 1 instância → S1, 2 instâncias	R$ 75	R$ 760
Azure SQL Database	Basic (5 DTU) → Standard S1 (20 DTU)	R$ 28	R$ 165
Container Registry	Basic → Standard	R$ 28	R$ 110
Key Vault	operações de leitura de segredos	R$ 5	R$ 10
Application Insights	cota gratuita → ingestão paga	R$ 0	R$ 60
Armazenamento e tráfego	backups e saída de dados	R$ 15	R$ 45
Domínio	registro .com.br (rateio mensal)	R$ 5	R$ 5
Total estimado		**R$ 156**	**R$ 1.155**

Fonte: elaborado pelos autores (2026), valores estimados.

O produto será vendido em três planos de assinatura, apresentados na Tabela 2 com a participação esperada de cada um na carteira. Com essa distribuição, o valor médio por cliente (ticket médio) é de R$ 231,50 por mês.

Tabela 2 – Planos de assinatura propostos

Plano	Mensalidade	Inclui	Participação estimada
Essencial	R$ 149	Até 3 usuários, painel web, 500 materiais	40%
Profissional	R$ 249	Até 10 usuários, app com modo offline, fornecedores	45%
Empresarial	R$ 399	Usuários ilimitados, suporte prioritário, importação assistida	15%

Fonte: elaborado pelos autores (2026), valores estimados.

Os custos fixos mensais da operação, listados na Tabela 3, somam R$ 4.650. O custo variável por cliente foi estimado em R$ 25, valor que reúne impostos do Simples Nacional sobre a mensalidade, a taxa do meio de pagamento e uma parcela de suporte e armazenamento.

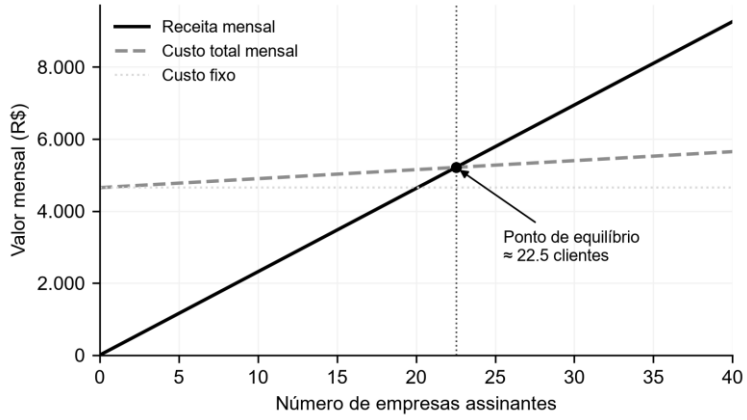
Tabela 3 – Custos fixos mensais estimados

Item	Valor (R$/mês)
Infraestrutura em nuvem (entre piloto e crescimento)	R$ 600
Ferramentas, e-mail e repositório	R$ 150
Suporte e manutenção (meio período)	R$ 3.000
Contabilidade	R$ 400
Marketing digital	R$ 500
Total de custos fixos	**R$ 4.650**

Fonte: elaborado pelos autores (2026), valores estimados.

O ponto de equilíbrio, em número de clientes, é obtido dividindo o custo fixo pela margem de contribuição unitária (ticket médio menos custo variável): R$ 4.650 ÷ (R$ 231,50 − R$ 25) ≈ 22,5. Portanto, a operação se paga a partir de 23 empresas assinantes, como mostra a Figura 2. Com 40 clientes, a margem mensal seria de aproximadamente R$ 3.610, o que recuperaria o investimento inicial estimado em R$ 25.000 (horas de desenvolvimento e registro de marca) em cerca de 7 meses. Os riscos principais, concorrência e resistência à troca da planilha, são mitigados pelo teste gratuito e pela importação dos dados existentes.

Figura 2 – Ponto de equilíbrio do PIM Estoque



Fonte: elaborado pelos autores (2026), valores estimados.

4 RESPONSABILIDADE SOCIAL, DIVERSIDADE E RELAÇÕES ÉTNICO-RACIAIS

Uma solução tecnológica não é neutra: as escolhas de projeto podem ampliar ou reduzir barreiras para as pessoas que a utilizam. Almeida (2019) mostra que o racismo é estrutural, isto é, reproduz-se também por meio de regras e procedimentos institucionais aparentemente imparciais. Por isso, o grupo tratou inclusão e diversidade como requisitos do PIM Estoque, e não como acessórios. Esta seção descreve as ações previstas no sistema e na implantação na Fixar.

4.1 Combate à discriminação

O Estatuto da Igualdade Racial define discriminação racial como toda distinção, exclusão, restrição ou preferência baseada em raça, cor, descendência ou origem que anule ou restrinja direitos em igualdade de condições, e atribui também ao setor privado a promoção da igualdade de oportunidades no trabalho (Brasil, 2010). A Lei nº 7.716/1989 tipifica como crime as práticas resultantes de preconceito de raça ou de cor (Brasil, 1989). No PIM Estoque, essas diretrizes se traduzem em decisões concretas:

a) o cadastro de usuários coleta apenas nome, e-mail e perfil; não há campos de raça, gênero, idade ou foto, seguindo o princípio da necessidade da LGPD (Brasil, 2018) e evitando dados que poderiam servir de base para tratamento discriminatório;
b) o campo de nome aceita o nome social, que é o exibido nas telas e no histórico;
c) o sistema não toma decisões automatizadas sobre pessoas: o histórico registra fatos (quem retirou, quanto e por quê), sem pontuações ou rankings de colaboradores;
d) o histórico é imutável e auditável, o que protege o trabalhador contra acusações infundadas de extravio, pois cada saída tem responsável, data e motivo registrados.

4.2 Valorização da diversidade

A equipe da Fixar reúne pessoas de diferentes origens, idades e níveis de familiaridade com tecnologia. A interface usa linguagem simples, sem jargões, e o material de treinamento usará exemplos e imagens que representem a diversidade étnico-racial e de gênero dos colaboradores. A Lei nº 10.639/2003 tornou obrigatório o ensino de história e cultura afro-brasileira (Brasil, 2003); embora dirigida à educação, ela inspira a empresa a incluir, nas capacitações internas, a contribuição da população negra para o trabalho técnico e a construção civil no país, combatendo estereótipos.

4.3 Recursos tecnológicos inclusivos e acessibilidade

A Lei Brasileira de Inclusão determina a acessibilidade dos sítios mantidos por empresas no país:

> É obrigatória a acessibilidade nos sítios da internet mantidos por empresas com sede ou representação comercial no País ou por órgãos de governo, para uso da pessoa com deficiência, garantindo-lhe acesso às informações disponíveis, conforme as melhores práticas e diretrizes de acessibilidade adotadas internacionalmente (Brasil, 2015, art. 63).

A referência internacional adotada foi a WCAG 2.1, nível AA, que organiza os critérios em quatro princípios: perceptível, operável, compreensível e robusto (W3C, 2018). O Quadro 3 relaciona critérios da norma às soluções implementadas no painel web e no aplicativo.

Quadro 3 – Critérios WCAG 2.1 e recursos implementados

Critério WCAG 2.1	Painel web	Aplicativo Flutter
1.3.1 Informações e relações	HTML semântico (header, nav, main, section), 25 rótulos de formulário e 5 tabelas com caption	Widgets do Material com rótulos e dicas (tooltip) em botões de ícone
1.4.3 Contraste mínimo (4,5:1)	Texto principal 14,4:1; botão primário 6,7:1	Tema com cores de alto contraste
1.4.4 Redimensionar texto	Tamanhos em rem, respeitando o zoom do navegador	Layout testado com texto ampliado e tela pequena
2.1.1 Teclado e 2.4.1 Ignorar blocos	Navegação completa por teclado e link “Pular para o conteúdo”	Ordem de foco natural dos widgets
2.4.7 Foco visível	Contorno de foco destacado (:focus-visible)	Indicador de foco padrão do Material
3.1.1 Idioma da página	Atributo lang="pt-BR"	Textos em português
4.1.3 Mensagens de status	Região aria-live para avisos de sucesso e erro	Semantics com liveRegion nas mensagens do estoque

Fonte: elaborado pelos autores (2026).

Além desses itens, o painel respeita a preferência do sistema por movimento reduzido (prefers-reduced-motion) e associa mensagens de erro aos campos por aria-describedby, o que permite a leitores de tela como NVDA e TalkBack anunciar o problema junto ao campo. No aplicativo, o Flutter expõe a árvore de semântica dos widgets aos serviços de acessibilidade do Android (Google, 2026a), e a navegação adapta-se a telas largas com trilho lateral. Para pessoas com baixa visão ou pouca familiaridade digital, as ações principais usam botões grandes com texto, e não apenas ícones.

4.4 Tecnologia para a promoção da cidadania

Ao organizar o almoxarifado, o PIM Estoque reduz desperdício e compras emergenciais. O grupo propõe à Fixar três ações de cidadania: vagas de aprendiz de almoxarifado para jovens de comunidades periféricas de Sorocaba, com atenção à equidade racial; oficinas gratuitas de letramento digital usando o próprio aplicativo; e doação de materiais com defeito estético a projetos comunitários, com saída registrada no sistema. No produto SaaS, prevê-se desconto para cooperativas e microempreendedores individuais.

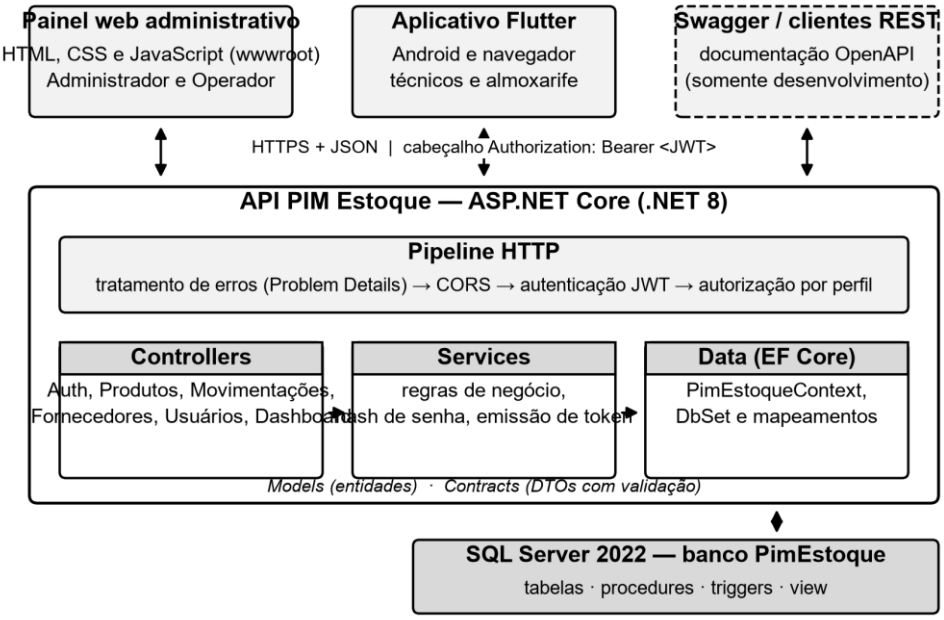
5 APLICAÇÃO WEB (.NET)

A aplicação web é o núcleo do PIM Estoque: concentra as regras de negócio, expõe as APIs consumidas pelo painel e pelo aplicativo e serve o próprio painel administrativo. Foi construída em ASP.NET Core 8 com Entity Framework Core e SQL Server, tecnologias estudadas na disciplina de Desenvolvimento Web com .NET (Peres, 2026).

5.1 Arquitetura da aplicação

A Figura 3 mostra a arquitetura. Os clientes se comunicam com a API por HTTPS, trocando JSON. Dentro da API, cada requisição atravessa o pipeline de middlewares do ASP.NET Core, responsável por tratamento de erros, cabeçalhos de segurança, CORS, autenticação e autorização, antes de chegar ao controller adequado (Peres, 2026, p. 14). Os controllers apenas traduzem HTTP; as regras ficam nos serviços, que acessam o banco pelo contexto do EF Core.

Figura 3 – Arquitetura da aplicação web



Fonte: elaborado pelos autores (2026).

Optou-se por uma API única que também entrega o painel como arquivos estáticos (pasta wwwroot), o que simplifica a implantação em um único container e elimina problemas de CORS entre painel e API, estratégia de implantação integrada discutida por Peres (2026, p. 200).

5.2 APIs REST

A API segue o estilo arquitetural REST, em que os recursos são identificados por URIs e manipulados por uma interface uniforme de métodos HTTP, sem estado de sessão no servidor (Fielding, 2000). O Quadro 4 lista os endpoints, todos com prefixo /api e corpo em JSON com nomes em camelCase.

Quadro 4 – Endpoints da API do PIM Estoque

Método	Rota	Perfil exigido	Finalidade
POST	/api/auth/login	público	Autentica e devolve o token JWT
GET	/api/auth/me	autenticado	Dados do usuário logado
GET	/api/produtos?busca=&reposicao=	autenticado	Lista e pesquisa materiais
GET	/api/produtos/{id}	autenticado	Consulta um material
POST / PUT	/api/produtos[/{id}]	Administrador, Operador	Cadastra ou edita
DELETE	/api/produtos/{id}	Administrador	Exclui (409 se houver histórico)
GET	/api/movimentacoes?produtoId=	autenticado	Últimas 100 movimentações
POST	/api/movimentacoes	Administrador, Operador	Registra entrada ou saída (409 se faltar saldo)
GET	/api/dashboard/resumo	autenticado	Indicadores e saldo por categoria
GET / POST / PUT	/api/fornecedores[/{id}]	leitura: autenticado; escrita: Administrador, Operador	Cadastro de fornecedores
GET / POST / PUT	/api/usuarios[/{id}]	Administrador	Gestão de usuários
GET	/health	público	Verificação de disponibilidade

Fonte: elaborado pelos autores (2026).

Os códigos de status comunicam o resultado: 201 para criação, 400 para dados inválidos, 401 sem autenticação, 403 sem permissão, 404 para recurso inexistente e 409 para conflitos de regra, como part number duplicado ou saldo insuficiente. Erros de regra trazem um JSON com o campo message, exibido diretamente pelos clientes; falhas inesperadas seguem o formato padronizado Problem Details, sem expor detalhes internos. A documentação OpenAPI, com suporte ao token Bearer, é gerada pelo Swagger no ambiente de desenvolvimento (Peres, 2026, p. 93-100).

5.3 Autenticação

Peres (2026, p. 144) distingue autenticação, que confirma a identidade do usuário, de autorização, que decide o que ele pode fazer. No PIM Estoque, a autenticação usa JSON Web Token: após o login com e-mail e senha, a API emite um token assinado com HMAC-SHA256 contendo o identificador, o nome, o e-mail e o perfil do usuário, válido por 8 horas. O token é autocontido e verificável pela assinatura, o que dispensa sessão no servidor (Jones; Bradley; Sakimura, 2015). O cliente o envia no cabeçalho Authorization: Bearer em cada requisição. O Quadro 5 mostra a configuração real da validação.

Quadro 5 – Configuração da autenticação JWT (Program.cs)

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
{
    options.MapInboundClaims = false;
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidIssuer = config["Jwt:Issuer"],
        ValidAudience = config["Jwt:Issuer"],
        IssuerSigningKey = AuthService.ChaveAssinatura(config),
        NameClaimType = "name",
        RoleClaimType = "role",
        ClockSkew = TimeSpan.FromMinutes(1),
    };
    // Usuário desativado perde o acesso imediatamente, mesmo com token ainda
        válido.
    options.Events = new JwtBearerEvents
    {
        OnTokenValidated = async context =>
        {
            var db = context.HttpContext.RequestServices
                .GetRequiredService<PimEstoqueContext>();
            var id = context.Principal!.UsuarioId();
            if (!await db.Usuarios.AnyAsync(u => u.Id == id && u.Ativo))
                context.Fail("Usuário inativo.");
        },
    };
});
```

Fonte: elaborado pelos autores (2026).

Dois cuidados merecem destaque. Primeiro, a aplicação não inicia se a chave de assinatura tiver menos de 32 bytes, impedindo tokens fracos por erro de configuração. Segundo, a cada requisição a API confirma que o usuário continua ativo: um colaborador desligado perde o acesso imediatamente, mesmo que ainda tenha um token válido. As senhas são armazenadas apenas como hash PBKDF2-SHA256, com 100.000 iterações e salt aleatório, e a comparação é feita em tempo constante. Quando o e-mail não existe, o login calcula um hash fictício para gastar o mesmo tempo, sem revelar quais contas estão cadastradas (Microsoft, 2026d).

5.4 Segurança

As medidas de segurança foram escolhidas com base nos riscos mais frequentes em aplicações web catalogados no OWASP Top 10 (OWASP Foundation, 2021):

a) controle de acesso quebrado: política padrão que exige autenticação em todos os endpoints, com exceções explícitas apenas para login e /health, e atributos de perfil nas operações de escrita;
b) falhas criptográficas: tráfego apenas por HTTPS no ambiente de nuvem, senhas com PBKDF2 e segredos fornecidos por variáveis de ambiente, nunca gravados no repositório;
c) injeção: todo acesso a dados passa por consultas parametrizadas do EF Core e por procedures com parâmetros tipados;
d) configuração insegura: cabeçalhos X-Content-Type-Options, X-Frame-Options, Referrer-Policy e Content-Security-Policy restritiva no painel, Swagger habilitado somente em desenvolvimento e CORS limitado às origens configuradas;
e) validação de entrada: DTOs com Data Annotations (part number de oito dígitos com prefixo válido, unidade, CNPJ de 14 dígitos), repetidas por restrições CHECK no banco.

5.5 Estrutura administrativa

O painel administrativo é uma aplicação de página única em HTML, CSS e JavaScript, servida pela própria API. Após o login, o menu apresenta as seções Painel (indicadores e saldo por categoria), Materiais, Movimentações, Fornecedores e Usuários; esta última só aparece para administradores. O token fica em sessionStorage, sendo descartado ao fechar a aba. A interface esconde ações não permitidas, mas a decisão final é sempre da API, conforme o Quadro 6, porque regras críticas não podem depender apenas do cliente (Peres, 2026, p. 155).

Quadro 6 – Permissões por perfil de acesso

Funcionalidade	Administrador	Operador	Consulta
Consultar materiais, saldos, histórico e painel	Sim	Sim	Sim
Cadastrar e editar materiais e fornecedores	Sim	Sim	Não
Registrar entradas e saídas	Sim	Sim	Não
Excluir material sem histórico	Sim	Não	Não
Gerenciar usuários e perfis	Sim	Não	Não

Fonte: elaborado pelos autores (2026).

5.6 Integração entre módulos

A integração acontece em três níveis. No nível de aplicação, painel e aplicativo consomem exatamente os mesmos endpoints, de modo que uma saída registrada no celular aparece no painel na próxima consulta. No nível de negócio, o registro de movimentação atualiza o saldo e grava o histórico na mesma transação, com atualização condicional que só ocorre se o saldo resultante não ficar negativo, conforme o Quadro 7. No nível de dados, as triggers de auditoria e de imutabilidade atuam qualquer que seja a origem da alteração, e o EF Core é informado da existência delas para gerar comandos compatíveis (Microsoft, 2026c). Erros de regra são lançados como exceções de negócio e convertidos em respostas HTTP por um filtro global, com mensagens iguais para os dois clientes.

Quadro 7 – Registro de movimentação no serviço (MovimentacaoService.cs)

```
public async Task<MovimentacaoResponse> RegistrarAsync(MovimentacaoRequest
    request, int usuarioId)
{
    await using var transaction = await db.Database.BeginTransactionAsync();
    if (!await db.Produtos.AnyAsync(p => p.Id == request.ProdutoId)) throw
        ErroNegocio.NaoEncontrado("Material não encontrado.");
    string? fornecedorNome = null;
    if (request.FornecedorId is { } fornecedorId)
        fornecedorNome = await db.Fornecedores.Where(f => f.Id == fornecedorId &&
            f.Ativo).Select(f => f.Nome).SingleOrDefaultAsync()
            ?? throw ErroNegocio.Invalido("Fornecedor inexistente ou inativo.");
    var delta = request.Tipo == "entrada" ? request.Quantidade :
        -request.Quantidade;
    var updated = await db.Produtos.Where(p => p.Id == request.ProdutoId &&
        p.Saldo + delta >= 0 && p.Saldo <= 999999999 - Math.Max(delta, 0))
        .ExecuteUpdateAsync(set => set.SetProperty(p => p.Saldo, p => p.Saldo +
            delta));
    if (updated == 0) throw ErroNegocio.Conflito("Saldo insuficiente ou limite de
        saldo excedido.");
    var movement = new Movimentacao { ProdutoId = request.ProdutoId, Quantidade =
        delta, Motivo = request.Motivo.Trim(), CriadaEmUtc = DateTime.UtcNow,
            UsuarioId = usuarioId, FornecedorId = request.FornecedorId };
    db.Movimentacoes.Add(movement);
    await db.SaveChangesAsync();
    await transaction.CommitAsync();
    // [...]
```

Fonte: elaborado pelos autores (2026).

6 APLICAÇÃO MOBILE

O aplicativo foi desenvolvido em Flutter, framework do Google que compila um único código Dart para Android, web e outras plataformas (Google, 2026a). Atende técnicos e almoxarife, que movimentam materiais longe do computador e muitas vezes sem sinal.

6.1 Telas principais

O Quadro 8 descreve as telas principais e a função de cada uma.

Quadro 8 – Telas principais do aplicativo

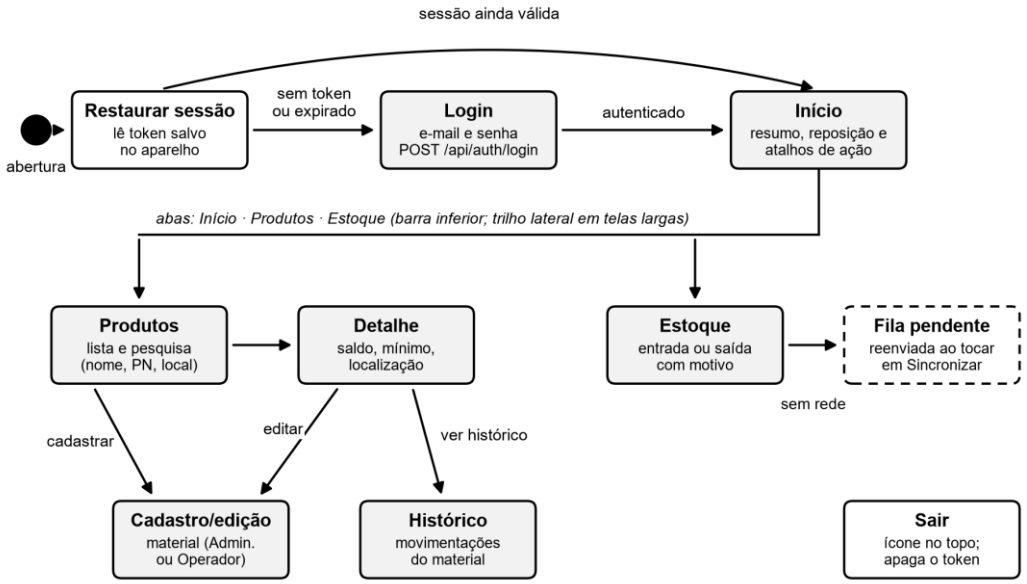
Tela	Função
Login	Entrada com e-mail e senha; mensagens de erro claras e anunciadas
Início	Resumo do estoque, itens para repor e atalhos para nova movimentação e consulta
Produtos	Lista com pesquisa por nome, part number, categoria ou localização; detalhe com saldo, mínimo e unidade
Cadastro/edição	Formulário de material com validação de part number, unidade e estoque mínimo (perfis Administrador e Operador)
Histórico	Movimentações do material, com data, quantidade, motivo e responsável
Estoque	Registro de entrada ou saída com motivo, filtro de reposição, contador de pendências e botão Sincronizar

Fonte: elaborado pelos autores (2026).

6.2 Fluxo de navegação

Ao abrir, o aplicativo tenta restaurar a sessão salva; sem token válido, exibe o login. Autenticado, o usuário navega pelas abas Início, Produtos e Estoque (Figura 4).

Figura 4 – Fluxo de navegação do aplicativo



Fonte: elaborado pelos autores (2026).

6.3 Autenticação

O login chama POST /api/auth/login e recebe o token e os dados do usuário. A sessão é guardada por meio de uma interface de armazenamento com duas implementações, escolhidas por importação condicional: arquivo no diretório privado do aplicativo, no Android, e armazenamento local, no navegador. O aplicativo não guarda senha nem credenciais embutidas, e o perfil recebido controla quais botões de edição aparecem.

6.4 Sincronização de dados

Para funcionar com conexão instável, o aplicativo mantém uma cópia local do catálogo e uma fila de movimentações pendentes, gravadas em arquivo JSON. Sem rede, a movimentação entra na fila e o contador de pendências é exibido. Ao tocar em Sincronizar, ou ao reabrir o aplicativo com conexão, a fila é enviada em ordem cronológica e cada item só sai dela após a confirmação da API; se o servidor recusar um item, por exemplo por saldo insuficiente, o usuário é avisado, pois a regra de saldo é sempre decidida no servidor (Quadro 9).

Quadro 9 – Envio da fila de pendências (product_catalog.dart)

```
Future<void> sync() async {
  if (api == null) return;
  final remaining = <StockMovement>[];
  for (final movement in List.of(_pending)) {
    try {
      await api!.createMovement(movement);
    } on ApiException catch (error) {
      if (error.statusCode == 409) {
        _conflicts.add(movement);
        continue;
      }
      remaining.add(movement);
      break;
    } catch (_) {
      remaining.add(movement);
      break;
    }
  }
  _pending
    ..clear()
    ..addAll(remaining);
  await _persist();
  // [...]
```

Fonte: elaborado pelos autores (2026).

6.5 Integração com as APIs

A classe ApiClient concentra a comunicação: usa o pacote http, acrescenta o cabeçalho Authorization, converte erros em exceções com a mensagem da API e, diante de uma resposta 401, encerra a sessão. O endereço do servidor é informado na compilação (parâmetro API_URL), e os modelos Product e StockMovement espelham os contratos JSON.

6.6 Ciclo de vida no Android

No Android, a tela é uma Activity cujo ciclo de vida é controlado pelo sistema por meio dos métodos onCreate, onStart, onResume, onPause, onStop e onDestroy (Google, 2026b). Como o sistema pode encerrar um aplicativo em segundo plano, o PIM Estoque grava cada alteração imediatamente, com arquivo temporário e renomeação (Quadro 10).

Quadro 10 – Ciclo de vida da Activity e comportamento do aplicativo

Callback	Momento	Comportamento do PIM Estoque
onCreate	Criação da Activity	Inicia o Flutter, restaura sessão e catálogo salvos
onStart / onResume	Tela visível e interativa	Exibe dados locais e permite sincronizar pendências
onPause / onStop	Outra tela à frente ou app em segundo plano	Nada a salvar: dados e fila já foram gravados a cada operação
onDestroy	Encerramento ou rotação	Ao reabrir, o estado é reconstruído a partir dos arquivos

Fonte: elaborado pelos autores (2026).

7 ARQUITETURA DE SOFTWARE

Martin (2017) defende que uma boa arquitetura mantém as regras de negócio independentes dos detalhes, como frameworks, bancos e interfaces, e permite que as partes evoluam separadamente. Essa ideia orientou a organização do PIM Estoque, alinhada ao desenvolvimento web e mobile descrito nas seções anteriores.

7.1 Organização em camadas

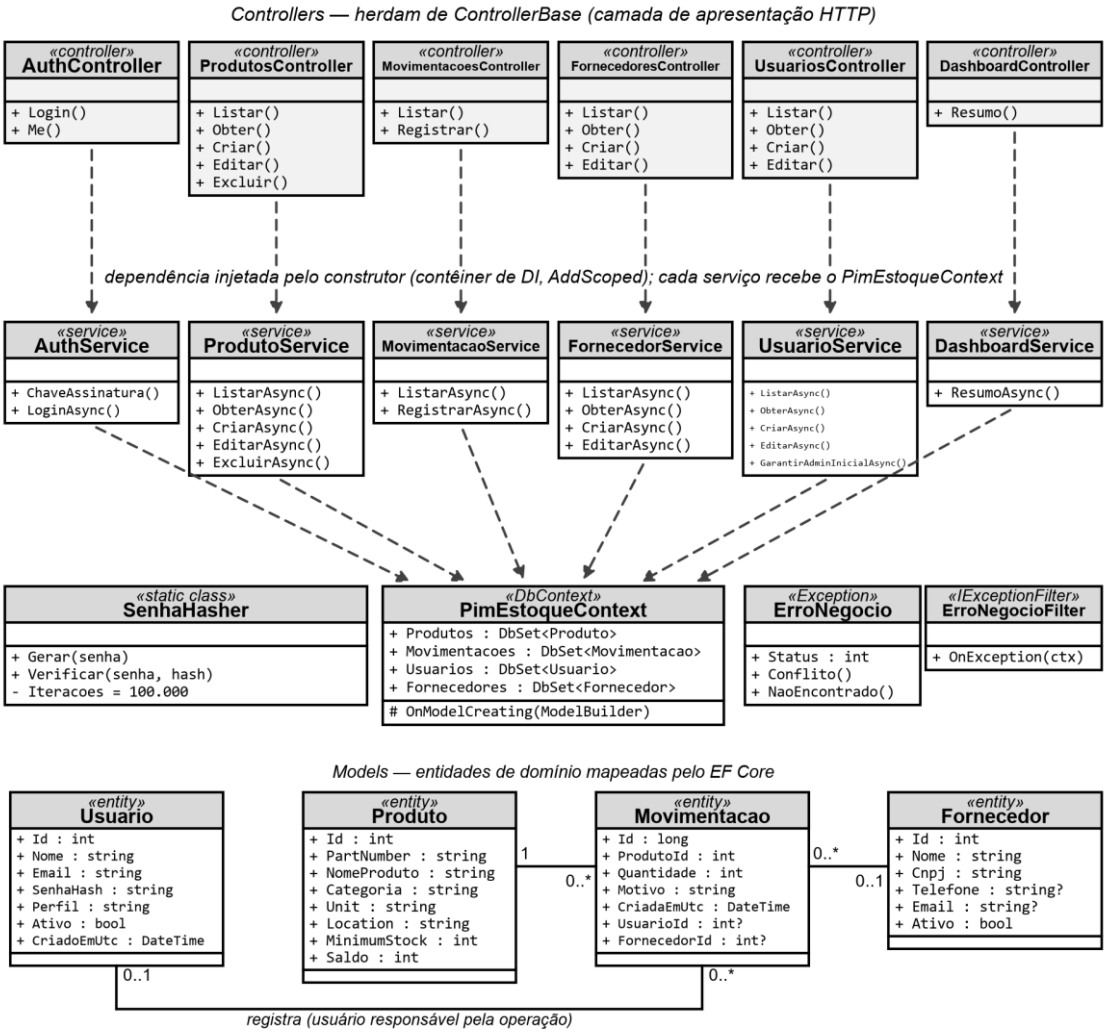
O back-end segue o padrão de camadas descrito por Fowler (2002), separando apresentação, domínio e acesso a dados:

a) Controllers (apresentação): recebem a requisição HTTP, aplicam a autorização por perfil e devolvem códigos de status, sem conter regras de negócio;
b) Services (negócio): concentram as regras, como saldo não negativo, unidade bloqueada após movimentações e impedimento de que um administrador remova o próprio acesso;
c) Data (persistência): o PimEstoqueContext mapeia as entidades para as tabelas e controla transações;
d) Models e Contracts: entidades de domínio e objetos de entrada e saída (DTOs), que impedem a exposição direta das entidades, como o hash de senha.

7.2 Orientação a objetos

A Figura 5 apresenta o diagrama de classes simplificado do back-end, gerado a partir do código. Os pilares da orientação a objetos aparecem da seguinte forma:

Figura 5 – Diagrama de classes do back-end



Fonte: elaborado pelos autores (2026).

a) classes e objetos: cada conceito do domínio (Produto, Movimentacao, Usuario, Fornecedor) é uma classe com propriedades tipadas;

b) encapsulamento: os serviços recebem o contexto pelo construtor e o mantêm privado; detalhes como o número de iterações do hash são constantes privadas da classe SenhaHasher, acessíveis apenas pelos métodos Gerar e Verificar;

c) herança: os controllers herdam de ControllerBase, o contexto herda de DbContext e a exceção de negócio herda de Exception, reaproveitando comportamento do framework;

d) interfaces e polimorfismo: os DTOs implementam IValidatableObject para validações próprias, o filtro de erros implementa IExceptionFilter e, no

aplicativo, a interface AuthStorage possui implementações distintas para Android e navegador;

e) injeção de dependência: os serviços são registrados no contêiner do ASP.NET Core com tempo de vida por requisição e entregues aos controllers pelo construtor, o que reduz o acoplamento e facilita os testes.

7.3 Modularização

A solução é dividida em módulos versionados no mesmo repositório: a API na raiz, organizada por camadas; o painel em wwwroot; o aplicativo em mobile, organizado por funcionalidade (core para cliente HTTP, autenticação e tema; features para início e produtos); os scripts numerados do banco em database; e os testes automatizados. Sommerville (2011) observa que módulos coesos e pouco acoplados reduzem o custo de manutenção, o que se confirmou quando a integração do aplicativo com a API se concentrou na camada de catálogo e no cliente HTTP. O código-fonte integral dos módulos está nos apêndices: API no Apêndice B, painel web no Apêndice C e aplicativo no Apêndice D.

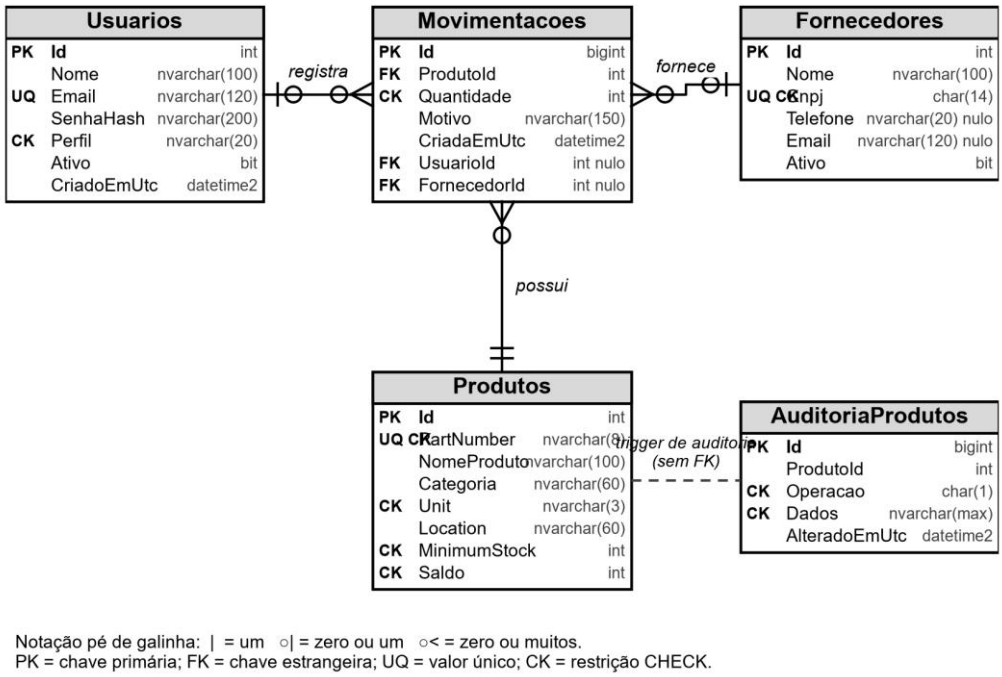
8 BANCO DE DADOS

O banco de dados do PIM Estoque foi projetado em SQL Server, ambiente usado nas aulas práticas de Banco de Dados (Bueno, 2026), e é o único repositório de dados compartilhado pela API, pelo painel e pelo aplicativo. O projeto seguiu as etapas clássicas de modelo conceitual, lógico e físico (Elmasri; Navathe, 2011).

8.1 Modelo entidade-relacionamento

A Figura 6 apresenta o MER em notação pé de galinha. Cada produto tem zero ou muitas movimentações, que podem estar ligadas ao usuário que as registrou e, nas entradas, a um fornecedor. A auditoria guarda o produto sem chave estrangeira, para sobreviver à exclusão do material.

Figura 6 – Modelo entidade-relacionamento do PIM Estoque



Fonte: elaborado pelos autores (2026).

8.2 Modelo lógico

O modelo lógico, resumido no Quadro 11, está na terceira forma normal: cada tabela trata de um único assunto e os atributos dependem apenas da chave

primária. A única redundância controlada é a coluna Saldo, mantida em Produtos por desempenho e atualizada na mesma transação que grava a movimentação.

Quadro 11 – Modelo lógico: tabelas, chaves e relacionamentos

Tabela	Chave primária	Chaves estrangeiras e únicas	Finalidade
Usuarios	Id	UQ Email	Quem opera o sistema e seu perfil
Fornecedores	Id	UQ Cnpj	Origem das compras
Produtos	Id	UQ PartNumber	Materiais, localização, mínimo e saldo
Movimentacoes	Id	FK ProdutoId, FK UsuarioId (nulo), FK FornecedorId (nulo)	Histórico de entradas (+) e saídas (−)
AuditoriaProdutos	Id	ProdutoId (vínculo lógico)	Trilha de alterações do cadastro em JSON

Fonte: elaborado pelos autores (2026).

8.3 Modelo físico

No modelo físico foram definidos tipos, restrições e índices. As quantidades são inteiras e o saldo tem restrição CHECK entre 0 e 999.999.999; o part number deve ter oito dígitos com prefixo 47, 48, 84 ou 86; a unidade é limitada a UN, PC, PAR, CX, PCT ou RL; o CNPJ tem 14 dígitos; e o perfil aceita apenas Administrador, Operador ou Consulta. Há índices para pesquisa por nome, reposição (índice filtrado), histórico, extrato por período e movimentações do dia, e todos os scripts são reexecutáveis.

8.4 Procedures e view

Foram criadas quatro procedures: sp_RegistrarMovimentacao (regra de movimentação), sp_ProdutosParaReposicao (itens no mínimo), sp_ResumoEstoque (indicadores do painel) e sp_ExtratoProduto (extrato por período com saldo acumulado), além da view vw_SaldoPorCategoria. O Quadro 12 mostra o núcleo da primeira, que usa transação e bloqueio de atualização para que duas saídas simultâneas não consumam o mesmo saldo.

Quadro 12 – Trecho da procedure sp_RegistrarMovimentacao (003_procedures.sql)

```
CREATE OR ALTER PROCEDURE dbo.sp_RegistrarMovimentacao
    -- [...]
    DECLARE @Delta int = CASE @Tipo WHEN 'entrada' THEN @Quantidade ELSE
        -@Quantidade END;
        DECLARE @Saldo int, @MovimentacaoId bigint;

        BEGIN TRY
            BEGIN TRANSACTION;

            SELECT @Saldo = Saldo
            FROM dbo.Produtos WITH (UPDLOCK, HOLDLOCK)
            WHERE Id = @ProdutoId;

            IF @Saldo IS NULL
                THROW 50005, N'Produto não encontrado.', 1;
            IF @Saldo + CAST(@Delta AS bigint) < 0
                THROW 50002, N'Saldo insuficiente para a saída.', 1;
            IF @Saldo + CAST(@Delta AS bigint) > 999999999
                THROW 50006, N'Saldo ultrapassaria o limite de 999999999.', 1;

            UPDATE dbo.Produtos SET Saldo = Saldo + @Delta WHERE Id = @ProdutoId;

            INSERT dbo.Movimentacoes (ProdutoId, Quantidade, Motivo, CriadaEmUtc,
                UsuarioId, FornecedorId)
            VALUES (@ProdutoId, @Delta, LTRIM(RTRIM(@Motivo)), SYSUTCDATETIME(),
                @UsuarioId, @FornecedorId);
            SET @MovimentacaoId = SCOPE_IDENTITY();

            COMMIT;
        END TRY
        BEGIN CATCH
            IF @@TRANCOUNT > 0 ROLLBACK;
            THROW;
        END CATCH;
    -- [...]
```

Fonte: elaborado pelos autores (2026).

8.5 Triggers

A trigger trg_Produtos_Auditoria grava em AuditoriaProdutos, em JSON, cada inserção, alteração ou exclusão de produto, sem alterar saldo. A trg_Movimentacoes_Imutavel, do tipo INSTEAD OF, impede editar ou excluir o histórico, corrigido apenas por estorno (Quadro 13).

Quadro 13 – Trigger de imutabilidade do histórico (002_triggers.sql)

```
CREATE OR ALTER TRIGGER dbo.trg_Movimentacoes_Imutavel
ON dbo.Movimentacoes
INSTEAD OF UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM deleted)
        THROW 50010, N'Movimentações não podem ser alteradas nem excluídas.
            Registre um estorno.', 1;
END;
```

Fonte: elaborado pelos autores (2026).

8.6 Script completo do banco

Os objetos estão em quatro scripts numerados: 001_schema.sql (tabelas, restrições e índices), 002_triggers.sql, 003_procedures.sql (view e procedures) e

004_seed.sql (dados de demonstração: 3 fornecedores, 12 produtos, 22 movimentações), consolidados em database/entrega/pim_estoque_completo.sql, que cria o banco PimEstoque e executa os quatro na ordem. Esse script completo, usado pela API, pelo painel e pelo aplicativo, está no Apêndice A.

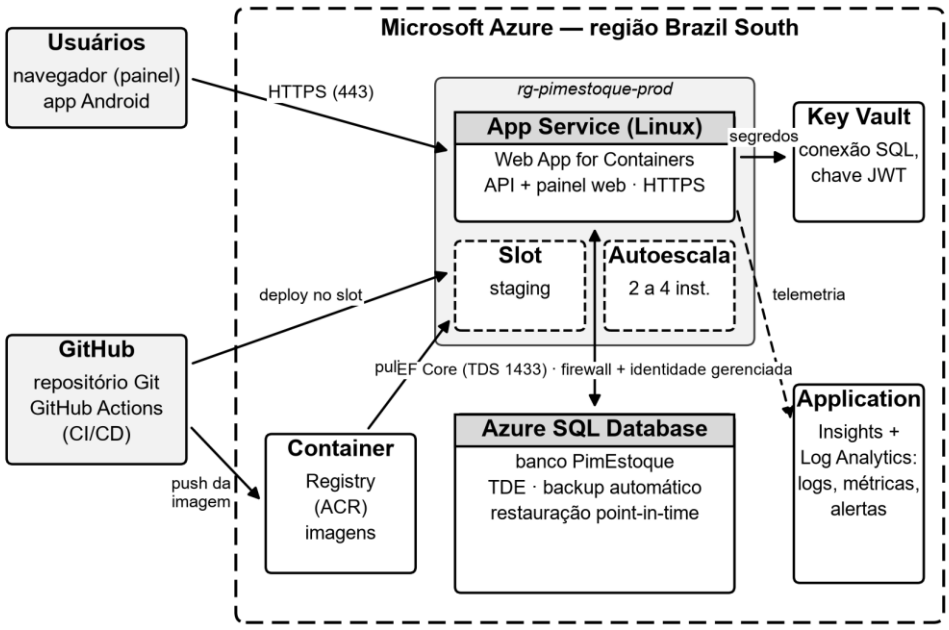
9 INFRAESTRUTURA EM NUVEM E DEVOPS

Mell e Grance (2011) definem computação em nuvem como um modelo de acesso sob demanda, pela rede, a um conjunto compartilhado de recursos configuráveis, provisionados e liberados rapidamente. Para uma pequena empresa e para um produto SaaS em início de operação, esse modelo troca investimento em servidores por custo mensal proporcional ao uso. Esta seção apresenta o plano de execução da infraestrutura.

9.1 Arquitetura em nuvem

Foi escolhido o Microsoft Azure, pela integração com .NET e SQL Server e pela região Brazil South, que mantém os dados no país. A arquitetura (Figura 7) usa serviços PaaS, em que o provedor administra sistema operacional, atualizações e alta disponibilidade, e segue os pilares do Azure Well-Architected Framework (Microsoft, 2026a).

Figura 7 – Arquitetura de implantação no Microsoft Azure



Fonte: elaborado pelos autores (2026).

9.2 Serviços utilizados

O Quadro 14 descreve a função de cada serviço no plano.

Quadro 14 – Serviços de nuvem do plano de implantação

Serviço	Função no PIM Estoque
App Service (containers)	API e painel com HTTPS, slots e autoescala
Azure SQL Database	Banco com backup, restauração e criptografia (TDE)
Container Registry	Imagens Docker versionadas pelo commit
Key Vault	Conexão e chave JWT, lidas por identidade gerenciada
Application Insights	Logs, métricas, disponibilidade e alertas
GitHub Actions	Esteira de integração e entrega contínuas

Fonte: elaborado pelos autores (2026).

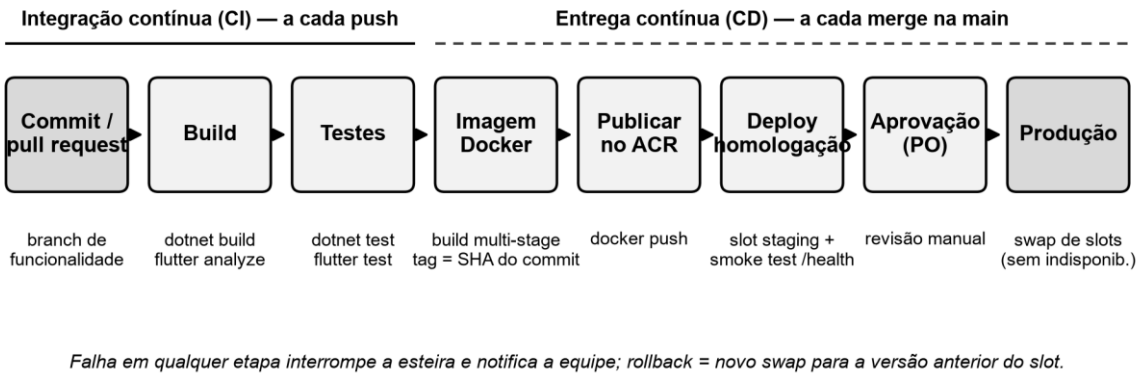
9.3 Containers

A API é empacotada em uma imagem Docker de dois estágios: o SDK do .NET compila e publica, e a imagem final contém só o runtime, ficando menor e com menos superfície de ataque. Como recomenda Peres (2026, p. 193), nenhum segredo vai para a imagem: chave JWT e conexão chegam por variáveis de ambiente, e o processo roda sem privilégios. O docker-compose.yml sobe o SQL Server 2022, um serviço que aplica os scripts do banco e a API publicada localmente, reproduzindo o ambiente com um único comando.

9.4 Pipeline CI/CD

Kim et al. (2016) mostram que automatizar build, testes e implantação torna as entregas mais rápidas e menos arriscadas, e Humble e Farley (2010) acrescentam que todo commit deve gerar um artefato candidato a produção. O repositório já possui fluxos no GitHub Actions que testam a API e o aplicativo e publicam a versão web de demonstração. O plano completo (Figura 8) acrescenta imagem, registro e homologação, com aprovação do Product Owner antes da troca de slots para produção.

Figura 8 – Pipeline de integração e entrega contínuas



Fonte: elaborado pelos autores (2026).

9.5 Monitoramento

Segundo Peres (2026, p. 171), “Observabilidade é a capacidade de inferir o estado interno de um sistema a partir dos sinais que ele produz durante sua execução”. O plano combina logs do ASP.NET Core no Application Insights, métricas de tempo de resposta, erros, CPU e memória, teste de disponibilidade no endpoint /health a cada cinco minutos e alertas quando as respostas 5xx passarem de 2% ou o banco ultrapassar 80% de uso, sem gravar dados pessoais ou tokens em logs.

9.6 Escalabilidade

Como o JWT dispensa sessão no servidor, a API escala horizontalmente: a autoescala mantém de duas a quatro instâncias, guiada pela CPU (acima de 70% adiciona, abaixo de 30% remove). O banco escala verticalmente, sem mudar a aplicação.

9.7 Segurança da infraestrutura

A segurança combina HTTPS obrigatório, segredos no Key Vault por identidade gerenciada, firewall do banco restrito ao App Service, criptografia em repouso e em trânsito, backups testados, acesso por funções com autenticação multifator e varredura de dependências, atendendo à LGPD (Brasil, 2018).

9.8 Implantação realizada

Para validar o plano antes da contratação do Azure, a solução foi publicada em uma instância de computação da Oracle Cloud Infrastructure, na região de São Paulo, e está disponível em https://estoque.vinnisantos.com.br. A instância usa processadores ARM, para os quais a imagem do SQL Server 2022 não é distribuída; por isso, o banco roda no Azure SQL Edge, versão do motor do SQL Server compilada para ARM64, que executa os mesmos scripts T-SQL do Apêndice A. A bateria de validação do banco, com procedures, triggers, auditoria e restrições, passou integralmente nesse ambiente.

A API e o banco rodam em containers descritos no arquivo docker-compose.prod.yml. O banco não expõe porta à internet e é acessado pela API com um usuário próprio, restrito a leitura, escrita e execução de procedures, sem uso da conta administrativa. Um servidor Nginx recebe as requisições, aplica HTTPS com certificado Let's Encrypt renovado automaticamente e as encaminha à API; senhas e chave JWT ficam em um arquivo de ambiente fora do repositório. O aplicativo Android aponta para esse endereço por padrão. Como a Microsoft encerrou o suporte ao SQL Edge em 2025, essa implantação serve à demonstração do projeto; para a operação definitiva, a recomendação continua sendo o Azure SQL Database descrito na seção 9.1.

10 GERENCIAMENTO ÁGIL DO PROJETO

O projeto usou Scrum com um quadro Kanban. O Scrum é um framework leve que gera valor por soluções adaptativas, em ciclos curtos de inspeção e adaptação chamados sprints (Schwaber; Sutherland, 2020); o guia PMBOK recomenda ciclos adaptativos quando os requisitos são incertos (Project Management Institute, 2017), como neste projeto.

10.1 Papéis e eventos

O Quadro 15 relaciona os papéis aos integrantes. Os eventos foram adaptados à rotina de quem estuda e trabalha: planejamento às segundas-feiras, reunião diária assíncrona por mensagem e revisão e retrospectiva na última sexta-feira da sprint.

Quadro 15 – Papéis do Scrum na equipe

Papel	Integrante	Responsabilidades no projeto
Product Owner	Vinnicius Gabriel Matos dos Santos	Prioriza o backlog, representa a Fixar e aceita as entregas; também atua no back-end
Scrum Master	Hendryo Batista Sarmento	Facilita eventos, remove impedimentos e cuida do banco de dados
Desenvolvedor	Lucas Alcântara Leal	Aplicativo Flutter e testes de interface
Desenvolvedor	Matheus Terci	Painel web, containers e pipeline

Fonte: elaborado pelos autores (2026).

10.2 Product Backlog

O Product Backlog (Quadro 16) foi escrito em histórias de usuário, priorizado pelo valor para a Fixar e estimado em pontos de Fibonacci por planning poker.

Quadro 16 – Product Backlog

ID	História de usuário	Prioridade	Pontos	Sprint
US01	Como administrador, quero entrar com meu perfil de acesso.	Alta	5	S4
US02	Como almoxarife, quero cadastrar materiais.	Alta	5	S3
US03	Como almoxarife, quero registrar entradas.	Alta	8	S3
US04	Como técnico, quero retirar sem exceder o saldo.	Alta	8	S3
US05	Como comprador, quero ver os itens para repor.	Alta	3	S3
US06	Como gestor, quero indicadores por categoria.	Média	5	S4
US07	Como administrador, quero gerenciar usuários.	Média	5	S4
US08	Como comprador, quero cadastrar fornecedores.	Média	3	S4
US09	Como diretor, quero histórico imutável e auditado.	Média	5	S2
US10	Como técnico, quero entrar no aplicativo.	Alta	5	S6
US11	Como técnico, quero movimentar sem internet.	Alta	8	S6
US12	Como equipe, quero CI/CD com containers.	Média	8	S5

ID	História de usuário	Prioridade	Pontos	Sprint
US13	Como usuário de leitor de tela, quero usar o painel.	Alta	3	S5
US14	Como gestor, quero relatórios por período.	Baixa	5	Futura
US15	Como almoxarife, quero fazer inventário físico.	Baixa	8	Futura

Fonte: elaborado pelos autores (2026).

10.3 Sprint Backlog

O Quadro 17 mostra o Sprint Backlog da Sprint 6, cuja meta é integrar o aplicativo à API com login e sincronização sem conexão; as tarefas somam 49 horas, e a situação refere-se a 10/11/2026.

Quadro 17 – Sprint Backlog da Sprint 6

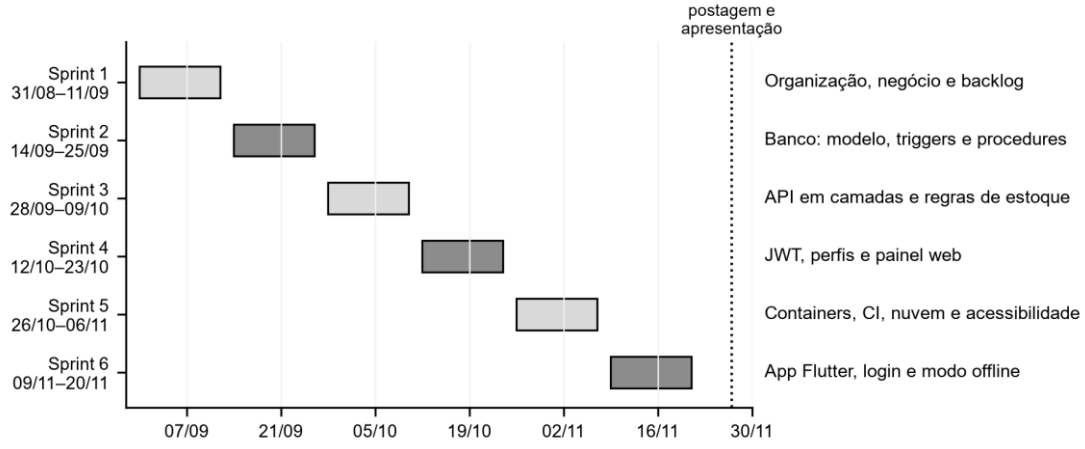
Item	Tarefa	Responsável	Horas	Situação
US10	Tela de login e restauração da sessão salva	Lucas	6	Concluída
US10	Cliente HTTP com token e tratamento de 401	Lucas	5	Concluída
US10	Armazenamento da sessão (Android e web)	Vinnicius	4	Concluída
US11	Fila local de movimentações pendentes	Lucas	8	Em andamento
US11	Envio da fila e tratamento de recusas	Vinnicius	6	Em andamento
US11	Testes de widget e da fila de sincronização	Hendryo	5	A fazer
—	Build do APK e publicação da versão web	Matheus	3	Em revisão
—	Documento acadêmico e apresentação	Vinnicius	12	Em andamento

Fonte: elaborado pelos autores (2026).

10.4 Cronograma das sprints

O desenvolvimento foi dividido em seis sprints de duas semanas, entre 31 de agosto e 20 de novembro de 2026, seguidas da preparação da postagem e da apresentação. A Figura 9 mostra o período e a meta de cada sprint. A velocidade média foi de 16 pontos por sprint (S1: 13, S2: 18, S3: 24, S4: 18, S5: 11, S6: 13).

Figura 9 – Cronograma das sprints

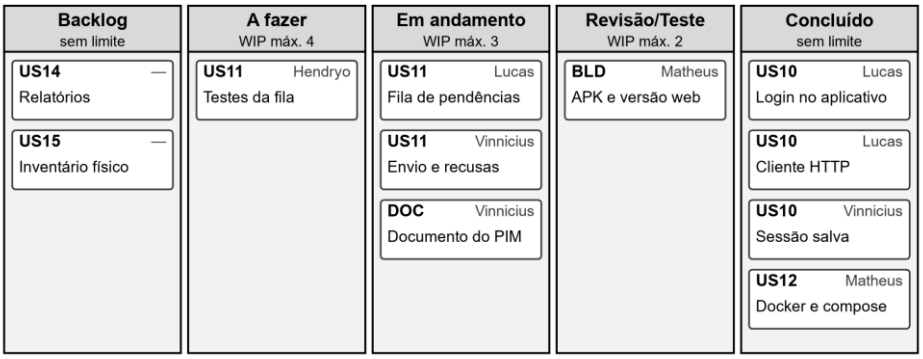


Fonte: elaborado pelos autores (2026).

10.5 Quadro Kanban

O Kanban torna o trabalho visível e limita as tarefas simultâneas, favorecendo a conclusão em vez do acúmulo de itens iniciados (Anderson, 2010). O quadro da equipe (Figura 10) tem cinco colunas, com limites de WIP nas intermediárias.

Figura 10 – Quadro Kanban da equipe em 10/11/2026



Fonte: elaborado pelos autores (2026).

10.6 Definição das entregas

Cada sprint entrega um incremento utilizável. Um item está pronto quando atende à Definition of Done (DoD) da equipe:

a) código revisado por outro integrante e testes aprovados no GitHub Actions;

b) regras validadas na API e, quando houver, reforçadas no banco;

c) telas verificadas por teclado ou leitor de tela e com texto ampliado;

d) documentação atualizada;

e) aceite do Product Owner na revisão.

As entregas finais são a API e o painel em container, o APK e a versão web do aplicativo, o script do banco, o plano de nuvem e este documento.

11 CONCLUSÃO

O PIM IV teve como objetivo desenvolver uma solução tecnológica integrada para apoiar a transformação digital da Fixar Manutenção, a partir do estudo iniciado no PIM III. O objetivo foi atingido com o PIM Estoque, que substitui planilhas e papel por um controle centralizado de materiais, acessível pelo navegador e pelo celular.

Cada etapa contribuiu com uma parte da solução. A caracterização da organização mostrou que os problemas do almoxarifado tinham origem no registro tardio da informação. O planejamento empreendedor transformou a necessidade de um cliente em um produto por assinatura, cujo ponto de equilíbrio, estimado em cerca de 23 empresas, é alcançável. A etapa de responsabilidade social incorporou acessibilidade e combate à discriminação como requisitos verificáveis. As aplicações web e mobile compartilham uma única API, protegida por JWT e perfis de acesso, e o aplicativo continua útil sem conexão graças à fila de sincronização. A arquitetura em camadas e orientada a objetos manteve as regras de negócio em um só lugar; o banco de dados garante, por restrições, procedures e triggers, que o saldo nunca fique negativo e que o histórico não seja alterado. O plano de nuvem e a esteira de CI/CD mostram como levar a solução à produção com segurança e escalabilidade, e o Scrum com Kanban permitiu entregas incrementais em seis sprints.

Entre as limitações, destacam-se a ausência de testes com usuários reais da empresa, a operação ainda restrita a uma única empresa e a falta de relatórios exportáveis. Como trabalhos futuros, propõem-se: inventário físico com ajustes justificados e estornos; relatórios por período em PDF e planilha; requisições de material por obra; leitura de código de barras pelo aplicativo; separação de dados por empresa para a oferta SaaS; e um piloto acompanhado na Fixar, com medição da redução de compras emergenciais.

Por fim, o projeto evidenciou o valor da integração entre as disciplinas do semestre: decisões de negócio, de interface, de arquitetura, de dados e de infraestrutura influenciam-se mutuamente, e somente tratadas em conjunto produzem uma solução coerente, útil e inclusiva.

REFERÊNCIAS

ALMEIDA, Silvio. **Racismo estrutural**. São Paulo: Pólen, 2019. (Feminismos plurais).

ANDERSON, David J. **Kanban**: successful evolutionary change for your technology business. Sequim: Blue Hole Press, 2010.

BIBLIOTECA DA UNIVERSIDADE PAULISTA - UNIP. **Guia de normalização para apresentação de trabalhos acadêmicos da Universidade Paulista**: ABNT. São Paulo: UNIP, 2025. Disponível em: https://unip.br/servicos/biblioteca/guia.aspx. Acesso em: 10 nov. 2026.

BRASIL. Lei nº 7.716, de 5 de janeiro de 1989. Define os crimes resultantes de preconceito de raça ou de cor. **Diário Oficial da União**: seção 1, Brasília, DF, 6 jan. 1989. Disponível em: https://www.planalto.gov.br/ccivil_03/leis/l7716.htm. Acesso em: 10 nov. 2026.

BRASIL. Lei nº 10.639, de 9 de janeiro de 2003. Altera a Lei nº 9.394, de 20 de dezembro de 1996, que estabelece as diretrizes e bases da educação nacional, para incluir no currículo oficial da Rede de Ensino a obrigatoriedade da temática “História e Cultura Afro-Brasileira”, e dá outras providências. **Diário Oficial da União**: seção 1, Brasília, DF, 10 jan. 2003. Disponível em: https://www.planalto.gov.br/ccivil_03/leis/2003/l10.639.htm. Acesso em: 10 nov. 2026.

BRASIL. Lei nº 12.288, de 20 de julho de 2010. Institui o Estatuto da Igualdade Racial. **Diário Oficial da União**: seção 1, Brasília, DF, 21 jul. 2010. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2007-2010/2010/lei/l12288.htm. Acesso em: 10 nov. 2026.

BRASIL. Lei nº 13.146, de 6 de julho de 2015. Institui a Lei Brasileira de Inclusão da Pessoa com Deficiência (Estatuto da Pessoa com Deficiência). **Diário Oficial da União**: seção 1, Brasília, DF, 7 jul. 2015. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2015/lei/l13146.htm. Acesso em: 10 nov. 2026.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). **Diário Oficial da União**: seção 1, Brasília, DF, 15 ago. 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 10 nov. 2026.

BUENO, Fernando A. G. **Banco de dados II**: aulas práticas de SQL Server. Sorocaba: UNIP, 2026. Material de aula.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 6. ed. São Paulo: Pearson Addison Wesley, 2011.

FIELDING, Roy Thomas. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Information and Computer Science) – University of California, Irvine, 2000.

FOWLER, Martin. **Patterns of enterprise application architecture**. Boston: Addison-Wesley, 2002.

GOOGLE. **Flutter documentation**. [S. l.]: Google, 2026a. Disponível em: https://docs.flutter.dev/. Acesso em: 10 nov. 2026.

GOOGLE. **The activity lifecycle**. Android Developers. [S. l.]: Google, 2026b. Disponível em: https://developer.android.com/guide/components/activities/activity-lifecycle. Acesso em: 10 nov. 2026.

HUMBLE, Jez; FARLEY, David. **Continuous delivery**: reliable software releases through build, test, and deployment automation. Upper Saddle River: Addison-Wesley, 2010.

JONES, Michael B.; BRADLEY, John; SAKIMURA, Nat. **RFC 7519**: JSON Web Token (JWT). [S. l.]: Internet Engineering Task Force, 2015. Disponível em: https://www.rfc-editor.org/rfc/rfc7519. Acesso em: 10 nov. 2026.

KIM, Gene; HUMBLE, Jez; DEBOIS, Patrick; WILLIS, John. **The DevOps handbook**: how to create world-class agility, reliability, and security in technology organizations. Portland: IT Revolution Press, 2016.

MARTIN, Robert C. **Clean architecture**: a craftsman's guide to software structure and design. Boston: Prentice Hall, 2017.

MELL, Peter; GRANCE, Timothy. **The NIST definition of cloud computing**. Gaithersburg: National Institute of Standards and Technology, 2011. (Special Publication 800-145).

MICROSOFT. **Azure Well-Architected Framework**. [S. l.]: Microsoft Learn, 2026a. Disponível em: https://learn.microsoft.com/azure/well-architected/. Acesso em: 10 nov. 2026.

MICROSOFT. **Calculadora de preços do Azure**. [S. l.]: Microsoft, 2026b. Disponível em: https://azure.microsoft.com/pt-br/pricing/calculator/. Acesso em: 10 nov. 2026.

MICROSOFT. **Entity Framework Core**. [S. l.]: Microsoft Learn, 2026c. Disponível em: https://learn.microsoft.com/ef/core/. Acesso em: 10 nov. 2026.

MICROSOFT. **Overview of ASP.NET Core authentication**. [S. l.]: Microsoft Learn, 2026d. Disponível em: https://learn.microsoft.com/aspnet/core/security/authentication/. Acesso em: 10 nov. 2026.

OSTERWALDER, Alexander; PIGNEUR, Yves. **Business model generation**: inovação em modelos de negócios. Rio de Janeiro: Alta Books, 2011.

OWASP FOUNDATION. **OWASP Top 10:2021**. [S. l.]: OWASP Foundation, 2021. Disponível em: https://owasp.org/Top10/. Acesso em: 10 nov. 2026.

PERES, Tarcísio de Souza. **Desenvolvimento web com .NET**. São Paulo: Editora Sol, 2026.

PROJECT MANAGEMENT INSTITUTE. **Um guia do conhecimento em gerenciamento de projetos (guia PMBOK)**. 6. ed. Newtown Square: Project Management Institute, 2017.

SCHWABER, Ken; SUTHERLAND, Jeff. **O guia do Scrum**: o guia definitivo para o Scrum: as regras do jogo. [S. l.]: Scrum.org, 2020. Disponível em: https://scrumguides.org/. Acesso em: 10 nov. 2026.

SOMMERVILLE, Ian. **Engenharia de software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

W3C. **Web Content Accessibility Guidelines (WCAG) 2.1**. [S. l.]: World Wide Web Consortium, 2018. Disponível em: https://www.w3.org/TR/WCAG21/. Acesso em: 10 nov. 2026.

APÊNDICE A – SCRIPT SQL COMPLETO DO BANCO DE DADOS

Script consolidado utilizado pela API, pelo painel web e pelo aplicativo, disponível no repositório em database/entrega/pim_estoque_completo.sql.

```
-- PIM Estoque — script completo do banco (gerado por database/entrega/gerar.ps1; não editar à mão).
-- Cria o banco PimEstoque e executa 001_schema, 002_triggers, 003_procedures e 004_seed.
-- Usado pela API ASP.NET Core, pelo painel web e pelo app mobile. Gerado em 2026-11-10.
IF DB_ID(N'PimEstoque') IS NULL CREATE DATABASE PimEstoque;
GO
USE PimEstoque;
GO
-- ============================== 001_schema.sql ==============================
-- PIM Estoque — 001: tabelas, restrições e índices (SQL Server, schema dbo).
-- Pode ser executado mais de uma vez: cada objeto só é criado se ainda não existir.
SET ANSI_NULLS ON;
SET QUOTED_IDENTIFIER ON; -- obrigatório para índices filtrados; o sqlcmd clássico vem com OFF
GO

IF OBJECT_ID(N'dbo.Usuarios', N'U') IS NULL
CREATE TABLE dbo.Usuarios (
    Id          int IDENTITY(1,1) NOT NULL,
    Nome        nvarchar(100) NOT NULL,
    Email       nvarchar(120) NOT NULL,
    SenhaHash   nvarchar(200) NOT NULL,
    Perfil      nvarchar(20)  NOT NULL,
    Ativo       bit           NOT NULL CONSTRAINT DF_Usuarios_Ativo DEFAULT 1,
    CriadoEmUtc datetime2     NOT NULL CONSTRAINT DF_Usuarios_CriadoEmUtc DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_Usuarios PRIMARY KEY (Id),
    CONSTRAINT UQ_Usuarios_Email UNIQUE (Email),
    CONSTRAINT CK_Usuarios_Perfil CHECK (Perfil IN ('Administrador','Operador','Consulta'))
);
GO

IF OBJECT_ID(N'dbo.Fornecedores', N'U') IS NULL
CREATE TABLE dbo.Fornecedores (
    Id       int IDENTITY(1,1) NOT NULL,
    Nome     nvarchar(100) NOT NULL,
    Cnpj     char(14)      NOT NULL,
    Telefone nvarchar(20)  NULL,
    Email    nvarchar(120) NULL,
    Ativo    bit           NOT NULL CONSTRAINT DF_Fornecedores_Ativo DEFAULT 1,
    CONSTRAINT PK_Fornecedores PRIMARY KEY (Id),
    CONSTRAINT UQ_Fornecedores_Cnpj UNIQUE (Cnpj),
    -- char(14) completa com espaços, então "somente dígitos" já garante os 14
    CONSTRAINT CK_Fornecedores_Cnpj CHECK (Cnpj COLLATE Latin1_General_100_BIN2 NOT LIKE '%[^0-9]%')
);
GO

IF OBJECT_ID(N'dbo.Produtos', N'U') IS NULL
CREATE TABLE dbo.Produtos (
    Id           int IDENTITY(1,1) NOT NULL,
    PartNumber   nvarchar(8)   NOT NULL,
    NomeProduto  nvarchar(100) NOT NULL,
    Categoria    nvarchar(60)  NOT NULL,
    Unit         nvarchar(3)   NOT NULL CONSTRAINT DF_Produtos_Unit DEFAULT 'UN',
    Location     nvarchar(60)  NOT NULL CONSTRAINT DF_Produtos_Location DEFAULT '',
    MinimumStock int           NOT NULL CONSTRAINT DF_Produtos_MinimumStock DEFAULT 0,
    Saldo        int           NOT NULL CONSTRAINT DF_Produtos_Saldo DEFAULT 0,
    CONSTRAINT PK_Produtos PRIMARY KEY (Id),
    CONSTRAINT UQ_Produtos_PartNumber UNIQUE (PartNumber),
    CONSTRAINT CK_Produtos_PartNumber CHECK (LEN(PartNumber) = 8 AND LEFT(PartNumber, 2) IN
        ('47','48','84','86')
        AND PartNumber COLLATE Latin1_General_100_BIN2 NOT LIKE '%[^0-9]%'),
    CONSTRAINT CK_Produtos_Unidade CHECK (Unit IN ('UN','PC','PAR','CX','PCT','RL')),
    CONSTRAINT CK_Produtos_Minimo CHECK (MinimumStock BETWEEN 0 AND 999999999),
    CONSTRAINT CK_Produtos_Saldo CHECK (Saldo BETWEEN 0 AND 999999999)
);
GO

IF OBJECT_ID(N'dbo.Movimentacoes', N'U') IS NULL
CREATE TABLE dbo.Movimentacoes (
    Id          bigint IDENTITY(1,1) NOT NULL,
    ProdutoId   int           NOT NULL,
    Quantidade  int           NOT NULL, -- positivo = entrada, negativo = saída
    Motivo      nvarchar(150) NOT NULL,
    CriadaEmUtc datetime2     NOT NULL CONSTRAINT DF_Movimentacoes_CriadaEmUtc DEFAULT
```

```
        SYSUTCDATETIME(),
    UsuarioId    int           NULL,
    FornecedorId int           NULL,
    CONSTRAINT PK_Movimentacoes PRIMARY KEY (Id),
    CONSTRAINT FK_Movimentacoes_Produtos FOREIGN KEY (ProdutoId) REFERENCES dbo.Produtos (Id),
    CONSTRAINT FK_Movimentacoes_Usuarios FOREIGN KEY (UsuarioId) REFERENCES dbo.Usuarios (Id),
    CONSTRAINT FK_Movimentacoes_Fornecedores FOREIGN KEY (FornecedorId) REFERENCES dbo.Fornecedores
        (Id),
    CONSTRAINT CK_Movimentacoes_Quantidade CHECK (Quantidade <> 0 AND Quantidade BETWEEN -999999999
        AND 999999999)
);
GO

IF OBJECT_ID(N'dbo.AuditoriaProdutos', N'U') IS NULL
CREATE TABLE dbo.AuditoriaProdutos (
    Id            bigint IDENTITY(1,1) NOT NULL,
    ProdutoId     int           NOT NULL, -- sem FK: o registro sobrevive à exclusão do produto
    Operacao      char(1)       NOT NULL,
    Dados         nvarchar(max) NOT NULL,
    AlteradoEmUtc datetime2     NOT NULL CONSTRAINT DF_AuditoriaProdutos_AlteradoEmUtc DEFAULT
        SYSUTCDATETIME(),
    CONSTRAINT PK_AuditoriaProdutos PRIMARY KEY (Id),
    CONSTRAINT CK_AuditoriaProdutos_Operacao CHECK (Operacao IN ('I','U','D')),
    CONSTRAINT CK_AuditoriaProdutos_Dados CHECK (ISJSON(Dados) = 1)
);
GO

-- Índices para as consultas reais da API (a PK e as UNIQUE já criam os seus).
-- Busca por nome (GET /api/produtos?busca=); PartNumber já é coberto por UQ_Produtos_PartNumber.
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = N'IX_Produtos_NomeProduto' AND object_id =
    OBJECT_ID(N'dbo.Produtos'))
    CREATE INDEX IX_Produtos_NomeProduto ON dbo.Produtos (NomeProduto);
-- Reposição (MinimumStock > 0 AND Saldo <= MinimumStock): o filtro deixa de fora os itens sem
    mínimo.
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = N'IX_Produtos_Reposicao' AND object_id =
    OBJECT_ID(N'dbo.Produtos'))
    CREATE INDEX IX_Produtos_Reposicao ON dbo.Produtos (MinimumStock, Saldo) WHERE MinimumStock > 0;
-- Histórico por produto em ordem de lançamento (GET /api/movimentacoes?produtoId=).
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = N'IX_Movimentacoes_ProdutoId_Id' AND
    object_id = OBJECT_ID(N'dbo.Movimentacoes'))
    CREATE INDEX IX_Movimentacoes_ProdutoId_Id ON dbo.Movimentacoes (ProdutoId, Id);
-- Extrato por produto e período (sp_ExtratoProduto).
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = N'IX_Movimentacoes_ProdutoId_CriadaEmUtc' AND
    object_id = OBJECT_ID(N'dbo.Movimentacoes'))
    CREATE INDEX IX_Movimentacoes_ProdutoId_CriadaEmUtc ON dbo.Movimentacoes (ProdutoId,
        CriadaEmUtc) INCLUDE (Quantidade);
-- Movimentações do dia (dashboard / sp_ResumoEstoque).
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = N'IX_Movimentacoes_CriadaEmUtc' AND object_id
    = OBJECT_ID(N'dbo.Movimentacoes'))
    CREATE INDEX IX_Movimentacoes_CriadaEmUtc ON dbo.Movimentacoes (CriadaEmUtc);
-- Consulta da auditoria de um produto.
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = N'IX_AuditoriaProdutos_ProdutoId' AND
    object_id = OBJECT_ID(N'dbo.AuditoriaProdutos'))
    CREATE INDEX IX_AuditoriaProdutos_ProdutoId ON dbo.AuditoriaProdutos (ProdutoId, AlteradoEmUtc);
GO

-- ============================== 002_triggers.sql ==============================
-- PIM Estoque — 002: triggers. CREATE OR ALTER permite reexecutar o script.
SET ANSI_NULLS ON;
SET QUOTED_IDENTIFIER ON;
GO

-- Auditoria: grava um registro por linha afetada (funciona com INSERT/UPDATE/DELETE de várias
    linhas).
-- Dados = {"antes":{...},"depois":{...}}; "antes" falta no INSERT e "depois" falta no DELETE.
-- Não altera Saldo: só lê inserted/deleted, então não há dupla contagem.
CREATE OR ALTER TRIGGER dbo.trg_Produtos_Auditoria
ON dbo.Produtos
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;
    INSERT dbo.AuditoriaProdutos (ProdutoId, Operacao, Dados)
    SELECT k.Id,
           CASE WHEN NOT EXISTS (SELECT 1 FROM deleted d WHERE d.Id = k.Id) THEN 'I'
                WHEN NOT EXISTS (SELECT 1 FROM inserted i WHERE i.Id = k.Id) THEN 'D'
                ELSE 'U' END,
           (SELECT JSON_QUERY((SELECT d.* FROM deleted d WHERE d.Id = k.Id FOR JSON PATH,
               WITHOUT_ARRAY_WRAPPER)) AS antes,
                   JSON_QUERY((SELECT i.* FROM inserted i WHERE i.Id = k.Id FOR JSON PATH,
                       WITHOUT_ARRAY_WRAPPER)) AS depois
            FOR JSON PATH, WITHOUT_ARRAY_WRAPPER)
```

```
    FROM (SELECT Id FROM inserted UNION SELECT Id FROM deleted) AS k; -- Id é IDENTITY, não muda no
        UPDATE
END;
GO

-- Histórico imutável: correção de lançamento se faz com nova movimentação (estorno), nunca
    editando.
-- INSTEAD OF apenas em UPDATE/DELETE, para não interferir no INSERT ... OUTPUT do EF Core.
CREATE OR ALTER TRIGGER dbo.trg_Movimentacoes_Imutavel
ON dbo.Movimentacoes
INSTEAD OF UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (SELECT 1 FROM deleted)
        THROW 50010, N'Movimentações não podem ser alteradas nem excluídas. Registre um estorno.',
            1;
END;
GO

-- ============================== 003_procedures.sql ==============================
-- PIM Estoque — 003: view e procedures. CREATE OR ALTER permite reexecutar o script.
-- Regra de reposição (igual à API): MinimumStock > 0 AND Saldo <= MinimumStock.
SET ANSI_NULLS ON;
SET QUOTED_IDENTIFIER ON;
GO

CREATE OR ALTER VIEW dbo.vw_SaldoPorCategoria
AS
SELECT Categoria,
       COUNT(*)    AS Produtos,
       SUM(Saldo)  AS Saldo,
       SUM(CASE WHEN MinimumStock > 0 AND Saldo <= MinimumStock THEN 1 ELSE 0 END) AS ItensReposicao
FROM dbo.Produtos
GROUP BY Categoria;
GO

-- Entrada ou saída de material: atualiza o saldo e grava o histórico na mesma transação.
-- UPDLOCK + HOLDLOCK trava a linha do produto até o COMMIT, então duas saídas simultâneas
-- não conseguem ler o mesmo saldo e deixá-lo negativo.
CREATE OR ALTER PROCEDURE dbo.sp_RegistrarMovimentacao
    @ProdutoId    int,
    @Tipo         varchar(7),      -- 'entrada' ou 'saida'
    @Quantidade   int,             -- sempre positiva; o sinal vem do @Tipo
    @Motivo       nvarchar(150),
    @UsuarioId    int = NULL,
    @FornecedorId int = NULL
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    IF @Tipo NOT IN ('entrada', 'saida') OR @Tipo IS NULL
        THROW 50001, N'Tipo inválido: use ''entrada'' ou ''saida''.', 1;
    IF @Quantidade IS NULL OR @Quantidade NOT BETWEEN 1 AND 999999999
        THROW 50003, N'Quantidade deve estar entre 1 e 999999999.', 1;
    IF NULLIF(LTRIM(RTRIM(@Motivo)), N'') IS NULL
        THROW 50004, N'Informe o motivo da movimentação.', 1;

    DECLARE @Delta int = CASE @Tipo WHEN 'entrada' THEN @Quantidade ELSE -@Quantidade END;
    DECLARE @Saldo int, @MovimentacaoId bigint;

    BEGIN TRY
        BEGIN TRANSACTION;

        SELECT @Saldo = Saldo
        FROM dbo.Produtos WITH (UPDLOCK, HOLDLOCK)
        WHERE Id = @ProdutoId;

        IF @Saldo IS NULL
            THROW 50005, N'Produto não encontrado.', 1;
        IF @Saldo + CAST(@Delta AS bigint) < 0
            THROW 50002, N'Saldo insuficiente para a saída.', 1;
        IF @Saldo + CAST(@Delta AS bigint) > 999999999
            THROW 50006, N'Saldo ultrapassaria o limite de 999999999.', 1;

        UPDATE dbo.Produtos SET Saldo = Saldo + @Delta WHERE Id = @ProdutoId;

        INSERT dbo.Movimentacoes (ProdutoId, Quantidade, Motivo, CriadaEmUtc, UsuarioId,
            FornecedorId)
        VALUES (@ProdutoId, @Delta, LTRIM(RTRIM(@Motivo)), SYSUTCDATETIME(), @UsuarioId,
```

```
            @FornecedorId);
        SET @MovimentacaoId = SCOPE_IDENTITY();

        COMMIT;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK;
        THROW;
    END CATCH;

    SELECT @MovimentacaoId AS MovimentacaoId, @Saldo + @Delta AS SaldoAtual;
END;
GO

-- Itens no mínimo ou abaixo dele, os mais críticos primeiro.
CREATE OR ALTER PROCEDURE dbo.sp_ProdutosParaReposicao
AS
BEGIN
    SET NOCOUNT ON;
    SELECT Id, PartNumber, NomeProduto, Categoria, Unit, Location, MinimumStock, Saldo,
           MinimumStock - Saldo AS Falta
    FROM dbo.Produtos
    WHERE MinimumStock > 0 AND Saldo <= MinimumStock
    ORDER BY MinimumStock - Saldo DESC, NomeProduto;
END;
GO

-- Indicadores do painel (mesmos campos de GET /api/dashboard/resumo).
-- 1° resultado: totais; 2° resultado: saldo por categoria. "Hoje" é o dia UTC.
CREATE OR ALTER PROCEDURE dbo.sp_ResumoEstoque
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @Hoje datetime2 = CAST(CAST(SYSUTCDATETIME() AS date) AS datetime2);

    SELECT (SELECT COUNT(*) FROM dbo.Produtos) AS TotalProdutos,
           (SELECT COUNT(*) FROM dbo.Produtos WHERE MinimumStock > 0 AND Saldo <= MinimumStock) AS
               ItensReposicao,
           (SELECT ISNULL(SUM(CAST(Saldo AS bigint)), 0) FROM dbo.Produtos) AS SaldoTotal,
           (SELECT COUNT(*) FROM dbo.Movimentacoes WHERE CriadaEmUtc >= @Hoje AND CriadaEmUtc <
               DATEADD(day, 1, @Hoje)) AS MovimentacoesHoje;

    SELECT Categoria, Produtos, Saldo FROM dbo.vw_SaldoPorCategoria ORDER BY Categoria;
END;
GO

-- Extrato de um produto com saldo após cada lançamento. O saldo acumulado é calculado
-- sobre todo o histórico e só depois filtrado pelo período, para não começar do zero.
CREATE OR ALTER PROCEDURE dbo.sp_ExtratoProduto
    @ProdutoId int,
    @De        datetime2 = NULL,
    @Ate       datetime2 = NULL
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT EXISTS (SELECT 1 FROM dbo.Produtos WHERE Id = @ProdutoId)
        THROW 50005, N'Produto não encontrado.', 1;

    WITH Extrato AS (
        SELECT m.Id, m.CriadaEmUtc, m.Quantidade, m.Motivo,
               u.Nome AS UsuarioNome, f.Nome AS FornecedorNome,
               SUM(m.Quantidade) OVER (ORDER BY m.Id ROWS UNBOUNDED PRECEDING) AS SaldoApos
        FROM dbo.Movimentacoes m
        LEFT JOIN dbo.Usuarios u ON u.Id = m.UsuarioId
        LEFT JOIN dbo.Fornecedores f ON f.Id = m.FornecedorId
        WHERE m.ProdutoId = @ProdutoId
    )
    SELECT Id, CriadaEmUtc,
           CASE WHEN Quantidade > 0 THEN 'entrada' ELSE 'saida' END AS Tipo,
           ABS(Quantidade) AS Quantidade, Motivo, UsuarioNome, FornecedorNome, SaldoApos
    FROM Extrato
    WHERE (@De IS NULL OR CriadaEmUtc >= @De)
      AND (@Ate IS NULL OR CriadaEmUtc <= @Ate)
    ORDER BY Id;
END;
GO

-- ============================== 004_seed.sql ==============================
-- PIM Estoque — 004: dados de demonstração (empresa fictícia Fixar Manutenção).
-- Não cria usuários: o administrador inicial é semeado pela API (Seed:AdminEmail /
    Seed:AdminSenha).
```

```
-- Produtos começam com saldo 0 e recebem as quantidades por sp_RegistrarMovimentacao,
-- então o histórico sempre fecha com o saldo. Reexecutar não duplica nada.
SET ANSI_NULLS ON;
SET QUOTED_IDENTIFIER ON;
SET NOCOUNT ON;
GO

INSERT dbo.Fornecedores (Nome, Cnpj, Telefone, Email)
SELECT v.Nome, v.Cnpj, v.Telefone, v.Email
FROM (VALUES
    (N'Proteção Sorocaba EPIs Ltda.',     '12345678000195', N'(15) 3221-4400',
        N'vendas@protecaosorocaba.com.br'),
    (N'Fixa Forte Parafusos e Fixadores', '45678912000155', N'(15) 3232-1010',
        N'pedidos@fixaforte.com.br'),
    (N'EletroHidro Materiais Ltda.',      '78912345000109', N'(15) 3018-7788',
        N'comercial@eletrohidro.com.br')
) AS v (Nome, Cnpj, Telefone, Email)
WHERE NOT EXISTS (SELECT 1 FROM dbo.Fornecedores f WHERE f.Cnpj = v.Cnpj);
GO

INSERT dbo.Produtos (PartNumber, NomeProduto, Categoria, Unit, Location, MinimumStock)
SELECT v.PartNumber, v.NomeProduto, v.Categoria, v.Unit, v.Location, v.MinimumStock
FROM (VALUES
    (N'47000101', N'Luva de vaqueta tamanho 9',            N'EPI',        N'PAR', N'A-01-01', 20),
    (N'47000102', N'Óculos de proteção incolor',           N'EPI',        N'UN',  N'A-01-02', 10),
    (N'47000201', N'Detergente neutro 5 L',                N'Limpeza',    N'UN',  N'D-01-01', 4),
    (N'47000202', N'Pano de microfibra 40x40 cm',          N'Limpeza',    N'PCT', N'D-01-02', 6),
    (N'48000101', N'Parafuso sextavado M8x50 zincado',     N'Fixadores',  N'CX',  N'B-02-01', 5),
    (N'48000102', N'Bucha de nylon 8 mm',                  N'Fixadores',  N'PCT', N'B-02-02', 10),
    (N'84000101', N'Cabo flexível 2,5 mm² azul 100 m',     N'Elétrico',   N'RL',  N'C-01-01', 3),
    (N'84000102', N'Disjuntor monopolar DIN 20 A',         N'Elétrico',   N'UN',  N'C-01-02', 10),
    (N'84000103', N'Fita isolante 19 mm x 20 m',           N'Elétrico',   N'RL',  N'C-01-03', 15),
    (N'86000101', N'Tubo PVC soldável 25 mm barra 6 m',    N'Hidráulico', N'PC',  N'E-01-01', 10),
    (N'86000102', N'Registro de gaveta 3/4"',              N'Hidráulico', N'UN',  N'E-01-02', 4),
    (N'86000103', N'Fita veda-rosca 18 mm x 50 m',         N'Hidráulico', N'RL',  N'E-01-03', 12)
) AS v (PartNumber, NomeProduto, Categoria, Unit, Location, MinimumStock)
WHERE NOT EXISTS (SELECT 1 FROM dbo.Produtos p WHERE p.PartNumber = v.PartNumber);
GO

-- Movimentações só na primeira carga (banco sem histórico).
IF NOT EXISTS (SELECT 1 FROM dbo.Movimentacoes)
BEGIN
    DECLARE @Lancamentos TABLE (Ordem int IDENTITY, PartNumber nvarchar(8), Tipo varchar(7),
        Quantidade int, Motivo nvarchar(150), Cnpj char(14));
    INSERT @Lancamentos (PartNumber, Tipo, Quantidade, Motivo, Cnpj) VALUES
        ('47000101', 'entrada', 60,  N'NF 10231 - compra mensal de EPIs',
            '12345678000195'),
        ('47000102', 'entrada', 30,  N'NF 10231 - compra mensal de EPIs',
            '12345678000195'),
        ('47000201', 'entrada', 12,  N'NF 10240 - material de limpeza',
            '12345678000195'),
        ('47000202', 'entrada', 10,  N'NF 10240 - material de limpeza',
            '12345678000195'),
        ('48000101', 'entrada', 20,  N'NF 5521 - reposição de fixadores',
            '45678912000155'),
        ('48000102', 'entrada', 40,  N'NF 5521 - reposição de fixadores',
            '45678912000155'),
        ('84000101', 'entrada', 8,   N'NF 88310 - material elétrico',
            '78912345000109'),
        ('84000102', 'entrada', 25,  N'NF 88310 - material elétrico',
            '78912345000109'),
        ('84000103', 'entrada', 40,  N'NF 88310 - material elétrico',
            '78912345000109'),
        ('86000101', 'entrada', 30,  N'NF 88342 - material hidráulico',
            '78912345000109'),
        ('86000102', 'entrada', 10,  N'NF 88342 - material hidráulico',
            '78912345000109'),
        ('86000103', 'entrada', 36,  N'NF 88342 - material hidráulico',
            '78912345000109'),
        ('47000101', 'saida',   44,  N'OS 2026-118 - equipe de campo Cond. Ipanema', NULL),
        ('47000102', 'saida',   6,   N'OS 2026-118 - equipe de campo Cond. Ipanema', NULL),
        ('47000202', 'saida',   5,   N'Limpeza pós-obra loja Centro',                NULL),
        ('48000101', 'saida',   8,   N'OS 2026-121 - fixação de eletrocalhas',       NULL),
        ('84000101', 'saida',   6,   N'OS 2026-121 - troca de circuito',             NULL),
        ('84000102', 'saida',   12,  N'OS 2026-124 - quadro de distribuição',        NULL),
        ('84000103', 'saida',   10,  N'OS 2026-124 - quadro de distribuição',        NULL),
        ('86000101', 'saida',   24,  N'OS 2026-127 - prumada de água fria',          NULL),
        ('86000102', 'saida',   7,   N'OS 2026-127 - prumada de água fria',          NULL),
        ('86000103', 'saida',   20,  N'OS 2026-127 - prumada de água fria',          NULL);

    DECLARE @i int = 1, @Fim int = (SELECT MAX(Ordem) FROM @Lancamentos),
            @ProdutoId int, @Tipo varchar(7), @Quantidade int, @Motivo nvarchar(150), @FornecedorId
```

```
                int;
    DECLARE @Ignorar TABLE (MovimentacaoId bigint, SaldoAtual int); -- descarta o retorno da
        procedure

    WHILE @i <= @Fim
    BEGIN
        SELECT @ProdutoId = p.Id, @Tipo = l.Tipo, @Quantidade = l.Quantidade, @Motivo = l.Motivo,
            @FornecedorId = f.Id
        FROM @Lancamentos l
        JOIN dbo.Produtos p ON p.PartNumber = l.PartNumber
        LEFT JOIN dbo.Fornecedores f ON f.Cnpj = l.Cnpj
        WHERE l.Ordem = @i;

        INSERT @Ignorar EXEC dbo.sp_RegistrarMovimentacao @ProdutoId, @Tipo, @Quantidade, @Motivo,
            NULL, @FornecedorId;
        SET @i += 1;
    END;
END;
GO
```

APÊNDICE B – CÓDIGO-FONTE DA API (ASP.NET CORE)

Código-fonte integral da API REST que concentra as regras de negócio do PIM Estoque (seções 5 e 7), na ordem das camadas: inicialização, controllers, services, acesso a dados, entidades e contratos.

Arquivo: Program.cs

```
using System.Text;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Authorization;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi.Models;
using PimEstoque.Controllers;
using PimEstoque.Data;
using PimEstoque.Services;

var builder = WebApplication.CreateBuilder(args);
var config = builder.Configuration;

// Falha na inicialização em vez de emitir tokens com chave fraca ou ausente.
if (Encoding.UTF8.GetByteCount(config["Jwt:Key"] ?? "") < 32)
    throw new InvalidOperationException("Configure Jwt:Key com pelo menos 32 bytes (variável de
        ambiente Jwt__Key ou User Secrets).");

builder.Services.AddDbContext<PimEstoqueContext>(options =>
    options.UseSqlServer(config.GetConnectionString("DefaultConnection")));
builder.Services.AddScoped<ProdutoService>();
builder.Services.AddScoped<MovimentacaoService>();
builder.Services.AddScoped<FornecedorService>();
builder.Services.AddScoped<UsuarioService>();
builder.Services.AddScoped<AuthService>();
builder.Services.AddScoped<DashboardService>();

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(options =>
{
    options.MapInboundClaims = false;
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidIssuer = config["Jwt:Issuer"],
        ValidAudience = config["Jwt:Issuer"],
        IssuerSigningKey = AuthService.ChaveAssinatura(config),
        NameClaimType = "name",
        RoleClaimType = "role",
        ClockSkew = TimeSpan.FromMinutes(1),
    };
    // Usuário desativado perde o acesso imediatamente, mesmo com token ainda válido.
    options.Events = new JwtBearerEvents
    {
        OnTokenValidated = async context =>
        {
            var db = context.HttpContext.RequestServices.GetRequiredService<PimEstoqueContext>();
            var id = context.Principal!.UsuarioId();
            if (!await db.Usuarios.AnyAsync(u => u.Id == id && u.Ativo)) context.Fail("Usuário
                inativo.");
        },
    };
});
builder.Services.AddAuthorization(options => options.FallbackPolicy = new
    AuthorizationPolicyBuilder().RequireAuthenticatedUser().Build());

builder.Services.AddControllers(options => options.Filters.Add<ErroNegocioFilter>());
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(options =>
{
    var bearer = new OpenApiSecurityScheme { Name = "Authorization", Type =
        SecuritySchemeType.Http, Scheme = "bearer", BearerFormat = "JWT", In =
            ParameterLocation.Header, Description = "Token obtido em POST /api/auth/login." };
    options.AddSecurityDefinition("Bearer", bearer);
    options.AddSecurityRequirement(new OpenApiSecurityRequirement { [new OpenApiSecurityScheme {
        Reference = new OpenApiReference { Type = ReferenceType.SecurityScheme, Id = "Bearer" } }]
            = [] });
});
builder.Services.AddProblemDetails();
```

```
var origins = config.GetSection("Cors:Origins").Get<string[]>() ?? [];
builder.Services.AddCors(options => options.AddDefaultPolicy(policy => { if (origins.Length > 0)
    policy.WithOrigins(origins).AllowAnyHeader().AllowAnyMethod(); }));

var app = builder.Build();

if (app.Configuration["Seed:AdminEmail"] is { Length: > 0 } seedEmail &&
    app.Configuration["Seed:AdminSenha"] is { Length: > 0 } seedSenha)
{
    using var scope = app.Services.CreateScope();
    try { await scope.ServiceProvider.GetRequiredService<UsuarioService>()
        .GarantirAdminInicialAsync(seedEmail, seedSenha); }
    catch (Exception e) { app.Logger.LogError(e, "Administrador inicial não criado: verifique a
        conexão e se os scripts de database/ foram aplicados."); }
}

app.UseExceptionHandler();
app.Use(async (context, next) =>
{
    var headers = context.Response.Headers;
    headers.XContentTypeOptions = "nosniff";
    headers.XFrameOptions = "DENY";
    headers["Referrer-Policy"] = "no-referrer";
    // O Swagger UI usa script inline; o restante (API e painel) segue CSP restritiva.
    if (!context.Request.Path.StartsWithSegments("/swagger"))
        headers.ContentSecurityPolicy = "default-src 'self'; frame-ancestors 'none'; object-src
            'none'; base-uri 'self'; form-action 'self'";
    await next();
});
if (app.Environment.IsDevelopment()) { app.UseSwagger(); app.UseSwaggerUI(); }
app.UseDefaultFiles();
app.UseStaticFiles();
app.UseCors();
app.UseAuthentication();
app.UseAuthorization();
app.MapGet("/health", () => Results.Ok(new { status = "ok", database = "not_checked"
    })).AllowAnonymous();
app.MapControllers();
app.Run();
public partial class Program { }
```

Arquivo: Controllers/AuthController.cs

```
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.JsonWebTokens;
using PimEstoque.Contracts;
using PimEstoque.Services;

namespace PimEstoque.Controllers;

[ApiController, Route("api/auth")]
public class AuthController(AuthService auth, UsuarioService usuarios) : ControllerBase
{
    [HttpPost("login"), AllowAnonymous]
    public async Task<IActionResult> Login(LoginRequest request) =>
        await auth.LoginAsync(request) is { } resposta ? Ok(resposta) : Unauthorized(new { message
            = "E-mail ou senha inválidos." });

    [HttpGet("me")]
    public async Task<IActionResult> Me() => await usuarios.ObterAsync(User.UsuarioId()) is {
        Ativo: true } u ? Ok(u) : Unauthorized();
}

public static class UsuarioLogado
{
    /// <summary>Id do usuário autenticado (claim "sub" do JWT).</summary>
    public static int UsuarioId(this ClaimsPrincipal user) =>
        int.Parse(user.FindFirstValue(JwtRegisteredClaimNames.Sub)!);
}
```

Arquivo: Controllers/DashboardController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using PimEstoque.Services;

namespace PimEstoque.Controllers;

[ApiController, Route("api/dashboard"), Authorize]
```

```
public class DashboardController(DashboardService dashboard) : ControllerBase
{
    [HttpGet("resumo")]
    public async Task<IActionResult> Resumo() => Ok(await dashboard.ResumoAsync());
}
```

Arquivo: Controllers/FornecedoresController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using PimEstoque.Contracts;
using PimEstoque.Models;
using PimEstoque.Services;

namespace PimEstoque.Controllers;

[ApiController, Route("api/fornecedores"), Authorize]
public class FornecedoresController(FornecedorService fornecedores) : ControllerBase
{
    [HttpGet]
    public async Task<IActionResult> Listar() => Ok(await fornecedores.ListarAsync());

    [HttpGet("{id:int}")]
    public async Task<IActionResult> Obter(int id) => await fornecedores.ObterAsync(id) is { } f ?
        Ok(f) : NotFound();

    [HttpPost, Authorize(Roles = Perfis.Escrita)]
    public async Task<IActionResult> Criar(FornecedorRequest request)
    {
        var f = await fornecedores.CriarAsync(request);
        return CreatedAtAction(nameof(Obter), new { id = f.Id }, f);
    }

    [HttpPut("{id:int}"), Authorize(Roles = Perfis.Escrita)]
    public async Task<IActionResult> Editar(int id, FornecedorRequest request) => await
        fornecedores.EditarAsync(id, request) is { } f ? Ok(f) : NotFound();
}
```

Arquivo: Controllers/MovimentacoesController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using PimEstoque.Contracts;
using PimEstoque.Models;
using PimEstoque.Services;

namespace PimEstoque.Controllers;

[ApiController, Route("api/movimentacoes"), Authorize]
public class MovimentacoesController(MovimentacaoService movimentacoes) : ControllerBase
{
    [HttpGet]
    public async Task<IActionResult> Listar([FromQuery] int? produtoId) => Ok(await
        movimentacoes.ListarAsync(produtoId));

    [HttpPost, Authorize(Roles = Perfis.Escrita)]
    public async Task<IActionResult> Registrar(MovimentacaoRequest request) => StatusCode(201,
        await movimentacoes.RegistrarAsync(request, User.UsuarioId()));
}
```

Arquivo: Controllers/ProdutosController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using PimEstoque.Contracts;
using PimEstoque.Models;
using PimEstoque.Services;

namespace PimEstoque.Controllers;

[ApiController, Route("api/produtos"), Authorize]
public class ProdutosController(ProdutoService produtos) : ControllerBase
{
    [HttpGet]
    public async Task<IActionResult> Listar([FromQuery] string? busca, [FromQuery] bool reposicao =
        false) => Ok(await produtos.ListarAsync(busca, reposicao));

    [HttpGet("{id:int}")]
    public async Task<IActionResult> Obter(int id) => await produtos.ObterAsync(id) is { } product
        ? Ok(product) : NotFound();
```

```
    [HttpPost, Authorize(Roles = Perfis.Escrita)]
    public async Task<IActionResult> Criar(ProdutoRequest request)
    {
        var product = await produtos.CriarAsync(request);
        return CreatedAtAction(nameof(Obter), new { id = product.Id }, product);
    }

    [HttpPut("{id:int}"), Authorize(Roles = Perfis.Escrita)]
    public async Task<IActionResult> Editar(int id, ProdutoRequest request) => await
        produtos.EditarAsync(id, request) is { } product ? Ok(product) : NotFound();

    [HttpDelete("{id:int}"), Authorize(Roles = Perfis.Administrador)]
    public async Task<IActionResult> Excluir(int id) => await produtos.ExcluirAsync(id) ?
        NoContent() : NotFound();
}
```

Arquivo: Controllers/UsuariosController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using PimEstoque.Contracts;
using PimEstoque.Models;
using PimEstoque.Services;

namespace PimEstoque.Controllers;

[ApiController, Route("api/usuarios"), Authorize(Roles = Perfis.Administrador)]
public class UsuariosController(UsuarioService usuarios) : ControllerBase
{
    [HttpGet]
    public async Task<IActionResult> Listar() => Ok(await usuarios.ListarAsync());

    [HttpGet("{id:int}")]
    public async Task<IActionResult> Obter(int id) => await usuarios.ObterAsync(id) is { } u ?
        Ok(u) : NotFound();

    [HttpPost]
    public async Task<IActionResult> Criar(UsuarioRequest request)
    {
        var u = await usuarios.CriarAsync(request);
        return CreatedAtAction(nameof(Obter), new { id = u.Id }, u);
    }

    [HttpPut("{id:int}")]
    public async Task<IActionResult> Editar(int id, UsuarioRequest request) => await
        usuarios.EditarAsync(id, request, User.UsuarioId()) is { } u ? Ok(u) : NotFound();
}
```

Arquivo: Services/AuthService.cs

```
using System.Text;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.JsonWebTokens;
using Microsoft.IdentityModel.Tokens;
using PimEstoque.Contracts;
using PimEstoque.Data;

namespace PimEstoque.Services;

public class AuthService(PimEstoqueContext db, IConfiguration config)
{
    // Hash descartável: com e-mail inexistente o login gasta o mesmo tempo e não revela quais
        e-mails existem.
    private static readonly string HashFicticio = SenhaHasher.Gerar(Guid.NewGuid().ToString());

    public static SymmetricSecurityKey ChaveAssinatura(IConfiguration config) =>
        new(Encoding.UTF8.GetBytes(config["Jwt:Key"] ?? ""));

    public async Task<LoginResponse?> LoginAsync(LoginRequest request)
    {
        var email = request.Email.Trim().ToLowerInvariant();
        var user = await db.Usuarios.AsNoTracking().SingleOrDefaultAsync(u => u.Email == email);
        var senhaOk = SenhaHasher.Verificar(request.Senha, user?.SenhaHash ?? HashFicticio);
        if (user is null || !senhaOk || !user.Ativo) return null;

        var expira = DateTime.UtcNow.AddMinutes(config.GetValue("Jwt:ExpiraEmMinutos", 480));
        var issuer = config["Jwt:Issuer"];
        var token = new JsonWebTokenHandler().CreateToken(new SecurityTokenDescriptor
        {
            Issuer = issuer,
            Audience = issuer,
```

```
            Expires = expira,
            Claims = new Dictionary<string, object>
            {
                [JwtRegisteredClaimNames.Sub] = user.Id.ToString(),
                [JwtRegisteredClaimNames.Name] = user.Nome,
                [JwtRegisteredClaimNames.Email] = user.Email,
                ["role"] = user.Perfil,
            },
            SigningCredentials = new SigningCredentials(ChaveAssinatura(config),
                SecurityAlgorithms.HmacSha256),
        });
        return new LoginResponse(token, expira, UsuarioResponse.De(user));
    }
}
```

Arquivo: Services/DashboardService.cs

```
using Microsoft.EntityFrameworkCore;
using PimEstoque.Contracts;
using PimEstoque.Data;

namespace PimEstoque.Services;

public class DashboardService(PimEstoqueContext db)
{
    public async Task<DashboardResponse> ResumoAsync()
    {
        // shortcut: "hoje" é o dia em UTC; se o painel precisar do dia de Brasília, calcular o
            início do dia pelo fuso.
        var inicioHoje = DateTime.UtcNow.Date;
        var porCategoria = await db.Produtos.AsNoTracking().GroupBy(p => p.Categoria).OrderBy(g =>
            g.Key)
            .Select(g => new CategoriaResumo(g.Key, g.Count(), g.Sum(p => (long)p.Saldo)))
            .ToListAsync();
        return new DashboardResponse(
            porCategoria.Sum(c => c.Produtos),
            await db.Produtos.CountAsync(p => p.MinimumStock > 0 && p.Saldo <= p.MinimumStock),
            porCategoria.Sum(c => c.Saldo),
            await db.Movimentacoes.CountAsync(m => m.CriadaEmUtc >= inicioHoje),
            porCategoria);
    }
}
```

Arquivo: Services/ErroNegocio.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;
using Microsoft.Data.SqlClient;
using Microsoft.EntityFrameworkCore;

namespace PimEstoque.Services;

/// <summary>Regra de negócio violada; vira resposta HTTP <c>{ message }</c> com o status
    informado.</summary>
public sealed class ErroNegocio(int status, string mensagem) : Exception(mensagem)
{
    public int Status { get; } = status;

    public static ErroNegocio Conflito(string mensagem) => new(StatusCodes.Status409Conflict,
        mensagem);
    public static ErroNegocio Invalido(string mensagem) => new(StatusCodes.Status400BadRequest,
        mensagem);
    public static ErroNegocio NaoEncontrado(string mensagem) => new(StatusCodes.Status404NotFound,
        mensagem);

    /// <summary>Violação de índice único no SQL Server (corrida entre a verificação e o
        INSERT/UPDATE).</summary>
    public static bool EhDuplicidade(DbUpdateException e) => e.InnerException is SqlException {
        Number: 2601 or 2627 };
}

public sealed class ErroNegocioFilter : IExceptionFilter
{
    public void OnException(ExceptionContext context)
    {
        if (context.Exception is not ErroNegocio erro) return;
        context.Result = new ObjectResult(new { message = erro.Message }) { StatusCode =
            erro.Status };
        context.ExceptionHandled = true;
    }
}
```

Arquivo: Services/FornecedorService.cs

```
using Microsoft.EntityFrameworkCore;
using PimEstoque.Contracts;
using PimEstoque.Data;
using PimEstoque.Models;

namespace PimEstoque.Services;

public class FornecedorService(PimEstoqueContext db)
{
    private const string Duplicado = "CNPJ já cadastrado.";

    public Task<List<Fornecedor>> ListarAsync() => db.Fornecedores.AsNoTracking().OrderBy(f =>
        f.Nome).ToListAsync();

    public Task<Fornecedor?> ObterAsync(int id) =>
        db.Fornecedores.AsNoTracking().SingleOrDefaultAsync(f => f.Id == id);

    public async Task<Fornecedor> CriarAsync(FornecedorRequest request)
    {
        var supplier = new Fornecedor();
        db.Fornecedores.Add(supplier);
        await SalvarAsync(supplier, request);
        return supplier;
    }

    public async Task<Fornecedor?> EditarAsync(int id, FornecedorRequest request)
    {
        var supplier = await db.Fornecedores.SingleOrDefaultAsync(f => f.Id == id);
        if (supplier is null) return null;
        await SalvarAsync(supplier, request);
        return supplier;
    }

    private async Task SalvarAsync(Fornecedor supplier, FornecedorRequest request)
    {
        if (await db.Fornecedores.AnyAsync(f => f.Id != supplier.Id && f.Cnpj == request.Cnpj))
            throw ErroNegocio.Conflito(Duplicado);
        supplier.Nome = request.Nome.Trim();
        supplier.Cnpj = request.Cnpj;
        supplier.Telefone = string.IsNullOrWhiteSpace(request.Telefone) ? null :
            request.Telefone.Trim();
        supplier.Email = string.IsNullOrWhiteSpace(request.Email) ? null :
            request.Email.Trim().ToLowerInvariant();
        supplier.Ativo = request.Ativo;
        try { await db.SaveChangesAsync(); }
        catch (DbUpdateException e) when (ErroNegocio.EhDuplicidade(e)) { throw
            ErroNegocio.Conflito(Duplicado); }
    }
}
```

Arquivo: Services/MovimentacaoService.cs

```
using Microsoft.EntityFrameworkCore;
using PimEstoque.Contracts;
using PimEstoque.Data;
using PimEstoque.Models;

namespace PimEstoque.Services;

public class MovimentacaoService(PimEstoqueContext db)
{
    public Task<List<MovimentacaoResponse>> ListarAsync(int? produtoId)
    {
        var query = db.Movimentacoes.AsNoTracking();
        if (produtoId.HasValue) query = query.Where(m => m.ProdutoId == produtoId);
        return query.OrderByDescending(m => m.Id).Take(100)
            .Select(m => new MovimentacaoResponse(m.Id, m.ProdutoId, m.Quantidade, m.Motivo,
                m.CriadaEmUtc, m.Usuario!.Nome, m.Fornecedor!.Nome))
            .ToListAsync();
    }

    /// <summary>Mesma regra da sp_RegistrarMovimentacao: saldo atualizado de forma condicional e
        histórico gravado na mesma transação.</summary>
    public async Task<MovimentacaoResponse> RegistrarAsync(MovimentacaoRequest request, int
        usuarioId)
    {
        await using var transaction = await db.Database.BeginTransactionAsync();
        if (!await db.Produtos.AnyAsync(p => p.Id == request.ProdutoId)) throw
            ErroNegocio.NaoEncontrado("Material não encontrado.");
```

```
        string? fornecedorNome = null;
        if (request.FornecedorId is { } fornecedorId)
            fornecedorNome = await db.Fornecedores.Where(f => f.Id == fornecedorId &&
                f.Ativo).Select(f => f.Nome).SingleOrDefaultAsync()
                ?? throw ErroNegocio.Invalido("Fornecedor inexistente ou inativo.");
        var delta = request.Tipo == "entrada" ? request.Quantidade : -request.Quantidade;
        var updated = await db.Produtos.Where(p => p.Id == request.ProdutoId && p.Saldo + delta >=
            0 && p.Saldo <= 999999999 - Math.Max(delta, 0))
            .ExecuteUpdateAsync(set => set.SetProperty(p => p.Saldo, p => p.Saldo + delta));
        if (updated == 0) throw ErroNegocio.Conflito("Saldo insuficiente ou limite de saldo
            excedido.");
        var movement = new Movimentacao { ProdutoId = request.ProdutoId, Quantidade = delta, Motivo
            = request.Motivo.Trim(), CriadaEmUtc = DateTime.UtcNow, UsuarioId = usuarioId,
                FornecedorId = request.FornecedorId };
        db.Movimentacoes.Add(movement);
        await db.SaveChangesAsync();
        await transaction.CommitAsync();
        var usuarioNome = await db.Usuarios.Where(u => u.Id == usuarioId).Select(u =>
            u.Nome).SingleOrDefaultAsync();
        return new MovimentacaoResponse(movement.Id, movement.ProdutoId, movement.Quantidade,
            movement.Motivo, movement.CriadaEmUtc, usuarioNome, fornecedorNome);
    }
}
```

Arquivo: Services/ProdutoService.cs

```
using Microsoft.EntityFrameworkCore;
using PimEstoque.Contracts;
using PimEstoque.Data;
using PimEstoque.Models;

namespace PimEstoque.Services;

public class ProdutoService(PimEstoqueContext db)
{
    private const string Duplicado = "Part number já cadastrado.";
    private const string ComHistorico = "Material com movimentações não pode ser excluído; o
        histórico é preservado.";

    public async Task<List<Produto>> ListarAsync(string? busca, bool reposicao)
    {
        var query = db.Produtos.AsNoTracking();
        if (!string.IsNullOrWhiteSpace(busca)) query = query.Where(p =>
            p.PartNumber.Contains(busca) || p.NomeProduto.Contains(busca) ||
                p.Categoria.Contains(busca) || p.Location.Contains(busca));
        if (reposicao) query = query.Where(p => p.MinimumStock > 0 && p.Saldo <= p.MinimumStock);
        return await query.OrderBy(p => p.PartNumber).Take(500).ToListAsync();
    }

    public Task<Produto?> ObterAsync(int id) => db.Produtos.AsNoTracking().SingleOrDefaultAsync(p
        => p.Id == id);

    public async Task<Produto> CriarAsync(ProdutoRequest request)
    {
        if (await db.Produtos.AnyAsync(p => p.PartNumber == request.PartNumber)) throw
            ErroNegocio.Conflito(Duplicado);
        var product = new Produto();
        Aplicar(product, request);
        db.Produtos.Add(product);
        try { await db.SaveChangesAsync(); }
        catch (DbUpdateException e) when (ErroNegocio.EhDuplicidade(e)) { throw
            ErroNegocio.Conflito(Duplicado); }
        return product;
    }

    public async Task<Produto?> EditarAsync(int id, ProdutoRequest request)
    {
        await using var transaction = await
            db.Database.BeginTransactionAsync(System.Data.IsolationLevel.Serializable);
        var product = await db.Produtos.SingleOrDefaultAsync(p => p.Id == id);
        if (product is null) return null;
        if (await db.Produtos.AnyAsync(p => p.Id != id && p.PartNumber == request.PartNumber))
            throw ErroNegocio.Conflito(Duplicado);
        if (product.Unit != request.Unit && await db.Movimentacoes.AnyAsync(m => m.ProdutoId ==
            id)) throw ErroNegocio.Conflito("Unidade bloqueada após movimentações.");
        Aplicar(product, request);
        try { await db.SaveChangesAsync(); await transaction.CommitAsync(); }
        catch (DbUpdateException e) when (ErroNegocio.EhDuplicidade(e)) { throw
            ErroNegocio.Conflito(Duplicado); }
        return product;
    }
```

```
    public async Task<bool> ExcluirAsync(int id)
    {
        var product = await db.Produtos.SingleOrDefaultAsync(p => p.Id == id);
        if (product is null) return false;
        if (await db.Movimentacoes.AnyAsync(m => m.ProdutoId == id)) throw
            ErroNegocio.Conflito(ComHistorico);
        db.Produtos.Remove(product);
        // A FK restrita barra a exclusão se uma movimentação entrar entre a verificação e o DELETE.
        try { await db.SaveChangesAsync(); }
        catch (DbUpdateException) { throw ErroNegocio.Conflito(ComHistorico); }
        return true;
    }

    private static void Aplicar(Produto product, ProdutoRequest request)
    {
        product.PartNumber = request.PartNumber;
        product.NomeProduto = request.NomeProduto.Trim();
        product.Categoria = request.Categoria.Trim();
        product.Unit = request.Unit;
        product.Location = request.Location?.Trim() ?? "";
        product.MinimumStock = request.MinimumStock;
    }
}
```

Arquivo: Services/SenhaHasher.cs

```
using System.Security.Cryptography;

namespace PimEstoque.Services;

/// <summary>PBKDF2-SHA256 no formato <c>iteracoes.saltBase64.hashBase64</c>.</summary>
public static class SenhaHasher
{
    private const int Iteracoes = 100_000;
    private const int TamanhoSalt = 16;
    private const int TamanhoHash = 32;

    public static string Gerar(string senha)
    {
        var salt = RandomNumberGenerator.GetBytes(TamanhoSalt);
        var hash = Rfc2898DeriveBytes.Pbkdf2(senha, salt, Iteracoes, HashAlgorithmName.SHA256,
            TamanhoHash);
        return $"{Iteracoes}.{Convert.ToBase64String(salt)}.{Convert.ToBase64String(hash)}";
    }

    public static bool Verificar(string senha, string senhaHash)
    {
        var partes = senhaHash.Split('.');
        if (partes.Length != 3 || !int.TryParse(partes[0], out var iteracoes) || iteracoes < 1)
            return false;
        try
        {
            var salt = Convert.FromBase64String(partes[1]);
            var esperado = Convert.FromBase64String(partes[2]);
            if (esperado.Length == 0) return false;
            var obtido = Rfc2898DeriveBytes.Pbkdf2(senha, salt, iteracoes,
                HashAlgorithmName.SHA256, esperado.Length);
            return CryptographicOperations.FixedTimeEquals(obtido, esperado);
        }
        catch (FormatException) { return false; }
    }
}
```

Arquivo: Services/UsuarioService.cs

```
using Microsoft.EntityFrameworkCore;
using PimEstoque.Contracts;
using PimEstoque.Data;
using PimEstoque.Models;

namespace PimEstoque.Services;

public class UsuarioService(PimEstoqueContext db)
{
    private const string Duplicado = "E-mail já cadastrado.";

    public Task<List<UsuarioResponse>> ListarAsync() =>
        db.Usuarios.AsNoTracking().OrderBy(u => u.Nome).Select(u => new UsuarioResponse(u.Id,
            u.Nome, u.Email, u.Perfil, u.Ativo)).ToListAsync();
```

```
    public async Task<UsuarioResponse?> ObterAsync(int id) =>
        await db.Usuarios.AsNoTracking().SingleOrDefaultAsync(u => u.Id == id) is { } u ?
            UsuarioResponse.De(u) : null;

    public async Task<UsuarioResponse> CriarAsync(UsuarioRequest request)
    {
        if (string.IsNullOrEmpty(request.Senha)) throw ErroNegocio.Invalido("Informe a senha
            (mínimo de 8 caracteres).");
        var user = new Usuario { CriadoEmUtc = DateTime.UtcNow };
        db.Usuarios.Add(user);
        await SalvarAsync(user, request);
        return UsuarioResponse.De(user);
    }

    public async Task<UsuarioResponse?> EditarAsync(int id, UsuarioRequest request, int
        usuarioLogadoId)
    {
        var user = await db.Usuarios.SingleOrDefaultAsync(u => u.Id == id);
        if (user is null) return null;
        // Evita que o administrador perca o próprio acesso (sistema sem nenhum administrador
            ativo).
        if (id == usuarioLogadoId && (!request.Ativo || request.Perfil != Perfis.Administrador))
            throw ErroNegocio.Conflito("Você não pode desativar nem rebaixar o próprio usuário.");
        await SalvarAsync(user, request);
        return UsuarioResponse.De(user);
    }

    /// <summary>Cria o administrador de demonstração somente quando ainda não há
        usuários.</summary>
    public async Task GarantirAdminInicialAsync(string email, string senha)
    {
        if (await db.Usuarios.AnyAsync()) return;
        db.Usuarios.Add(new Usuario { Nome = "Administrador", Email =
            email.Trim().ToLowerInvariant(), SenhaHash = SenhaHasher.Gerar(senha), Perfil =
                Perfis.Administrador, CriadoEmUtc = DateTime.UtcNow });
        await db.SaveChangesAsync();
    }

    private async Task SalvarAsync(Usuario user, UsuarioRequest request)
    {
        var email = request.Email.Trim().ToLowerInvariant();
        if (await db.Usuarios.AnyAsync(u => u.Id != user.Id && u.Email == email)) throw
            ErroNegocio.Conflito(Duplicado);
        user.Nome = request.Nome.Trim();
        user.Email = email;
        user.Perfil = request.Perfil;
        user.Ativo = request.Ativo;
        if (!string.IsNullOrEmpty(request.Senha)) user.SenhaHash = SenhaHasher.Gerar(request.Senha);
        try { await db.SaveChangesAsync(); }
        catch (DbUpdateException e) when (ErroNegocio.EhDuplicidade(e)) { throw
            ErroNegocio.Conflito(Duplicado); }
    }
}
```

Arquivo: Data/PimEstoqueContext.cs

```
using Microsoft.EntityFrameworkCore;
using PimEstoque.Models;

namespace PimEstoque.Data;

public class PimEstoqueContext(DbContextOptions<PimEstoqueContext> options) : DbContext(options)
{
    public DbSet<Produto> Produtos => Set<Produto>();
    public DbSet<Movimentacao> Movimentacoes => Set<Movimentacao>();
    public DbSet<Usuario> Usuarios => Set<Usuario>();
    public DbSet<Fornecedor> Fornecedores => Set<Fornecedor>();

    protected override void OnModelCreating(ModelBuilder model)
    {
        // As triggers existem só no SQL Server; declará-las faz o EF não usar OUTPUT nos comandos
            (AuditoriaProdutos não é mapeada).
        var product = model.Entity<Produto>();
        product.ToTable(t => t.HasTrigger("trg_Produtos_Auditoria"));
        product.HasIndex(p => p.PartNumber).IsUnique();
        product.Property(p => p.PartNumber).HasMaxLength(8).IsRequired();
        product.Property(p => p.NomeProduto).HasMaxLength(100).IsRequired();
        product.Property(p => p.Categoria).HasMaxLength(60).IsRequired();
        product.Property(p => p.Unit).HasMaxLength(3).IsRequired();
        product.Property(p => p.Location).HasMaxLength(60).IsRequired();
```

```
        var movement = model.Entity<Movimentacao>();
        movement.ToTable(t => t.HasTrigger("trg_Movimentacoes_Imutavel"));
        movement.Property(m => m.Motivo).HasMaxLength(150).IsRequired();
        // datetime2 não guarda fuso; marca como UTC para o JSON sair com "Z".
        movement.Property(m => m.CriadaEmUtc).HasConversion(v => v, v => DateTime.SpecifyKind(v,
            DateTimeKind.Utc));
        movement.HasOne(m => m.Produto).WithMany().HasForeignKey(m =>
            m.ProdutoId).OnDelete(DeleteBehavior.Restrict);
        movement.HasOne(m => m.Usuario).WithMany().HasForeignKey(m =>
            m.UsuarioId).OnDelete(DeleteBehavior.Restrict);
        movement.HasOne(m => m.Fornecedor).WithMany().HasForeignKey(m =>
            m.FornecedorId).OnDelete(DeleteBehavior.Restrict);
        movement.HasIndex(m => new { m.ProdutoId, m.Id });

        var user = model.Entity<Usuario>();
        user.HasIndex(u => u.Email).IsUnique();
        user.Property(u => u.Nome).HasMaxLength(100).IsRequired();
        user.Property(u => u.Email).HasMaxLength(120).IsRequired();
        user.Property(u => u.SenhaHash).HasMaxLength(200).IsRequired();
        user.Property(u => u.Perfil).HasMaxLength(20).IsRequired();

        var supplier = model.Entity<Fornecedor>();
        supplier.HasIndex(f => f.Cnpj).IsUnique();
        supplier.Property(f => f.Nome).HasMaxLength(100).IsRequired();
        supplier.Property(f =>
            f.Cnpj).HasMaxLength(14).IsFixedLength().IsUnicode(false).IsRequired();
        supplier.Property(f => f.Telefone).HasMaxLength(20);
        supplier.Property(f => f.Email).HasMaxLength(120);
    }
}
```

Arquivo: Models/Fornecedor.cs

```
namespace PimEstoque.Models;

public class Fornecedor
{
    public int Id { get; set; }
    public string Nome { get; set; } = "";
    public string Cnpj { get; set; } = "";
    public string? Telefone { get; set; }
    public string? Email { get; set; }
    public bool Ativo { get; set; } = true;
}
```

Arquivo: Models/Movimentacao.cs

```
namespace PimEstoque.Models;

public class Movimentacao
{
    public long Id { get; set; }
    public int ProdutoId { get; set; }
    public Produto Produto { get; set; } = null!;
    public int Quantidade { get; set; }
    public string Motivo { get; set; } = "";
    public DateTime CriadaEmUtc { get; set; }
    public int? UsuarioId { get; set; }
    public Usuario? Usuario { get; set; }
    public int? FornecedorId { get; set; }
    public Fornecedor? Fornecedor { get; set; }
}
```

Arquivo: Models/Produto.cs

```
namespace PimEstoque.Models;

public class Produto
{
    public int Id { get; set; }
    public string PartNumber { get; set; } = "";
    public string NomeProduto { get; set; } = "";
    public string Categoria { get; set; } = "";
    public string Unit { get; set; } = "UN";
    public string Location { get; set; } = "";
    public int MinimumStock { get; set; }
    public int Saldo { get; set; }
}
```

Arquivo: Models/Usuario.cs

```
namespace PimEstoque.Models;
```

```
public class Usuario
{
    public int Id { get; set; }
    public string Nome { get; set; } = "";
    public string Email { get; set; } = "";
    public string SenhaHash { get; set; } = "";
    public string Perfil { get; set; } = Perfis.Consulta;
    public bool Ativo { get; set; } = true;
    public DateTime CriadoEmUtc { get; set; }
}

public static class Perfis
{
    public const string Administrador = "Administrador";
    public const string Operador = "Operador";
    public const string Consulta = "Consulta";
    public const string Escrita = Administrador + "," + Operador;
    public static readonly string[] Todos = [Administrador, Operador, Consulta];
}
```

Arquivo: Contracts/Requests.cs

```
using System.ComponentModel.DataAnnotations;
using PimEstoque.Models;

namespace PimEstoque.Contracts;

public sealed class ProdutoRequest : IValidatableObject
{
    [Required, StringLength(8, MinimumLength = 8), RegularExpression(@"^(47|48|84|86)[0-9]{6}$",
        ErrorMessage = "Use 8 dígitos com prefixo 47, 48, 84 ou 86.")]
    public string PartNumber { get; set; } = "";
    [Required, StringLength(100)] public string NomeProduto { get; set; } = "";
    [Required, StringLength(60)] public string Categoria { get; set; } = "";
    [Required] public string Unit { get; set; } = "UN";
    [StringLength(60)] public string Location { get; set; } = "";
    [Range(0, 999999999)] public int MinimumStock { get; set; }
    public IEnumerable<ValidationResult> Validate(ValidationContext context)
    {
        if (!new[] { "UN", "PC", "PAR", "CX", "PCT", "RL" }.Contains(Unit))
            yield return new ValidationResult("Unidade inválida.", new[] { nameof(Unit) });
    }
}

public sealed class MovimentacaoRequest
{
    [Range(1, int.MaxValue)] public int ProdutoId { get; set; }
    [Range(1, 999999999)] public int Quantidade { get; set; }
    [Required, RegularExpression("^(entrada|saida)$")] public string Tipo { get; set; } = "";
    [Required, StringLength(150)] public string Motivo { get; set; } = "";
    [Range(1, int.MaxValue)] public int? FornecedorId { get; set; }
}

public sealed class LoginRequest
{
    [Required, StringLength(120)] public string Email { get; set; } = "";
    [Required, StringLength(100)] public string Senha { get; set; } = "";
}

public sealed class FornecedorRequest
{
    [Required, StringLength(100)] public string Nome { get; set; } = "";
    [Required, RegularExpression("^[0-9]{14}$", ErrorMessage = "CNPJ deve ter 14 dígitos, sem
        pontuação.")] public string Cnpj { get; set; } = "";
    [StringLength(20)] public string? Telefone { get; set; }
    [StringLength(120), EmailAddress] public string? Email { get; set; }
    public bool Ativo { get; set; } = true;
}

public sealed class UsuarioRequest : IValidatableObject
{
    [Required, StringLength(100)] public string Nome { get; set; } = "";
    [Required, StringLength(120), EmailAddress] public string Email { get; set; } = "";
    // Obrigatória só na criação; na edição, vazia mantém a senha atual.
    [StringLength(100, MinimumLength = 8)] public string? Senha { get; set; }
    [Required] public string Perfil { get; set; } = Perfis.Consulta;
    public bool Ativo { get; set; } = true;
    public IEnumerable<ValidationResult> Validate(ValidationContext context)
    {
        if (!Perfis.Todos.Contains(Perfil))
```

```
            yield return new ValidationResult("Perfil inválido.", new[] { nameof(Perfil) });
    }
}
```

Arquivo: Contracts/Responses.cs

```
using PimEstoque.Models;

namespace PimEstoque.Contracts;

public sealed record UsuarioResponse(int Id, string Nome, string Email, string Perfil, bool Ativo)
{
    public static UsuarioResponse De(Usuario u) => new(u.Id, u.Nome, u.Email, u.Perfil, u.Ativo);
}

public sealed record LoginResponse(string Token, DateTime ExpiraEmUtc, UsuarioResponse Usuario);

public sealed record MovimentacaoResponse(long Id, int ProdutoId, int Quantidade, string Motivo,
    DateTime CriadaEmUtc, string? UsuarioNome, string? FornecedorNome);

public sealed record CategoriaResumo(string Categoria, int Produtos, long Saldo);

public sealed record DashboardResponse(int TotalProdutos, int ItensReposicao, long SaldoTotal, int
    MovimentacoesHoje, List<CategoriaResumo> PorCategoria);
```

APÊNDICE C – CÓDIGO-FONTE DO PAINEL WEB

Código-fonte integral do painel administrativo servido pela API a partir da pasta wwwroot (seção 5.5).

Arquivo: wwwroot/index.html

```
<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>PIM Estoque — Painel administrativo</title>
  <link rel="stylesheet" href="styles.css">
  <script src="app.js" defer></script>
</head>
<body>
  <a class="pular" href="#conteudo">Pular para o conteúdo</a>

  <header class="topo">
    <div class="marca">
      <strong>PIM Estoque</strong>
      <span>Fixar Manutenção — almoxarifado</span>
    </div>
    <div class="sessao" id="sessao" hidden>
      <span id="usuario-atual"></span>
      <button type="button" class="secundario" id="sair">Sair</button>
    </div>
  </header>

  <nav id="menu" aria-label="Seções do painel" hidden>
    <ul>
      <li><button type="button" data-view="painel">Painel</button></li>
      <li><button type="button" data-view="produtos">Materiais</button></li>
      <li><button type="button" data-view="movimentacoes">Movimentações</button></li>
      <li><button type="button" data-view="fornecedores">Fornecedores</button></li>
      <li data-admin><button type="button" data-view="usuarios">Usuários</button></li>
    </ul>
  </nav>

  <main id="conteudo" tabindex="-1">
    <p id="mensagem" role="status" aria-live="polite"></p>
    <noscript><p class="erro">O painel precisa de JavaScript habilitado.</p></noscript>

    <section id="login" class="cartao estreito" aria-labelledby="titulo-login">
      <h1 id="titulo-login" tabindex="-1">Entrar</h1>
      <form id="form-login">
        <label for="login-email">E-mail</label>
        <input id="login-email" name="email" type="email" autocomplete="username" required
            maxlength="120">
        <label for="login-senha">Senha</label>
        <input id="login-senha" name="senha" type="password" autocomplete="current-password"
            required maxlength="100">
        <button type="submit">Entrar</button>
      </form>
    </section>

    <section id="painel" class="view" aria-labelledby="titulo-painel" hidden>
      <h1 id="titulo-painel" tabindex="-1">Painel</h1>
      <dl class="indicadores">
        <div><dt>Materiais cadastrados</dt><dd id="ind-total">-</dd></div>
        <div><dt>Itens para repor</dt><dd id="ind-repor">-</dd></div>
        <div><dt>Saldo total (unidades)</dt><dd id="ind-saldo">-</dd></div>
        <div><dt>Movimentações hoje</dt><dd id="ind-hoje">-</dd></div>
      </dl>
      <div class="tabela">
        <table>
          <caption>Saldo por categoria</caption>
          <thead><tr><th scope="col">Categoria</th><th scope="col" class="num">Materiais</th><th
              scope="col" class="num">Saldo</th></tr></thead>
          <tbody id="tabela-categorias"></tbody>
        </table>
      </div>
    </section>

    <section id="produtos" class="view" aria-labelledby="titulo-produtos" hidden>
```

```
  <h1 id="titulo-produtos" tabindex="-1">Materiais</h1>
  <form id="form-produto" class="cartao grade" data-escrita>
    <fieldset>
      <legend>Novo material</legend>
      <input type="hidden" name="id">
      <div><label for="p-part">Part number</label>
        <input id="p-part" name="partNumber" required inputmode="numeric"
            pattern="(47|48|84|86)[0-9]{6}" maxlength="8" aria-describedby="p-part-ajuda">
        <small id="p-part-ajuda">8 dígitos, começando por 47, 48, 84 ou 86.</small></div>
      <div><label for="p-nome">Nome do material</label><input id="p-nome" name="nomeProduto"
          required maxlength="100"></div>
      <div><label for="p-cat">Categoria</label><input id="p-cat" name="categoria" required
          maxlength="60"></div>
      <div><label for="p-unit">Unidade</label>
        <select id="p-unit"
            name="unit"><option>UN</option><option>PC</option><option>PAR</option><option>CX</op
              tion><option>PCT</option><option>RL</option></select></div>
      <div><label for="p-loc">Localização</label><input id="p-loc" name="location"
          maxlength="60"></div>
      <div><label for="p-min">Estoque mínimo</label><input id="p-min" name="minimumStock"
          type="number" min="0" max="999999999" value="0" required></div>
    </fieldset>
    <div class="acoes"><button type="submit">Salvar material</button><button type="reset"
        class="secundario">Cancelar</button></div>
  </form>

  <form id="form-busca" class="filtros" role="search">
    <label for="busca">Buscar</label>
    <input id="busca" name="busca" type="search" placeholder="Part number, nome, categoria ou
        local">
    <label class="check"><input type="checkbox" name="reposicao"> Somente itens para
        repor</label>
    <button type="submit">Filtrar</button>
  </form>
  <div class="tabela">
    <table>
      <caption>Materiais cadastrados</caption>
      <thead><tr><th scope="col">Part number</th><th scope="col">Material</th><th
          scope="col">Categoria</th><th scope="col">Un.</th><th scope="col">Local</th><th
              scope="col" class="num">Mínimo</th><th scope="col" class="num">Saldo</th><th
                  scope="col">Situação</th><th scope="col" data-escrita>Ações</th></tr></thead>
      <tbody id="tabela-produtos"></tbody>
    </table>
  </div>
</section>

<section id="movimentacoes" class="view" aria-labelledby="titulo-mov" hidden>
  <h1 id="titulo-mov" tabindex="-1">Movimentações</h1>
  <form id="form-mov" class="cartao grade" data-escrita>
    <fieldset>
      <legend>Registrar entrada ou saída</legend>
      <div><label for="m-prod">Material</label><select id="m-prod" name="produtoId"
          required></select></div>
      <div><label for="m-tipo">Tipo</label>
        <select id="m-tipo" name="tipo"><option value="entrada">Entrada</option><option
            value="saida">Saída</option></select></div>
      <div><label for="m-qtd">Quantidade</label><input id="m-qtd" name="quantidade"
          type="number" min="1" max="999999999" required></div>
      <div><label for="m-motivo">Motivo</label><input id="m-motivo" name="motivo" required
          maxlength="150" placeholder="Ex.: NF 123, obra Condomínio Azul"></div>
      <div><label for="m-forn">Fornecedor (opcional)</label><select id="m-forn"
          name="fornecedorId"></select></div>
    </fieldset>
    <div class="acoes"><button type="submit">Registrar</button></div>
  </form>
  <div class="tabela">
    <table>
      <caption>100 movimentações mais recentes</caption>
      <thead><tr><th scope="col">Data</th><th scope="col">Material</th><th
          scope="col">Tipo</th><th scope="col" class="num">Quantidade</th><th
              scope="col">Motivo</th><th scope="col">Usuário</th><th
                  scope="col">Fornecedor</th></tr></thead>
      <tbody id="tabela-mov"></tbody>
    </table>
  </div>
</section>

<section id="fornecedores" class="view" aria-labelledby="titulo-forn" hidden>
  <h1 id="titulo-forn" tabindex="-1">Fornecedores</h1>
  <form id="form-forn" class="cartao grade" data-escrita>
    <fieldset>
      <legend>Novo fornecedor</legend>
```

```
          <input type="hidden" name="id">
          <div><label for="f-nome">Nome</label><input id="f-nome" name="nome" required
              maxlength="100"></div>
          <div><label for="f-cnpj">CNPJ (somente números)</label><input id="f-cnpj" name="cnpj"
              required inputmode="numeric" pattern="[0-9]{14}" maxlength="14"></div>
          <div><label for="f-tel">Telefone</label><input id="f-tel" name="telefone" type="tel"
              maxlength="20"></div>
          <div><label for="f-email">E-mail</label><input id="f-email" name="email" type="email"
              maxlength="120"></div>
          <div><label class="check"><input type="checkbox" name="ativo" checked> Ativo</label></div>
        </fieldset>
        <div class="acoes"><button type="submit">Salvar fornecedor</button><button type="reset"
            class="secundario">Cancelar</button></div>
      </form>
      <div class="tabela">
        <table>
          <caption>Fornecedores cadastrados</caption>
          <thead><tr><th scope="col">Nome</th><th scope="col">CNPJ</th><th
              scope="col">Telefone</th><th scope="col">E-mail</th><th scope="col">Situação</th><th
                  scope="col" data-escrita>Ações</th></tr></thead>
          <tbody id="tabela-forn"></tbody>
        </table>
      </div>
    </section>

    <section id="usuarios" class="view" aria-labelledby="titulo-usu" hidden>
      <h1 id="titulo-usu" tabindex="-1">Usuários</h1>
      <form id="form-usu" class="cartao grade">
        <fieldset>
          <legend>Novo usuário</legend>
          <input type="hidden" name="id">
          <div><label for="u-nome">Nome</label><input id="u-nome" name="nome" required
              maxlength="100"></div>
          <div><label for="u-email">E-mail</label><input id="u-email" name="email" type="email"
              required maxlength="120" autocomplete="off"></div>
          <div><label for="u-senha">Senha</label><input id="u-senha" name="senha" type="password"
              minlength="8" maxlength="100" autocomplete="new-password"
                  aria-describedby="u-senha-ajuda">
            <small id="u-senha-ajuda">Mínimo de 8 caracteres. Na edição, deixe em branco para
                manter a atual.</small></div>
          <div><label for="u-perfil">Perfil</label>
            <select id="u-perfil"
                name="perfil"><option>Operador</option><option>Consulta</option><option>Administrado
                    r</option></select></div>
          <div><label class="check"><input type="checkbox" name="ativo" checked> Ativo</label></div>
        </fieldset>
        <div class="acoes"><button type="submit">Salvar usuário</button><button type="reset"
            class="secundario">Cancelar</button></div>
      </form>
      <div class="tabela">
        <table>
          <caption>Usuários do sistema</caption>
          <thead><tr><th scope="col">Nome</th><th scope="col">E-mail</th><th
              scope="col">Perfil</th><th scope="col">Situação</th><th
                  scope="col">Ações</th></tr></thead>
          <tbody id="tabela-usu"></tbody>
        </table>
      </div>
    </section>
  </main>
</body>
</html>
```

Arquivo: wwwroot/app.js

```
'use strict';
// Painel administrativo do PIM Estoque: consome a mesma API REST do app mobile.

const $ = (seletor) => document.querySelector(seletor);
const estado = { usuario: null, produtos: [], fornecedores: [] };
const CHAVE_TOKEN = 'pimEstoqueToken';

// ---------- utilitários ----------

function el(tag, props = {}, ...filhos) {
  const elemento = document.createElement(tag);
  Object.assign(elemento, props);
  elemento.append(...filhos);
  return elemento;
}

function avisar(texto, erro = false) {
```

```
  const caixa = $('#mensagem');
  caixa.textContent = texto;
  caixa.classList.toggle('erro', erro);
}

function lerFormulario(form) {
  const dados = {};
  for (const campo of form.elements) {
    if (!campo.name) continue;
    if (campo.type === 'checkbox') dados[campo.name] = campo.checked;
    else if (campo.type === 'number') dados[campo.name] = campo.value === '' ? null :
        Number(campo.value);
    else dados[campo.name] = campo.value.trim();
  }
  return dados;
}

function preencherFormulario(form, objeto, legenda) {
  for (const campo of form.elements) {
    if (!campo.name) continue;
    const valor = objeto[campo.name];
    if (campo.type === 'checkbox') campo.checked = Boolean(valor);
    else campo.value = valor ?? '';
  }
  form.querySelector('legend').textContent = legenda;
  form.querySelector('input:not([type=hidden])').focus();
}

// Restaura a legenda de "novo" quando o formulário é limpo (botão Cancelar ou após salvar).
function prepararReset(form, legenda) {
  form.addEventListener('reset', () => {
    form.querySelector('legend').textContent = legenda;
    form.querySelector('input[name=id]')?.setAttribute('value', '');
  });
}

function dataLocal(iso) {
  // Datas do SQL Server chegam sem fuso; a API sempre grava em UTC.
  const texto = /Z|[+-]\d\d:\d\d$/.test(iso) ? iso : iso + 'Z';
  return new Date(texto).toLocaleString('pt-BR');
}

const numero = (valor) => Number(valor).toLocaleString('pt-BR');
const selo = (texto, classe = '') => el('span', { className: `selo ${classe}`, textContent: texto
    });
const celula = (conteudo, classe = '') => el('td', { className: classe }, conteudo ?? '');

function botao(texto, rotulo, acao, classe = 'secundario pequeno') {
  const b = el('button', { type: 'button', className: classe, textContent: texto });
  b.setAttribute('aria-label', rotulo);
  b.addEventListener('click', acao);
  return b;
}

function linhaVazia(tbody, colunas, texto) {
  tbody.replaceChildren(el('tr', {}, el('td', { colSpan: colunas, textContent: texto })));
}

// ---------- API ----------

async function api(caminho, { method = 'GET', body } = {}) {
  const token = sessionStorage.getItem(CHAVE_TOKEN);
  const headers = { Accept: 'application/json' };
  if (body !== undefined) headers['Content-Type'] = 'application/json';
  if (token) headers.Authorization = `Bearer ${token}`;
  const resposta = await fetch(`/api${caminho}`, { method, headers, body: body === undefined ?
      undefined : JSON.stringify(body) });
  if (resposta.status === 401 && caminho !== '/auth/login') {
    sair();
    throw new Error('Sessão expirada. Entre novamente.');
  }
  if (resposta.status === 403) throw new Error('Seu perfil não permite esta operação.');
  const dados = resposta.status === 204 ? null : await resposta.json().catch(() => null);
  if (!resposta.ok) throw new Error(mensagemDeErro(dados) || `Erro ${resposta.status}.`);
  return dados;
}

function mensagemDeErro(dados) {
  if (!dados) return '';
  if (dados.message) return dados.message;
  if (dados.errors) return Object.values(dados.errors).flat().join(' ');
```

```
  return dados.title || '';
}

// Executa uma ação mostrando sucesso ou erro na região aria-live.
async function executar(acao, sucesso) {
  try {
    await acao();
    if (sucesso) avisar(sucesso);
    return true;
  } catch (erro) {
    avisar(erro.message, true);
    return false;
  }
}

// ---------- sessão ----------

const podeEscrever = () => ['Administrador', 'Operador'].includes(estado.usuario?.perfil);
const ehAdmin = () => estado.usuario?.perfil === 'Administrador';

function entrarNoPainel(usuario) {
  estado.usuario = usuario;
  $('#usuario-atual').textContent = `${usuario.nome} (${usuario.perfil})`;
  $('#login').hidden = true;
  $('#sessao').hidden = false;
  $('#menu').hidden = false;
  document.querySelectorAll('[data-escrita]').forEach((e) => { e.hidden = !podeEscrever(); });
  document.querySelectorAll('[data-admin]').forEach((e) => { e.hidden = !ehAdmin(); });
  mostrar('painel');
}

function sair(mensagem) {
  sessionStorage.removeItem(CHAVE_TOKEN);
  estado.usuario = null;
  document.querySelectorAll('.view').forEach((v) => { v.hidden = true; });
  $('#menu').hidden = true;
  $('#sessao').hidden = true;
  $('#login').hidden = false;
  $('#form-login').reset();
  if (mensagem) avisar(mensagem);
  $('#titulo-login').focus();
}

$('#form-login').addEventListener('submit', async (evento) => {
  evento.preventDefault();
  const dados = lerFormulario(evento.target);
  await executar(async () => {
    const resposta = await api('/auth/login', { method: 'POST', body: { email: dados.email, senha:
        dados.senha } });
    sessionStorage.setItem(CHAVE_TOKEN, resposta.token);
    avisar('');
    entrarNoPainel(resposta.usuario);
  });
});

$('#sair').addEventListener('click', () => sair('Você saiu do sistema.'));

// ---------- navegação ----------

const carregadores = {
  painel: carregarPainel,
  produtos: carregarProdutos,
  movimentacoes: carregarMovimentacoes,
  fornecedores: carregarFornecedores,
  usuarios: carregarUsuarios,
};

async function mostrar(view) {
  document.querySelectorAll('.view').forEach((v) => { v.hidden = v.id !== view; });
  document.querySelectorAll('[data-view]').forEach((b) => {
    if (b.dataset.view === view) b.setAttribute('aria-current', 'page');
    else b.removeAttribute('aria-current');
  });
  $(`#${view} h1`).focus();
  await executar(carregadores[view]);
}

document.querySelectorAll('[data-view]').forEach((b) => b.addEventListener('click', () => {
  avisar('');
  mostrar(b.dataset.view);
}));
```

```
// ---------- painel ----------

async function carregarPainel() {
  const resumo = await api('/dashboard/resumo');
  $('#ind-total').textContent = numero(resumo.totalProdutos);
  $('#ind-repor').textContent = numero(resumo.itensReposicao);
  $('#ind-saldo').textContent = numero(resumo.saldoTotal);
  $('#ind-hoje').textContent = numero(resumo.movimentacoesHoje);
  const tbody = $('#tabela-categorias');
  if (!resumo.porCategoria.length) return linhaVazia(tbody, 3, 'Nenhum material cadastrado.');
  tbody.replaceChildren(...resumo.porCategoria.map((c) => el('tr', {},
    el('th', { scope: 'row', textContent: c.categoria }),
    celula(numero(c.produtos), 'num'),
    celula(numero(c.saldo), 'num'))));
}

// ---------- materiais ----------

const formProduto = $('#form-produto');
prepararReset(formProduto, 'Novo material');
const precisaRepor = (p) => p.minimumStock > 0 && p.saldo <= p.minimumStock;

async function carregarProdutos() {
  const filtro = lerFormulario($('#form-busca'));
  const parametros = new URLSearchParams();
  if (filtro.busca) parametros.set('busca', filtro.busca);
  if (filtro.reposicao) parametros.set('reposicao', 'true');
  const produtos = await api(`/produtos?${parametros}`);
  const tbody = $('#tabela-produtos');
  if (!produtos.length) return linhaVazia(tbody, 9, 'Nenhum material encontrado.');
  tbody.replaceChildren(...produtos.map((p) => {
    const acoes = el('div', { className: 'acoes' });
    if (podeEscrever()) acoes.append(botao('Editar', `Editar ${p.nomeProduto}`, () =>
        preencherFormulario(formProduto, p, `Editar material ${p.partNumber}`)));
    if (ehAdmin()) acoes.append(botao('Excluir', `Excluir ${p.nomeProduto}`, () =>
        excluirProduto(p), 'perigo pequeno'));
    const linha = el('tr', {},
      el('th', { scope: 'row', textContent: p.partNumber }),
      celula(p.nomeProduto), celula(p.categoria), celula(p.unit), celula(p.location),
      celula(numero(p.minimumStock), 'num'), celula(numero(p.saldo), 'num'),
      celula(precisaRepor(p) ? selo('Repor', 'repor') : selo('OK')));
    if (podeEscrever()) linha.append(celula(acoes));
    return linha;
  }));
}

async function excluirProduto(p) {
  if (!confirm(`Excluir o material ${p.partNumber} — ${p.nomeProduto}?`)) return;
  if (await executar(() => api(`/produtos/${p.id}`, { method: 'DELETE' }), 'Material excluído.'))
      await executar(carregarProdutos);
}

formProduto.addEventListener('submit', async (evento) => {
  evento.preventDefault();
  const { id, ...dados } = lerFormulario(formProduto);
  const ok = await executar(() => api(id ? `/produtos/${id}` : '/produtos', { method: id ? 'PUT' :
      'POST', body: dados }),
    id ? 'Material atualizado.' : 'Material cadastrado.');
  if (ok) { formProduto.reset(); await executar(carregarProdutos); }
});

$('#form-busca').addEventListener('submit', (evento) => {
  evento.preventDefault();
  executar(carregarProdutos);
});

// ---------- movimentações ----------

async function carregarMovimentacoes() {
  const [produtos, fornecedores, movimentacoes] = await Promise.all([api('/produtos'),
      api('/fornecedores'), api('/movimentacoes')]);
  estado.produtos = produtos;
  const nomes = new Map(produtos.map((p) => [p.id, `${p.partNumber} — ${p.nomeProduto}`]));

  const selProduto = $('#m-prod');
  selProduto.replaceChildren(el('option', { value: '', textContent: 'Selecione…' }),
    ...produtos.map((p) => el('option', { value: p.id, textContent: `${nomes.get(p.id)} (saldo
        ${numero(p.saldo)} ${p.unit})` })));
  $('#m-forn').replaceChildren(el('option', { value: '', textContent: 'Nenhum' }),
    ...fornecedores.filter((f) => f.ativo).map((f) => el('option', { value: f.id, textContent:
```

```
        f.nome })));

  const tbody = $('#tabela-mov');
  if (!movimentacoes.length) return linhaVazia(tbody, 7, 'Nenhuma movimentação registrada.');
  tbody.replaceChildren(...movimentacoes.map((m) => el('tr', {},
    celula(dataLocal(m.criadaEmUtc)),
    celula(nomes.get(m.produtoId) ?? `#${m.produtoId}`),
    celula(m.quantidade > 0 ? 'Entrada' : 'Saída'),
    celula(numero(Math.abs(m.quantidade)), 'num'),
    celula(m.motivo), celula(m.usuarioNome ?? '—'), celula(m.fornecedorNome ?? '—'))));
}

$('#form-mov').addEventListener('submit', async (evento) => {
  evento.preventDefault();
  const form = evento.target;
  const dados = lerFormulario(form);
  const corpo = { produtoId: Number(dados.produtoId), quantidade: dados.quantidade, tipo:
      dados.tipo, motivo: dados.motivo, fornecedorId: dados.fornecedorId ?
          Number(dados.fornecedorId) : null };
  const ok = await executar(() => api('/movimentacoes', { method: 'POST', body: corpo }),
    `${dados.tipo === 'entrada' ? 'Entrada' : 'Saída'} registrada.`);
  if (ok) { form.reset(); await executar(carregarMovimentacoes); }
});

// ---------- fornecedores ----------

const formFornecedor = $('#form-forn');
prepararReset(formFornecedor, 'Novo fornecedor');

async function carregarFornecedores() {
  const fornecedores = await api('/fornecedores');
  const tbody = $('#tabela-forn');
  if (!fornecedores.length) return linhaVazia(tbody, 6, 'Nenhum fornecedor cadastrado.');
  tbody.replaceChildren(...fornecedores.map((f) => {
    const linha = el('tr', {},
      el('th', { scope: 'row', textContent: f.nome }),
      celula(f.cnpj), celula(f.telefone ?? '—'), celula(f.email ?? '—'),
      celula(f.ativo ? selo('Ativo') : selo('Inativo', 'inativo')));
    if (podeEscrever()) linha.append(celula(botao('Editar', `Editar ${f.nome}`, () =>
        preencherFormulario(formFornecedor, f, `Editar fornecedor ${f.nome}`))));
    return linha;
  }));
}

formFornecedor.addEventListener('submit', async (evento) => {
  evento.preventDefault();
  const { id, ...dados } = lerFormulario(formFornecedor);
  const ok = await executar(() => api(id ? `/fornecedores/${id}` : '/fornecedores', { method: id ?
      'PUT' : 'POST', body: dados }),
    id ? 'Fornecedor atualizado.' : 'Fornecedor cadastrado.');
  if (ok) { formFornecedor.reset(); await executar(carregarFornecedores); }
});

// ---------- usuários (somente Administrador) ----------

const formUsuario = $('#form-usu');
prepararReset(formUsuario, 'Novo usuário');

async function carregarUsuarios() {
  const usuarios = await api('/usuarios');
  $('#tabela-usu').replaceChildren(...usuarios.map((u) => el('tr', {},
    el('th', { scope: 'row', textContent: u.nome }),
    celula(u.email), celula(u.perfil),
    celula(u.ativo ? selo('Ativo') : selo('Inativo', 'inativo')),
    celula(botao('Editar', `Editar ${u.nome}`, () => preencherFormulario(formUsuario, { ...u,
        senha: '' }, `Editar usuário ${u.nome}`))))));
}

formUsuario.addEventListener('submit', async (evento) => {
  evento.preventDefault();
  const { id, senha, ...dados } = lerFormulario(formUsuario);
  const corpo = senha ? { ...dados, senha } : dados;
  const ok = await executar(() => api(id ? `/usuarios/${id}` : '/usuarios', { method: id ? 'PUT' :
      'POST', body: corpo }),
    id ? 'Usuário atualizado.' : 'Usuário cadastrado.');
  if (ok) { formUsuario.reset(); await executar(carregarUsuarios); }
});

// ---------- início ----------

(async () => {
```

```
  if (!sessionStorage.getItem(CHAVE_TOKEN)) return $('#login-email').focus();
  try { entrarNoPainel(await api('/auth/me')); } catch { sair(); }
})();
```

Arquivo: wwwroot/styles.css

```
:root {
  --fundo: #f4f6f8;
  --superficie: #ffffff;
  --texto: #1b2430;
  --texto-suave: #4a5565;
  --borda: #c9d1db;
  --primaria: #0b5cad;
  --primaria-escura: #084682;
  --foco: #1d4ed8;
  --erro: #a3191b;
  --erro-fundo: #fdecec;
  --ok: #17643a;
  --ok-fundo: #e7f5ec;
  --alerta: #7a4b00;
  --alerta-fundo: #fff3d6;
  font-family: system-ui, -apple-system, "Segoe UI", Roboto, Arial, sans-serif;
  font-size: 100%;
  line-height: 1.5;
  color: var(--texto);
}

*, *::before, *::after { box-sizing: border-box; }
[hidden] { display: none !important; }
body { margin: 0; background: var(--fundo); }

:focus-visible { outline: 3px solid var(--foco); outline-offset: 2px; }

.pular { position: absolute; left: -999px; top: 0; background: var(--superficie); padding: .5rem
    1rem; z-index: 10; }
.pular:focus { left: 1rem; }

.topo { display: flex; justify-content: space-between; align-items: center; gap: 1rem; flex-wrap:
    wrap; padding: .75rem 1.5rem; background: #12263f; color: #fff; }
.marca { display: flex; flex-direction: column; }
.marca strong { font-size: 1.25rem; }
.marca span { font-size: .875rem; color: #d4dde8; }
.sessao { display: flex; align-items: center; gap: .75rem; }
.topo :focus-visible { outline-color: #ffd166; }

nav ul { list-style: none; display: flex; flex-wrap: wrap; gap: .25rem; margin: 0; padding: .5rem
    1.5rem; background: var(--superficie); border-bottom: 1px solid var(--borda); }
nav button { background: transparent; color: var(--texto); border: 2px solid transparent; }
nav button:hover { background: var(--fundo); }
nav button[aria-current="page"] { border-color: var(--primaria); color: var(--primaria-escura);
    font-weight: 700; }

main { max-width: 72rem; margin: 0 auto; padding: 1rem 1.5rem 3rem; }
main:focus { outline: none; }
h1 { font-size: 1.5rem; margin: .5rem 0 1rem; }

#mensagem:empty { display: none; }
#mensagem { padding: .75rem 1rem; border-radius: 6px; border-left: 4px solid var(--ok); background:
    var(--ok-fundo); color: var(--ok); }
#mensagem.erro { border-color: var(--erro); background: var(--erro-fundo); color: var(--erro); }

.cartao { background: var(--superficie); border: 1px solid var(--borda); border-radius: 8px;
    padding: 1rem 1.25rem; margin-bottom: 1.25rem; }
.estreito { max-width: 24rem; margin: 2rem auto; }
.estreito form { display: flex; flex-direction: column; gap: .5rem; }

fieldset { border: 0; padding: 0; margin: 0; display: grid; grid-template-columns: repeat(auto-fit,
    minmax(14rem, 1fr)); gap: .75rem 1rem; }
legend { font-weight: 700; font-size: 1.1rem; margin-bottom: .5rem; padding: 0; }
fieldset > div { display: flex; flex-direction: column; gap: .25rem; }
label { font-weight: 600; }
small { color: var(--texto-suave); }
.check { display: flex; align-items: center; gap: .5rem; font-weight: 600; }
.check input { width: 1.25rem; height: 1.25rem; }

input, select { font: inherit; padding: .5rem .625rem; border: 1px solid #7b8794; border-radius:
    6px; background: #fff; color: var(--texto); min-height: 2.5rem; }

button { font: inherit; font-weight: 600; cursor: pointer; padding: .5rem 1rem; min-height: 2.5rem;
    border-radius: 6px; border: 2px solid var(--primaria); background: var(--primaria); color:
        #fff; }
```

```
button:hover { background: var(--primaria-escura); border-color: var(--primaria-escura); }
button.secundario { background: #fff; color: var(--primaria-escura); }
button.secundario:hover { background: var(--fundo); }
button.perigo { background: #fff; color: var(--erro); border-color: var(--erro); }
button.pequeno { padding: .25rem .625rem; min-height: 2rem; font-size: .875rem; }
.acoes { display: flex; gap: .5rem; margin-top: 1rem; flex-wrap: wrap; }

.filtros { display: flex; flex-wrap: wrap; align-items: center; gap: .5rem 1rem; margin-bottom:
    1rem; }
.filtros input[type="search"] { flex: 1 1 16rem; }

.indicadores { display: grid; grid-template-columns: repeat(auto-fit, minmax(12rem, 1fr)); gap:
    1rem; margin: 0 0 1.5rem; }
.indicadores div { background: var(--superficie); border: 1px solid var(--borda); border-radius:
    8px; padding: 1rem; }
.indicadores dt { color: var(--texto-suave); font-weight: 600; }
.indicadores dd { margin: .25rem 0 0; font-size: 2rem; font-weight: 700; }

.tabela { overflow-x: auto; background: var(--superficie); border: 1px solid var(--borda);
    border-radius: 8px; }
table { width: 100%; border-collapse: collapse; }
caption { text-align: left; font-weight: 700; padding: .75rem 1rem; }
th, td { text-align: left; padding: .5rem 1rem; border-top: 1px solid var(--borda); vertical-align:
    top; }
thead th { background: #eef2f6; }
.num { text-align: right; font-variant-numeric: tabular-nums; }
td .acoes { margin: 0; }

.selo { display: inline-block; padding: 0 .5rem; border-radius: 999px; font-size: .875rem;
    font-weight: 600; background: var(--ok-fundo); color: var(--ok); }
.selo.repor { background: var(--alerta-fundo); color: var(--alerta); }
.selo.inativo { background: #eceff3; color: var(--texto-suave); }

@media (prefers-reduced-motion: reduce) { * { transition: none !important; animation: none
    !important; } }
```

APÊNDICE D – CÓDIGO-FONTE DO APLICATIVO MOBILE (FLUTTER)

Código-fonte integral do aplicativo descrito na seção 6, escrito em Dart: inicialização, núcleo (cliente HTTP, autenticação e tema) e funcionalidades.

Arquivo: mobile/lib/main.dart

```
import 'package:flutter/material.dart';

import 'app.dart';

void main() => runApp(const PimEstoqueApp());
```

Arquivo: mobile/lib/app.dart

```
import 'package:flutter/material.dart';

import 'core/app_theme.dart';
import 'core/api_client.dart';
import 'core/auth.dart';
import 'core/auth_storage.dart';
import 'features/home/home_page.dart';
import 'features/products/products_page.dart';
import 'features/products/product_catalog.dart';
import 'features/products/product_storage.dart';
import 'features/products/stock_page.dart';

class PimEstoqueApp extends StatelessWidget {
  const PimEstoqueApp({super.key, this.catalog});

  final ProductCatalog? catalog;

  @override
  Widget build(BuildContext context) => MaterialApp(
    title: 'PIM Estoque',
    debugShowCheckedModeBanner: false,
    theme: AppTheme.light,
    home: catalog != null ? AppShell(catalog: catalog) : const LoginPage(),
  );
}

class LoginPage extends StatefulWidget {
  const LoginPage({super.key});
  @override
  State<LoginPage> createState() => _LoginPageState();
}

class _LoginPageState extends State<LoginPage> {
  final _form = GlobalKey<FormState>();
  final _email = TextEditingController();
  final _password = TextEditingController();
  final _server = TextEditingController(text: defaultApiUrl());
  final _storage = createAuthStorage();
  String? _error;
  bool _loading = false;

  @override
  void initState() {
    super.initState();
    _restore();
  }

  Future<void> _restore() async {
    final value = await _storage.read();
    if (!mounted || value == null) return;
    try {
      _open(LoginResult.decode(value));
    } catch (_) {
      await _storage.clear();
    }
  }

  Future<void> _login() async {
    if (!_form.currentState!.validate() || _loading) return;
    setState(() {
      _loading = true;
```

```
    _error = null;
  });
  try {
    final server = _server.text.trim();
    final login = await ApiClient(baseUrl: server)
        .login(_email.text.trim(), _password.text);
    final session = LoginResult(
      token: login.token,
      user: login.user,
      server: server,
    );
    await _storage.write(session.encode());
    if (mounted) _open(session);
  } on ApiException catch (error) {
    if (mounted) {
      setState(
        () => _error = error.statusCode == 401
            ? 'Email ou senha inválidos.'
            : 'Não foi possível conectar à API.',
      );
    }
  } catch (_) {
    if (mounted) setState(() => _error = 'Não foi possível conectar à API.');
  } finally {
    if (mounted) setState(() => _loading = false);
  }
}

void _open(LoginResult session) => Navigator.of(context).pushReplacement(
  MaterialPageRoute(builder: (_) => AppShell(session: session)),
);
void _demo() =>
    Navigator.of(context)
        .pushReplacement(MaterialPageRoute(builder: (_) => const AppShell()));
@override
void dispose() {
  _email.dispose();
  _password.dispose();
  _server.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) => Scaffold(
  body: Center(
    child: SingleChildScrollView(
      padding: const EdgeInsets.all(24),
      child: ConstrainedBox(
        constraints: const BoxConstraints(maxWidth: 420),
        child: Form(
          key: _form,
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              Text(
                'PIM Estoque',
                style: Theme.of(context).textTheme.headlineMedium,
              ),
              const SizedBox(height: 8),
              const Text('Entre para acessar o estoque sincronizado.'),
              const SizedBox(height: 24),
              TextFormField(
                key: const Key('loginEmailField'),
                controller: _email,
                decoration: const InputDecoration(labelText: 'Email'),
                validator: (v) => v == null || !v.contains('@')
                    ? 'Informe um email válido.'
                    : null,
              ),
              const SizedBox(height: 12),
              TextFormField(
                key: const Key('loginPasswordField'),
                controller: _password,
                obscureText: true,
                decoration: const InputDecoration(labelText: 'Senha'),
                validator: (v) =>
                    v == null || v.isEmpty ? 'Informe a senha.' : null,
              ),
              const SizedBox(height: 12),
              TextFormField(
                key: const Key('loginServerField'),
                controller: _server,
```

```
                  keyboardType: TextInputType.url,
                  decoration: const InputDecoration(
                    labelText: 'Servidor da API',
                    helperText: 'Produção: https://estoque.vinnisantos.com.br',
                  ),
                  validator: (v) =>
                      Uri.tryParse(v?.trim() ?? '')?.hasAuthority == true &&
                          v!.trim().startsWith('http')
                      ? null
                      : 'Informe http://endereço:porta.',
                ),
                const SizedBox(height: 16),
                if (_error != null)
                  Text(
                    _error!,
                    style: TextStyle(
                      color: Theme.of(context).colorScheme.error,
                    ),
                  ),
                const SizedBox(height: 8),
                FilledButton(
                  key: const Key('loginButton'),
                  onPressed: _loading ? null : _login,
                  child: Text(_loading ? 'Entrando…' : 'Entrar'),
                ),
                TextButton(
                  key: const Key('demoButton'),
                  onPressed: _loading ? null : _demo,
                  child: const Text('Usar sem servidor (demonstração)'),
                ),
              ],
            ),
          ),
        ),
      ),
    ),
  );
}

class AppShell extends StatefulWidget {
  const AppShell({super.key, this.catalog, this.session});

  final ProductCatalog? catalog;
  final LoginResult? session;

  @override
  State<AppShell> createState() => _AppShellState();
}

class _AppShellState extends State<AppShell> {
  int _index = 0;
  late final ProductCatalog _catalog;

  @override
  void initState() {
    super.initState();
    _catalog =
        widget.catalog ??
        ProductCatalog(
          createProductStorage(),
          api: widget.session == null
              ? null
              : ApiClient(
                  baseUrl: widget.session!.server,
                  token: widget.session!.token,
                  onUnauthorized: _logout,
                ),
        );
    _catalog.canWrite =
        widget.session == null || widget.session!.user.profile != 'Consulta';
    _catalog.load().then((_) {
      if (widget.session != null) _catalog.sync();
    });
  }

  @override
  void dispose() {
    if (widget.catalog == null) _catalog.dispose();
    super.dispose();
  }
```

```
Widget _content() {
  if (_catalog.isLoading) {
    return const Center(child: CircularProgressIndicator());
  }
  if (!_catalog.isReady) {
    return Center(
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisSize: MainAxisSize.min,
          children: [
            Text(_catalog.loadError ?? 'Catálogo indisponível.'),
            const SizedBox(height: 16),
            FilledButton(
              onPressed: _catalog.load,
              child: const Text('Tentar novamente'),
            ),
          ],
        ),
      ),
    );
  }
  return IndexedStack(
    index: _index,
    children: [
      HomePage(
        products: _catalog.products,
        catalog: _catalog,
        onOpenProducts: () => setState(() => _index = 1),
        onOpenStock: () => setState(() => _index = 2),
      ),
      ProductsPage(catalog: _catalog),
      StockPage(catalog: _catalog),
    ],
  );
}

Future<void> _logout() async {
  await createAuthStorage().clear();
  if (!mounted) return;
  Navigator.of(context).pushAndRemoveUntil(
    MaterialPageRoute(builder: (_) => const LoginPage()),
    (_) => false,
  );
}

@override
Widget build(BuildContext context) => AnimatedBuilder(
  animation: _catalog,
  builder: (context, _) => Scaffold(
    appBar: AppBar(
      title: const Text('PIM Estoque'),
      actions: [
        if (widget.session != null)
          IconButton(
            onPressed: _logout,
            tooltip: 'Sair',
            icon: const Icon(Icons.logout),
          ),
        Padding(
          padding: EdgeInsets.only(right: 16),
          child: Chip(label: Text('NOV • 2026')),
        ),
      ],
    ),
    body: Row(
      children: [
        if (MediaQuery.sizeOf(context).width >= 900) ...[
          NavigationRail(
            extended: true,
            selectedIndex: _index,
            onDestinationSelected: (value) => setState(() => _index = value),
            leading: const Padding(
              padding: EdgeInsets.all(20),
              child: Text('ALMOXARIFADO'),
            ),
            destinations: const [
              NavigationRailDestination(
                icon: Icon(Icons.space_dashboard_outlined),
                label: Text('Início'),
              ),
              NavigationRailDestination(
```

```
                  icon: Icon(Icons.inventory_2_outlined),
                  label: Text('Produtos'),
                ),
                NavigationRailDestination(
                  icon: Icon(Icons.swap_vert),
                  label: Text('Estoque'),
                ),
              ],
            ),
            const VerticalDivider(width: 1),
          ],
          Expanded(
            child: SafeArea(
              child: Column(
                children: [
                  Container(
                    width: double.infinity,
                    padding: const EdgeInsets.symmetric(
                      horizontal: 20,
                      vertical: 10,
                    ),
                    color: Theme.of(context).colorScheme.secondaryContainer,
                    child: Text(
                      widget.session == null
                          ? 'Demonstração • catálogo local, sem conexão com a API.'
                          : 'API conectada • ${widget.session!.user.name}
                              (${widget.session!.user.profile})',
                    ),
                  ),
                  Expanded(child: _content()),
                ],
              ),
            ),
          ),
        ],
      ),
      bottomNavigationBar: MediaQuery.sizeOf(context).width >= 900
          ? null
          : NavigationBar(
              selectedIndex: _index,
              onDestinationSelected: (value) => setState(() => _index = value),
              destinations: const [
                NavigationDestination(
                  icon: Icon(Icons.space_dashboard_outlined),
                  selectedIcon: Icon(Icons.space_dashboard),
                  label: 'Início',
                ),
                NavigationDestination(
                  icon: Icon(Icons.inventory_2_outlined),
                  selectedIcon: Icon(Icons.inventory_2),
                  label: 'Produtos',
                ),
                NavigationDestination(
                  icon: Icon(Icons.swap_vert),
                  label: 'Estoque',
                ),
              ],
            ),
    ),
  );
}
```

Arquivo: mobile/lib/core/api_client.dart

```
import 'dart:convert';
import 'dart:async';

import 'package:http/http.dart' as http;

import 'auth.dart';
import '../features/products/product.dart';
import '../features/products/stock_movement.dart';

String defaultApiUrl() => const String.fromEnvironment(
  'API_URL',
  defaultValue: 'https://estoque.vinnisantos.com.br',
);

class ApiException implements Exception {
  const ApiException(this.statusCode, this.message);
  final int statusCode;
  final String message;
```

```
}

class ApiClient {
  ApiClient({
    http.Client? client,
    String? baseUrl,
    this.token,
    this.onUnauthorized,
  }) : _client = client ?? http.Client(),
       baseUrl = (baseUrl ?? defaultApiUrl()).replaceFirst(RegExp(r'/$'), '');

  final http.Client _client;
  final String baseUrl;
  final String? token;
  final FutureOr<void> Function()? onUnauthorized;

  Map<String, String> get _headers => {
    'Accept': 'application/json',
    'Content-Type': 'application/json',
    if (token != null) 'Authorization': 'Bearer $token',
  };

  Future<dynamic> _request(String method, String path, {Object? body}) async {
    final uri = Uri.parse('$baseUrl$path');
    final request = http.Request(method, uri)..headers.addAll(_headers);
    if (body != null) request.body = jsonEncode(body);
    final response = await _client.send(request).then(http.Response.fromStream);
    if (response.statusCode == 401) {
      onUnauthorized?.call();
      throw const ApiException(401, 'Sessão expirada.');
    }
    if (response.statusCode < 200 || response.statusCode >= 300) {
      throw ApiException(response.statusCode, response.body);
    }
    return response.body.isEmpty ? null : jsonDecode(response.body);
  }

  Future<LoginResult> login(String email, String password) async =>
      LoginResult.fromJson(
        await _request(
          'POST',
          '/api/auth/login',
          body: {'email': email, 'senha': password},
        ) as Map<String, dynamic>,
      );

  Future<List<Product>> products({String search = ''}) async {
    final suffix = search.trim().isEmpty
        ? ''
        : '?busca=${Uri.encodeQueryComponent(search)}';
    final data = await _request('GET', '/api/produtos$suffix') as List;
    return data
        .map((row) => Product.fromJson(row as Map<String, dynamic>))
        .toList();
  }

  Future<Product> product(int id) async => Product.fromJson(
    await _request('GET', '/api/produtos/$id') as Map<String, dynamic>,
  );

  Future<Product> saveProduct(Product product, {bool update = false}) async {
    final data = await _request(
      update ? 'PUT' : 'POST',
      update ? '/api/produtos/${product.id}' : '/api/produtos',
      body: product.toApiJson(),
    );
    return data is Map<String, dynamic> ? Product.fromJson(data) : product;
  }

  Future<StockMovement> createMovement(StockMovement movement) async =>
      StockMovement.fromJson(
        await _request('POST', '/api/movimentacoes', body: movement.toApiJson())
            as Map<String, dynamic>,
      );

  Future<List<StockMovement>> movements(int productId) async {
    final data = await _request(
      'GET',
      '/api/movimentacoes?produtoId=$productId',
    ) as List;
    return data
```

```
        .map((row) => StockMovement.fromJson(row as Map<String, dynamic>))
        .toList();
  }

  Future<Map<String, dynamic>> dashboard() async =>
      await _request('GET', '/api/dashboard/resumo') as Map<String, dynamic>;
}
```

Arquivo: mobile/lib/core/app_theme.dart

```
import 'package:flutter/material.dart';

abstract final class AppTheme {
  static ThemeData get light => ThemeData(
    useMaterial3: true,
    colorScheme: ColorScheme.fromSeed(seedColor: const Color(0xFF176B5B)),
    scaffoldBackgroundColor: const Color(0xFFF5F7F8),
    appBarTheme: const AppBarTheme(
      backgroundColor: Color(0xFFF5F7F8),
      centerTitle: false,
    ),
    inputDecorationTheme: const InputDecorationTheme(
      filled: true,
      fillColor: Colors.white,
      border: OutlineInputBorder(
        borderRadius: BorderRadius.all(Radius.circular(16)),
      ),
    ),
  );
}
```

Arquivo: mobile/lib/core/auth.dart

```
import 'dart:convert';

class AuthUser {
  const AuthUser({
    required this.id,
    required this.name,
    required this.email,
    required this.profile,
  });

  final int id;
  final String name;
  final String email;
  final String profile;

  factory AuthUser.fromJson(Map<String, dynamic> json) => AuthUser(
    id: json['id'] as int,
    name: (json['nome'] ?? json['name']) as String,
    email: json['email'] as String,
    profile: (json['perfil'] ?? json['profile']) as String,
  );

  Map<String, Object> toJson() => {
    'id': id,
    'nome': name,
    'email': email,
    'perfil': profile,
  };
}

class LoginResult {
  const LoginResult({required this.token, required this.user, this.server});

  final String token;
  final AuthUser user;

  /// Endereço da API usado no login; nulo usa o padrão de compilação.
  final String? server;

  factory LoginResult.fromJson(Map<String, dynamic> json) {
    final token = json['token'];
    final user = json['usuario'];
    if (token is! String || token.isEmpty || user is! Map<String, dynamic>) {
      throw const FormatException('Resposta de login inválida.');
    }
    return LoginResult(
      token: token,
      user: AuthUser.fromJson(user),
```

```
      server: json['servidor'] as String?,
    );
  }

  String encode() => jsonEncode({
    'token': token,
    'usuario': user.toJson(),
    if (server != null) 'servidor': server,
  });

  factory LoginResult.decode(String value) =>
      LoginResult.fromJson(jsonDecode(value) as Map<String, dynamic>);
}
```

Arquivo: mobile/lib/core/auth_storage.dart

```
import 'auth_storage_io.dart'
    if (dart.library.js_interop) 'auth_storage_web.dart';

abstract interface class AuthStorage {
  Future<String?> read();
  Future<void> write(String value);
  Future<void> clear();
}

AuthStorage createAuthStorage() => createPlatformAuthStorage();
```

Arquivo: mobile/lib/core/auth_storage_io.dart

```
import 'dart:io';

import 'package:path_provider/path_provider.dart';

import 'auth_storage.dart';

AuthStorage createPlatformAuthStorage() => _FileAuthStorage();

class _FileAuthStorage implements AuthStorage {
  Future<File> get _file async {
    final directory = await getApplicationDocumentsDirectory();
    await directory.create(recursive: true);
    return File('${directory.path}/pim_auth_v1.json');
  }

  @override
  Future<String?> read() async {
    final file = await _file;
    return file.existsSync() ? file.readAsString() : null;
  }

  @override
  Future<void> write(String value) async => (await _file).writeAsString(value);

  @override
  Future<void> clear() async {
    final file = await _file;
    if (file.existsSync()) await file.delete();
  }
}
```

Arquivo: mobile/lib/core/auth_storage_web.dart

```
import 'package:web/web.dart' as web;

import 'auth_storage.dart';

AuthStorage createPlatformAuthStorage() => _BrowserAuthStorage();

class _BrowserAuthStorage implements AuthStorage {
  static const key = 'pim_estoque_auth_v1';
  @override
  Future<String?> read() async => web.window.localStorage.getItem(key);
  @override
  Future<void> write(String value) async =>
      web.window.localStorage.setItem(key, value);
  @override
  Future<void> clear() async => web.window.localStorage.removeItem(key);
}
```

Arquivo: mobile/lib/features/home/home_page.dart

```
import 'package:flutter/material.dart';

import '../products/product.dart';
import '../products/product_catalog.dart';

class HomePage extends StatelessWidget {
  const HomePage({
    super.key,
    required this.onOpenProducts,
    required this.onOpenStock,
    required this.products,
    required this.catalog,
  });
  final VoidCallback onOpenProducts;
  final VoidCallback onOpenStock;
  final List<Product> products;
  final ProductCatalog catalog;

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    final categories = products.map((p) => p.category).toSet().length;
    final noStock = products.where((p) => catalog.balance(p.id) == 0).length;
    return ListView(
      padding: const EdgeInsets.all(20),
      children: [
        Text(
          'Visão geral do almoxarifado',
          style: theme.textTheme.headlineMedium,
        ),
        const SizedBox(height: 8),
        Text('${products.length} produtos • $categories categorias'),
        const SizedBox(height: 16),
        Wrap(
          spacing: 12,
          runSpacing: 12,
          children: [
            FilledButton.icon(
              onPressed: onOpenStock,
              icon: const Icon(Icons.swap_vert),
              label: const Text('Nova movimentação'),
            ),
            OutlinedButton.icon(
              onPressed: onOpenProducts,
              icon: const Icon(Icons.inventory_2_outlined),
              label: const Text('Consultar produtos'),
            ),
          ],
        ),
        const SizedBox(height: 24),
        LayoutBuilder(
          builder: (context, constraints) {
            final columns = constraints.maxWidth >= 850
                ? 4
                : constraints.maxWidth >= 480
                ? 2
                : 1;
            final width = (constraints.maxWidth - (columns - 1) * 12) / columns;
            return Wrap(
              spacing: 12,
              runSpacing: 12,
              children: [
                _indicator(
                  context,
                  width,
                  'Materiais cadastrados',
                  products.length,
                  Icons.inventory_2_outlined,
                  theme.colorScheme.primary,
                ),
                _indicator(
                  context,
                  width,
                  'Categorias',
                  categories,
                  Icons.category_outlined,
                  const Color(0xFF3D5A96),
                ),
                _indicator(
                  context,
```

```
          width,
          'Materiais sem saldo',
          noStock,
          Icons.remove_circle_outline,
          const Color(0xFF88651B),
        ),
        _indicator(
          context,
          width,
          'Precisam de reposição',
          catalog.replenishment.length,
          Icons.notification_important_outlined,
          const Color(0xFF9F403E),
        ),
      ],
    );
  },
),
const SizedBox(height: 24),
Card(
  child: Padding(
    padding: const EdgeInsets.all(20),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          'Reposição: ${catalog.replenishment.length} material(is)',
          style: theme.textTheme.titleLarge,
        ),
        const SizedBox(height: 8),
        const Text(
          'Saldo igual ou abaixo do mínimo. Mínimo zero desativa o alerta.',
        ),
        if (catalog.replenishment.isEmpty)
          const Padding(
            padding: EdgeInsets.symmetric(vertical: 16),
            child: Text(
              'Nenhuma reposição indicada pelos mínimos configurados.',
            ),
          ),
        for (final product in catalog.replenishment)
          ListTile(
            contentPadding: EdgeInsets.zero,
            title: Text(product.name),
            subtitle: Text(
              '${product.partNumber} • ${product.location.isEmpty ? "Sem localização" :
                  product.location}',
            ),
            trailing: Text(
              '${catalog.balance(product.id)} / ${product.minimumStock} ${product.unit}',
            ),
          ),
        TextButton.icon(
          onPressed: onOpenStock,
          icon: const Icon(Icons.arrow_forward),
          label: const Text('Abrir controle de estoque'),
        ),
      ],
    ),
  ),
),
const SizedBox(height: 16),
Card(
  child: Padding(
    padding: const EdgeInsets.all(20),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text('Atividade recente', style: theme.textTheme.titleLarge),
        const SizedBox(height: 8),
        Text(
          '${catalog.movements.length} movimentação(ões) registrada(s)',
        ),
        if (catalog.movements.isEmpty)
          const Padding(
            padding: EdgeInsets.symmetric(vertical: 12),
            child: Text(
              'Registre a primeira entrada para iniciar o saldo dos materiais.',
            ),
          ),
        for (final movement in catalog.movements.take(5))
          ListTile(
```

```
                    contentPadding: EdgeInsets.zero,
                    leading: Icon(
                      movement.quantity > 0
                          ? Icons.south_west
                          : Icons.north_east,
                    ),
                    title: Text(
                      '${products.firstWhere((p) => p.id == movement.productId).partNumber} •
                          ${movement.quantity > 0 ? "Entrada" : "Saída"}',
                    ),
                    subtitle: Text(movement.reason),
                    trailing: Text(
                      '${movement.quantity.abs()} ${products.firstWhere((p) => p.id ==
                          movement.productId).unit}',
                    ),
                  ),
              ],
            ),
          ),
        ),
        const SizedBox(height: 16),
        const Text(
          'Dados salvos neste navegador ou aparelho. Banco central e acesso por equipe ainda estão
              em desenvolvimento.',
        ),
      ],
    );
  }

  Widget _indicator(
    BuildContext context,
    double width,
    String label,
    int value,
    IconData icon,
    Color color,
  ) => SizedBox(
    width: width,
    child: Card(
      margin: EdgeInsets.zero,
      child: Padding(
        padding: const EdgeInsets.all(18),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Icon(icon, color: color),
            const SizedBox(height: 12),
            Text(
              '$value',
              style: Theme.of(context).textTheme.headlineMedium
                  ?.copyWith(color: color),
            ),
            const SizedBox(height: 6),
            Text(label),
          ],
        ),
      ),
    ),
  );
}
```

Arquivo: mobile/lib/features/products/demo_products.dart

```
import 'product.dart';

/// Materiais fictícios carregados somente quando ainda não existe catálogo salvo.
const demoProducts = <Product>[
  Product(
    id: 1,
    partNumber: '47000001',
    name: 'Filtro de óleo',
    category: 'Filtros',
  ),
  Product(
    id: 2,
    partNumber: '47000002',
    name: 'Filtro de ar',
    category: 'Filtros',
  ),
  Product(
    id: 3,
    partNumber: '48000001',
```

```
    name: 'Parafuso M10 × 30',
    category: 'Fixadores',
  ),
  Product(
    id: 4,
    partNumber: '48000002',
    name: 'Porca M10',
    category: 'Fixadores',
  ),
  Product(
    id: 5,
    partNumber: '48000003',
    name: 'Arruela M10',
    category: 'Fixadores',
  ),
  Product(
    id: 6,
    partNumber: '84000001',
    name: 'Rolamento 6204',
    category: 'Rolamentos',
  ),
  Product(
    id: 7,
    partNumber: '86000001',
    name: 'Correia de transmissão',
    category: 'Transmissão',
  ),
  Product(
    id: 8,
    partNumber: '86000002',
    name: 'Mangueira hidráulica',
    category: 'Hidráulica',
  ),
];
```

Arquivo: mobile/lib/features/products/product.dart

```
/// Cadastro de material do almoxarifado; campos novos aceitam registros legados.
class Product {
  const Product({
    required this.id,
    required this.partNumber,
    required this.name,
    required this.category,
    this.unit = 'UN',
    this.minimumStock = 0,
    this.location = '',
  });

  final int id;
  final String partNumber;
  final String name;
  final String category;
  static bool validPartNumber(String value) =>
      value.length == 8 && RegExp(r'^(47|48|84|86)[0-9]{6}$').hasMatch(value);
  bool get needsCodeReview => !validPartNumber(partNumber);
  static const units = ['UN', 'PC', 'PAR', 'CX', 'PCT', 'RL'];
  final String unit;
  final int minimumStock;
  final String location;

  factory Product.fromJson(Map<String, dynamic> json) {
    final id = json['id'];
    final partNumber = json['partNumber'];
    final name = json['nomeProduto'] ?? json['name'];
    final category = json['categoria'] ?? json['category'];
    final unit = json['unit'] ?? 'UN';
    final minimum = json['minimumStock'] ?? 0;
    final location = json['location'] ?? '';
    if (unit is! String ||
        !units.contains(unit) ||
        minimum is! int ||
        minimum < 0 ||
        minimum > 999999999 ||
        location is! String ||
        location.length > 60) {
      throw const FormatException('Dados de almoxarifado inválidos.');
    }
    if (id is! int ||
        id <= 0 ||
        partNumber is! String ||
        partNumber.trim().isEmpty ||
```

```
        partNumber.length > 50 ||
        name is! String ||
        name.trim().isEmpty ||
        name.length > 100 ||
        category is! String ||
        category.trim().isEmpty ||
        category.length > 60) {
      throw const FormatException('Material salvo inválido.');
    }
    return Product(
      id: id,
      partNumber: partNumber.trim().toUpperCase(),
      name: name.trim(),
      category: category.trim(),
      unit: unit,
      minimumStock: minimum,
      location: location.trim(),
    );
  }

  Map<String, Object> toJson() => {
    'id': id,
    'partNumber': partNumber,
    'nomeProduto': name,
    'categoria': category,
    'unit': unit,
    'minimumStock': minimumStock,
    'location': location,
  };

  Map<String, Object> toApiJson() => {
    'partNumber': partNumber,
    'nomeProduto': name,
    'categoria': category,
    'unit': unit,
    'location': location,
    'minimumStock': minimumStock,
  };

  bool matches(String query) {
    final term = query.trim().toLowerCase();
    return [
      partNumber,
      name,
      category,
      location,
    ].any((value) => value.toLowerCase().contains(term));
  }
}
```

Arquivo: mobile/lib/features/products/product_catalog.dart

```
import 'dart:convert';

import 'package:flutter/foundation.dart';

import 'demo_products.dart';
import 'product.dart';
import 'product_storage.dart';
import 'stock_movement.dart';
import '../../core/api_client.dart';

class ProductCatalog extends ChangeNotifier {
  ProductCatalog(this.storage, {this.api});

  final ProductStorage storage;
  final ApiClient? api;
  bool canWrite = true;
  final List<StockMovement> _pending = [];
  final List<StockMovement> _conflicts = [];
  int get pendingCount => _pending.length;
  List<StockMovement> get conflicts => List.unmodifiable(_conflicts);
  List<Product> _products = [];
  List<StockMovement> _movements = [];
  List<StockMovement> get movements => List.unmodifiable(_movements.reversed);
  int balance(int productId) => _movements
      .where((m) => m.productId == productId)
      .fold(0, (sum, m) => sum + m.quantity);
  bool needsReplenishment(Product product) =>
      product.minimumStock > 0 && balance(product.id) <= product.minimumStock;
  List<Product> get replenishment =>
```

```
    _products.where(needsReplenishment).toList();
List<Product> get products => List.unmodifiable(_products);
bool isLoading = false;
bool isSaving = false;
bool isReady = false;
String? loadError;
bool _disposed = false;

void _notify() {
  if (!_disposed) notifyListeners();
}

@override
void dispose() {
  _disposed = true;
  super.dispose();
}

Future<void> load() async {
  if (isLoading || isSaving) return;
  isLoading = true;
  isReady = false;
  loadError = null;
  _notify();
  try {
    final raw = await storage.read();
    final List<Product> loaded;
    List<StockMovement> loadedMovements = [];
    if (raw == null) {
      loaded = List.of(demoProducts);
      await storage.write(_encode(loaded));
    } else {
      final data = jsonDecode(raw);
      if (data is! Map<String, dynamic> ||
          data['version'] != 1 ||
          data['products'] is! List) {
        throw const FormatException('Formato de catálogo inválido.');
      }
      loaded = (data['products'] as List).map((row) {
        if (row is! Map<String, dynamic>) {
          throw const FormatException('Material inválido.');
        }
        final old = row['partNumber'];
        const demoCodes = {
          'PIM-1001': '47000001',
          'PIM-1002': '47000002',
          'PIM-2001': '48000001',
          'PIM-2002': '48000002',
          'PIM-2003': '48000003',
          'PIM-3001': '84000001',
          'PIM-4001': '86000001',
          'PIM-5001': '86000002',
        };
        final replacement = demoCodes[old];
        final collision =
            replacement != null &&
            (data['products'] as List).any(
              (other) => other is Map && other['partNumber'] == replacement,
            );
        // Manter o id, unidade e histórico; não criar colisão com códigos existentes.
        return Product.fromJson(
          replacement == null || collision
              ? row
              : {...row, 'partNumber': replacement},
        );
      }).toList();
      if (loaded.map((p) => p.id).toSet().length != loaded.length ||
          loaded.map((p) => p.partNumber.toUpperCase()).toSet().length !=
              loaded.length) {
        throw const FormatException('Materiais duplicados no arquivo.');
      }
      final rows = data['movements'] ?? [];
      if (rows is! List) throw const FormatException('Histórico inválido.');
      final running = <int, int>{};
      loadedMovements = rows.map((row) {
        if (row is! Map<String, dynamic>) {
          throw const FormatException('Histórico inválido.');
        }
        final movement = StockMovement.fromJson(row);
        if (!loaded.any((p) => p.id == movement.productId)) {
          throw const FormatException('Produto ausente.');
        }
```

```
        final saldo = (running[movement.productId] ?? 0) + movement.quantity;
        if (saldo < 0) {
          throw const FormatException('Saldo inválido no histórico.');
        }
        running[movement.productId] = saldo;
        return movement;
      }).toList();
      final pendingRows = data['pendingMovements'] ?? [];
      if (pendingRows is! List) throw const FormatException('Fila inválida.');
      _pending
        ..clear()
        ..addAll(
          pendingRows.map(
            (row) => StockMovement.fromJson(row as Map<String, dynamic>),
          ),
        );
    }
    _movements = loadedMovements;
    _products = loaded;
    isReady = true;
    if (api != null) {
      try {
        final remote = await api!.products();
        _products = remote;
        await api!.dashboard();
        await _persist();
      } catch (_) {
        // O cache local continua disponível sem conexão.
      }
    }
  } catch (_) {
    loadError =
        'Não foi possível abrir o catálogo local. '
        'Seus dados não foram substituídos. Tente novamente.';
  } finally {
    isLoading = false;
    _notify();
  }
}

String? validatePartNumber(String? value, {int? excludingId}) {
  final text = value?.trim() ?? '';
  if (text.isEmpty) return 'Informe o part number.';
  if (!Product.validPartNumber(text)) {
    return 'Use 8 dígitos, começando por 47, 48, 84 ou 86.';
  }
  if (_products.any(
    (p) =>
        p.id != excludingId &&
        p.partNumber.toUpperCase() == text.toUpperCase(),
  )) {
    return 'Já existe um material com este part number.';
  }
  return null;
}

Future<void> save({
  int? id,
  required String partNumber,
  required String name,
  required String category,
  String? unit,
  int? minimumStock,
  String? location,
}) async {
  if (!canWrite || !isReady || isLoading || isSaving) {
    throw StateError('Catálogo indisponível.');
  }
  final problem = validatePartNumber(partNumber, excludingId: id);
  if (problem != null) throw FormatException(problem);
  if (name.trim().isEmpty ||
      name.trim().length > 100 ||
      category.trim().isEmpty ||
      category.trim().length > 60) {
    throw const FormatException('Confira o nome e a categoria do material.');
  }
  if (id != null && !_products.any((p) => p.id == id)) {
    throw StateError('Material não encontrado.');
  }
  final previous = id == null
      ? null
      : _products.firstWhere((p) => p.id == id);
```

```
  final selectedUnit = unit ?? previous?.unit ?? 'UN';
  final minimum = minimumStock ?? previous?.minimumStock ?? 0;
  final selectedLocation = (location ?? previous?.location ?? '').trim();
  if (!Product.units.contains(selectedUnit) ||
      minimum < 0 ||
      minimum > 999999999 ||
      selectedLocation.length > 60) {
    throw const FormatException(
      'Confira unidade, estoque mínimo e localização.',
    );
  }
  if (previous != null &&
      selectedUnit != previous.unit &&
      _movements.any((m) => m.productId == id)) {
    throw const FormatException(
      'A unidade não pode ser alterada após movimentações.',
    );
  }
  final nextId =
      _products.fold<int>(0, (max, p) => p.id > max ? p.id : max) + 1;
  final product = Product(
    id: id ?? nextId,
    partNumber: partNumber.trim().toUpperCase(),
    name: name.trim(),
    category: category.trim(),
    unit: selectedUnit,
    minimumStock: minimum,
    location: selectedLocation,
  );
  final updated = id == null
      ? [..._products, product]
      : _products.map((p) => p.id == id ? product : p).toList();
  isSaving = true;
  _notify();
  try {
    if (api != null) {
      final saved = await api!.saveProduct(product, update: id != null);
      _products = id == null
          ? [..._products, saved]
          : _products.map((p) => p.id == id ? saved : p).toList();
      await _persist();
      return;
    }
    await storage.write(_encode(updated));
    _products = updated;
  } finally {
    isSaving = false;
    _notify();
  }
}

Future<void> moveStock({
  required int productId,
  required int quantity,
  required bool incoming,
  required String reason,
}) async {
  if (!canWrite || !isReady || isLoading || isSaving) {
    throw StateError('Catálogo indisponível.');
  }
  if (!_products.any((p) => p.id == productId)) {
    throw const FormatException('Selecione um material válido.');
  }
  if (_products.firstWhere((p) => p.id == productId).needsCodeReview) {
    throw const FormatException(
      'Atualize o part number deste material para o padrão de 8 dígitos antes de movimentar.',
    );
  }
  if (quantity <= 0 || quantity > 999999999) {
    throw const FormatException(
      'Informe uma quantidade inteira entre 1 e 999999999.',
    );
  }
  if (reason.trim().isEmpty || reason.trim().length > 150) {
    throw const FormatException('Informe o motivo com até 150 caracteres.');
  }
  if (!incoming && quantity > balance(productId)) {
    throw const FormatException('Saldo insuficiente para esta saída.');
  }
  final movement = StockMovement(
    productId: productId,
    quantity: incoming ? quantity : -quantity,
```

```
      reason: reason.trim(),
      createdAt: DateTime.now().toUtc(),
    );
    final updated = [..._movements, movement];
    isSaving = true;
    _notify();
    try {
      if (api != null) {
        try {
          await api!.createMovement(movement);
        } on ApiException catch (error) {
          if (error.statusCode == 409) {
            _conflicts.add(movement);
            await _persist();
            throw const FormatException(
              'Saldo insuficiente. Revise esta movimentação; ela não será reenviada.',
            );
          }
          _pending.add(movement);
        } catch (_) {
          _pending.add(movement);
        }
        _movements = updated;
        await _persist();
        return;
      }
      await storage.write(_encode(_products, movements: updated));
      _movements = updated;
    } finally {
      isSaving = false;
      _notify();
    }
  }

  String _encode(List<Product> products, {List<StockMovement>? movements}) =>
      jsonEncode({
        'version': 1,
        'movements': (movements ?? _movements).map((m) => m.toJson()).toList(),
        'products': products.map((p) => p.toJson()).toList(),
        'pendingMovements': _pending.map((m) => m.toJson()).toList(),
      });

  Future<void> _persist() async => storage.write(_encode(_products));

  Future<void> sync() async {
    if (api == null) return;
    final remaining = <StockMovement>[];
    for (final movement in List.of(_pending)) {
      try {
        await api!.createMovement(movement);
      } on ApiException catch (error) {
        if (error.statusCode == 409) {
          _conflicts.add(movement);
          continue;
        }
        remaining.add(movement);
        break;
      } catch (_) {
        remaining.add(movement);
        break;
      }
    }
    _pending
      ..clear()
      ..addAll(remaining);
    await _persist();
    _notify();
  }

  Future<List<StockMovement>> history(int productId) async {
    if (api == null) {
      return _movements
          .where((movement) => movement.productId == productId)
          .toList();
    }
    return api!.movements(productId);
  }
}
```

Arquivo: mobile/lib/features/products/product_form_page.dart

```
import 'package:flutter/material.dart';
```

```
import 'package:flutter/services.dart';

import 'product.dart';
import 'product_catalog.dart';

class ProductFormPage extends StatefulWidget {
  const ProductFormPage({super.key, required this.catalog, this.product});

  final ProductCatalog catalog;
  final Product? product;

  @override
  State<ProductFormPage> createState() => _ProductFormPageState();
}

class _ProductFormPageState extends State<ProductFormPage> {
  final _form = GlobalKey<FormState>();
  late final _partNumber = TextEditingController(
    text: widget.product?.partNumber,
  );
  late final _name = TextEditingController(text: widget.product?.name);
  late final _category = TextEditingController(text: widget.product?.category);
  late final _location = TextEditingController(text: widget.product?.location);
  late final _minimum = TextEditingController(
    text: (widget.product?.minimumStock ?? 0).toString(),
  );
  late String _unit = widget.product?.unit ?? 'UN';
  bool get _unitLocked =>
      widget.product != null &&
      widget.catalog.movements.any((m) => m.productId == widget.product!.id);
  bool _saving = false;
  String? _error;

  @override
  void dispose() {
    _partNumber.dispose();
    _name.dispose();
    _category.dispose();
    _location.dispose();
    _minimum.dispose();
    super.dispose();
  }

  Future<void> _save() async {
    if (_saving || !_form.currentState!.validate()) return;
    FocusScope.of(context).unfocus();
    setState(() {
      _saving = true;
      _error = null;
    });
    try {
      await widget.catalog.save(
        id: widget.product?.id,
        partNumber: _partNumber.text,
        name: _name.text,
        category: _category.text,
        unit: _unit,
        minimumStock: int.parse(_minimum.text.trim()),
        location: _location.text,
      );
      if (mounted) {
        setState(() => _saving = false);
        Navigator.pop(context, true);
      }
    } catch (_) {
      if (mounted) {
        setState(() {
          _saving = false;
          _error =
              'Não foi possível salvar o material. Confira os campos e tente novamente. '
              'A alteração não foi aplicada.';
        });
      }
    }
  }

  String? _required(String? value, String label, int limit) {
    if (value == null || value.trim().isEmpty) return 'Informe $label.';
    if (value.trim().length > limit) return 'Use até $limit caracteres.';
    return null;
  }
```

```
@override
Widget build(BuildContext context) => PopScope(
  canPop: !_saving,
  child: Scaffold(
    appBar: AppBar(
      title: Text(
        widget.product == null ? 'Novo material' : 'Editar material',
      ),
    ),
    body: SafeArea(
      child: Form(
        key: _form,
        child: SingleChildScrollView(
          padding: const EdgeInsets.all(20),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              const Text(
                'Cadastro local para demonstração. Não envia dados para a API.',
              ),
              const SizedBox(height: 20),
              TextFormField(
                key: const Key('partNumberField'),
                controller: _partNumber,
                enabled: !_saving,
                textCapitalization: TextCapitalization.characters,
                textInputAction: TextInputAction.next,
                maxLength: 8,
                keyboardType: TextInputType.number,
                inputFormatters: [FilteringTextInputFormatter.digitsOnly],
                decoration: const InputDecoration(
                  labelText: 'Part number',
                  helperText: '8 dígitos • prefixos 47, 48, 84 ou 86',
                ),
                validator: (value) => widget.catalog.validatePartNumber(
                  value,
                  excludingId: widget.product?.id,
                ),
              ),
              const SizedBox(height: 12),
              TextFormField(
                key: const Key('nameField'),
                controller: _name,
                enabled: !_saving,
                textCapitalization: TextCapitalization.sentences,
                textInputAction: TextInputAction.next,
                maxLength: 100,
                decoration: const InputDecoration(
                  labelText: 'Nome do material',
                ),
                validator: (value) =>
                    _required(value, 'o nome do material', 100),
              ),
              const SizedBox(height: 12),
              TextFormField(
                key: const Key('categoryField'),
                controller: _category,
                enabled: !_saving,
                textCapitalization: TextCapitalization.sentences,
                textInputAction: TextInputAction.done,
                onFieldSubmitted: (_) => _save(),
                maxLength: 60,
                decoration: const InputDecoration(labelText: 'Categoria'),
                validator: (value) => _required(value, 'a categoria', 60),
              ),
              const SizedBox(height: 12),
              DropdownButtonFormField<String>(
                initialValue: _unit,
                decoration: const InputDecoration(
                  labelText: 'Unidade de medida',
                ),
                items: Product.units
                    .map(
                      (unit) =>
                          DropdownMenuItem(value: unit, child: Text(unit)),
                    )
                    .toList(),
                onChanged: _saving || _unitLocked
                    ? null
                    : (value) => setState(() => _unit = value!),
              ),
```

```
                const SizedBox(height: 8),
                Text(
                  _unitLocked
                      ? 'Unidade bloqueada: este material já possui movimentações.'
                      : 'UN: unidade • PC: peça • PAR: par • CX: caixa • PCT: pacote • RL: rolo.
                          Quantidades inteiras; sem conversão automática.',
                ),
                const SizedBox(height: 16),
                TextFormField(
                  key: const Key('locationField'),
                  controller: _location,
                  enabled: !_saving,
                  maxLength: 60,
                  decoration: const InputDecoration(
                    labelText: 'Localização',
                    hintText: 'Ex.: A-01-02',
                  ),
                ),
                const SizedBox(height: 12),
                TextFormField(
                  key: const Key('minimumStockField'),
                  controller: _minimum,
                  enabled: !_saving,
                  keyboardType: TextInputType.number,
                  decoration: const InputDecoration(
                    labelText: 'Estoque mínimo',
                    helperText: '0 desativa o alerta de reposição.',
                  ),
                  validator: (value) {
                    final minimum = int.tryParse(value?.trim() ?? '');
                    return minimum == null || minimum < 0 || minimum > 999999999
                        ? 'Informe um inteiro entre 0 e 999999999.'
                        : null;
                  },
                ),
                if (_error != null) ...[
                  const SizedBox(height: 16),
                  Text(
                    _error!,
                    style: TextStyle(
                      color: Theme.of(context).colorScheme.error,
                    ),
                  ),
                ],
                const SizedBox(height: 20),
                FilledButton.icon(
                  key: const Key('saveProductButton'),
                  onPressed: _saving ? null : _save,
                  icon: _saving
                      ? const SizedBox(
                          width: 18,
                          height: 18,
                          child: CircularProgressIndicator(strokeWidth: 2),
                        )
                      : const Icon(Icons.save_outlined),
                  label: Text(_saving ? 'Salvando…' : 'Salvar material'),
                ),
              ],
            ),
          ),
        ),
      ),
    ),
  );
}
```

Arquivo: mobile/lib/features/products/product_storage.dart

```
export 'product_storage_contract.dart';
export 'product_storage_io.dart'
    if (dart.library.js_interop) 'product_storage_web.dart';
```

Arquivo: mobile/lib/features/products/product_storage_contract.dart

```
abstract interface class ProductStorage {
  Future<String?> read();
  Future<void> write(String content);
}
```

Arquivo: mobile/lib/features/products/product_storage_io.dart

```
import 'dart:io';
```

```
import 'package:path_provider/path_provider.dart';

import 'product_storage_contract.dart';

ProductStorage createProductStorage() => FileProductStorage();

/// Arquivo privado do app. A substituição ocorre somente após a gravação completa.
class FileProductStorage implements ProductStorage {
  FileProductStorage({Future<Directory> Function()? directory})
    : _directory = directory ?? getApplicationDocumentsDirectory;

  final Future<Directory> Function() _directory;

  Future<File> _file() async {
    final directory = await _directory();
    await directory.create(recursive: true);
    return File('${directory.path}/pim_products_v1.json');
  }

  @override
  Future<String?> read() async {
    final file = await _file();
    return await file.exists() ? file.readAsString() : null;
  }

  @override
  Future<void> write(String content) async {
    final file = await _file();
    final temporary = File('${file.path}.tmp');
    await temporary.writeAsString(content, flush: true);
    await temporary.rename(file.path);
  }
}
```

Arquivo: mobile/lib/features/products/product_storage_web.dart

```
import 'package:web/web.dart' as web;

import 'product_storage_contract.dart';

ProductStorage createProductStorage() => BrowserProductStorage();

/// Catálogo próprio do navegador. Falhas de quota ou acesso são propagadas à UI.
class BrowserProductStorage implements ProductStorage {
  static const key = 'pim_estoque_products_v1';

  @override
  Future<String?> read() async => web.window.localStorage.getItem(key);

  @override
  Future<void> write(String content) async {
    web.window.localStorage.setItem(key, content);
  }
}
```

Arquivo: mobile/lib/features/products/products_page.dart

```
import 'package:flutter/material.dart';

import 'product_catalog.dart';
import 'product_form_page.dart';
import 'product.dart';

class ProductsPage extends StatefulWidget {
  const ProductsPage({super.key, required this.catalog});

  final ProductCatalog catalog;

  @override
  State<ProductsPage> createState() => _ProductsPageState();
}

class _ProductsPageState extends State<ProductsPage> {
  final _search = TextEditingController();

  @override
  void dispose() {
    _search.dispose();
    super.dispose();
  }
```

```
Future<void> _openForm([Product? product]) async {
  ScaffoldMessenger.of(context).removeCurrentSnackBar();
  final saved = await Navigator.push<bool>(
    context,
    MaterialPageRoute(
      builder: (_) =>
          ProductFormPage(catalog: widget.catalog, product: product),
    ),
  );
  if (saved == true && mounted) {
    // Mostrar o material recém-cadastrado mesmo que uma pesquisa anterior o excluísse.
    setState(_search.clear);
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Material salvo localmente.')),
    );
  }
}

Future<void> _showProduct(Product product) async {
  final edit = await showModalBottomSheet<bool>(
    context: context,
    showDragHandle: true,
    isScrollControlled: true,
    builder: (context) => SafeArea(
      child: SingleChildScrollView(
        padding: const EdgeInsets.fromLTRB(24, 0, 24, 24),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          mainAxisSize: MainAxisSize.min,
          children: [
            Text(
              product.name,
              style: Theme.of(context).textTheme.headlineSmall,
            ),
            const SizedBox(height: 16),
            Text('Part number: ${product.partNumber}'),
            if (product.needsCodeReview)
              const Text(
                'Código antigo fora do padrão. Edite para informar o código correto de 8
                    dígitos.',
              ),
            const SizedBox(height: 8),
            Text('Categoria: ${product.category}'),
            const SizedBox(height: 12),
            Text(
              'Saldo: ${widget.catalog.balance(product.id)} ${product.unit}',
            ),
            Text('Estoque mínimo: ${product.minimumStock} ${product.unit}'),
            Text(
              'Localização: ${product.location.isEmpty ? "Não informada" : product.location}',
            ),
            if (widget.catalog.needsReplenishment(product))
              const Text('Reposição necessária'),
            const SizedBox(height: 20),
            const Text('Cadastro de almoxarifado. Dados salvos localmente.'),
            const SizedBox(height: 20),
            if (widget.catalog.canWrite)
              FilledButton.icon(
                onPressed: () => Navigator.pop(context, true),
                icon: const Icon(Icons.edit_outlined),
                label: const Text('Editar material'),
              ),
            TextButton.icon(
              onPressed: () {
                Navigator.pop(context);
                _showHistory(product);
              },
              icon: const Icon(Icons.history),
              label: const Text('Ver histórico'),
            ),
            const SizedBox(height: 8),
            TextButton(
              onPressed: () => Navigator.pop(context),
              child: const Text('Fechar'),
            ),
          ],
        ),
      ),
    ),
  );
  if (edit == true && mounted) await _openForm(product);
```

```
}

Future<void> _showHistory(Product product) async {
  try {
    final history = await widget.catalog.history(product.id);
    if (!mounted) return;
    await showDialog<void>(
      context: context,
      builder: (context) => AlertDialog(
        title: Text('Histórico: ${product.name}'),
        content: history.isEmpty
            ? const Text('Nenhuma movimentação registrada.')
            : SizedBox(
                width: 360,
                child: ListView(
                  shrinkWrap: true,
                  children: [
                    for (final movement in history)
                      ListTile(
                        title: Text(
                          '${movement.quantity > 0 ? 'Entrada' : 'Saída'}:
                              ${movement.quantity.abs()} ${product.unit}',
                        ),
                        subtitle: Text(movement.reason),
                      ),
                  ],
                ),
              ),
        actions: [
          TextButton(
            onPressed: () => Navigator.pop(context),
            child: const Text('Fechar'),
          ),
        ],
      ),
    );
  } catch (_) {
    if (mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('Não foi possível carregar o histórico.'),
        ),
      );
    }
  }
}

@override
Widget build(BuildContext context) {
  final products = widget.catalog.products
      .where((p) => p.matches(_search.text))
      .toList();
  return CustomScrollView(
    slivers: [
      SliverPadding(
        padding: const EdgeInsets.all(20),
        sliver: SliverToBoxAdapter(
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                'Produtos',
                style: Theme.of(context).textTheme.headlineMedium,
              ),
              const SizedBox(height: 16),
              if (widget.catalog.canWrite)
                FilledButton.icon(
                  onPressed: widget.catalog.isSaving
                      ? null
                      : () => _openForm(),
                  icon: const Icon(Icons.add),
                  label: const Text('Cadastrar material'),
                ),
              const SizedBox(height: 16),
              TextField(
                controller: _search,
                onChanged: (_) => setState(() {}),
                decoration: InputDecoration(
                  labelText: 'Nome, part number ou categoria',
                  prefixIcon: const Icon(Icons.search),
                  suffixIcon: _search.text.isEmpty
                      ? null
```

```
                        : IconButton(
                            tooltip: 'Limpar pesquisa',
                            onPressed: () => setState(_search.clear),
                            icon: const Icon(Icons.close),
                          ),
                    ),
                  ),
                  const SizedBox(height: 16),
                  Text('${products.length} produto(s) encontrado(s)'),
                ],
              ),
            ),
          ),
          if (products.isEmpty)
            const SliverToBoxAdapter(
              child: Padding(
                padding: EdgeInsets.all(24),
                child: Text(
                  'Nenhum produto encontrado. Tente outro nome ou part number.',
                ),
              ),
            )
          else
            SliverPadding(
              padding: const EdgeInsets.fromLTRB(16, 0, 16, 20),
              sliver: SliverList.builder(
                itemCount: products.length,
                itemBuilder: (context, index) {
                  final product = products[index];
                  return Card(
                    child: ListTile(
                      contentPadding: const EdgeInsets.all(16),
                      leading: const Icon(Icons.inventory_2_outlined),
                      title: Text(product.name),
                      subtitle: Text(
                        '${product.partNumber}\n${product.category}',
                      ),
                      isThreeLine: true,
                      trailing: const Icon(Icons.chevron_right),
                      onTap: () => _showProduct(product),
                    ),
                  );
                },
              ),
            ),
        ],
      );
  }
}
```

Arquivo: mobile/lib/features/products/stock_movement.dart

```
class StockMovement {
  const StockMovement({
    required this.productId,
    required this.quantity,
    required this.reason,
    required this.createdAt,
  });
  final int productId;

  /// Positivo para entrada e negativo para saída; nesta etapa, unidades inteiras.
  final int quantity;
  final String reason;
  final DateTime createdAt;

  factory StockMovement.fromJson(Map<String, dynamic> data) {
    final id = data['productId'] ?? data['produtoId'];
    final rawQuantity = data['quantity'] ?? data['quantidade'];
    final type = data['tipo'];
    final quantity = rawQuantity is int
        ? rawQuantity
        : (rawQuantity is num ? rawQuantity.toInt() : null);
    final normalizedQuantity = type == 'saida' && quantity != null
        ? -quantity.abs()
        : quantity;
    final reason = data['reason'] ?? data['motivo'];
    final rawDate = data['createdAt'] ?? data['criadaEmUtc'];
    final date = DateTime.tryParse(rawDate is String ? rawDate : '');
    if (id is! int ||
        id <= 0 ||
        normalizedQuantity is! int ||
```

```
        normalizedQuantity == 0 ||
        reason is! String ||
        reason.trim().isEmpty ||
        reason.length > 150 ||
        date == null) {
      throw const FormatException('Movimentação inválida.');
    }
    return StockMovement(
      productId: id,
      quantity: normalizedQuantity,
      reason: reason,
      createdAt: date,
    );
  }

  Map<String, Object> toJson() => {
    'productId': productId,
    'quantity': quantity,
    'reason': reason,
    'createdAt': createdAt.toIso8601String(),
  };

  Map<String, Object> toApiJson() => {
    'produtoId': productId,
    'quantidade': quantity.abs(),
    'tipo': quantity > 0 ? 'entrada' : 'saida',
    'motivo': reason,
  };
}
```

Arquivo: mobile/lib/features/products/stock_page.dart

```
import 'package:flutter/material.dart';

import 'product_catalog.dart';

class StockPage extends StatefulWidget {
  const StockPage({super.key, required this.catalog});
  final ProductCatalog catalog;
  @override
  State<StockPage> createState() => _StockPageState();
}

class _StockPageState extends State<StockPage> {
  final _form = GlobalKey<FormState>();
  final _quantity = TextEditingController();
  final _reason = TextEditingController();
  int? _productId;
  bool _incoming = true;
  String? _message;
  bool _onlyReplenishment = false;

  @override
  void dispose() {
    _quantity.dispose();
    _reason.dispose();
    super.dispose();
  }

  Future<void> _save() async {
    if (!_form.currentState!.validate() || widget.catalog.isSaving) return;
    setState(() => _message = null);
    try {
      await widget.catalog.moveStock(
        productId: _productId!,
        quantity: int.parse(_quantity.text.trim()),
        incoming: _incoming,
        reason: _reason.text,
      );
      if (mounted) {
        setState(() {
          _message = 'Movimentação salva localmente.';
          _quantity.clear();
          _reason.clear();
        });
      }
    } on FormatException catch (error) {
      if (mounted) setState(() => _message = error.message);
    } catch (_) {
      if (mounted) {
        setState(
```

```
        () => _message = 'Não foi possível salvar. O saldo não foi alterado.',
      );
    }
  }
}

Widget _summary() {
  final product = widget.catalog.products.firstWhere(
    (p) => p.id == _productId,
  );
  final quantity = int.tryParse(_quantity.text.trim());
  final current = widget.catalog.balance(product.id);
  final valid = quantity != null && quantity > 0 && quantity <= 999999999;
  final next = valid ? current + (_incoming ? quantity : -quantity) : null;
  final insufficient = next != null && next < 0;
  return Card.filled(
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            'Resumo da movimentação',
            style: Theme.of(context).textTheme.titleMedium,
          ),
          const SizedBox(height: 8),
          Text('${product.partNumber} • ${product.name}'),
          Text('Operação: ${_incoming ? "Entrada" : "Saída"}'),
          Text('Saldo atual: $current ${product.unit}'),
          Text(
            'Quantidade: ${valid ? "$quantity ${product.unit}" : "Informe uma quantidade
                válida"}',
          ),
          Text(
            'Saldo previsto: ${next == null ? "—" : "$next ${product.unit}"}',
          ),
          if (insufficient)
            Text(
              'Saldo insuficiente. Esta saída será recusada.',
              style: TextStyle(color: Theme.of(context).colorScheme.error),
            ),
          const Text('O saldo só muda após registrar a movimentação.'),
        ],
      ),
    ),
  );
}

@override
Widget build(BuildContext context) {
  final catalog = widget.catalog;
  final products = catalog.products;
  return ListView(
    padding: const EdgeInsets.all(20),
    children: [
      Text(
        'Controle de estoque',
        style: Theme.of(context).textTheme.headlineMedium,
      ),
      const SizedBox(height: 8),
      const Text(
        'Demonstração local • quantidades em unidades inteiras. Materiais começam com saldo zero;
            registre uma entrada para adicionar estoque.',
      ),
      const SizedBox(height: 20),
      if (catalog.api != null)
        Align(
          alignment: Alignment.centerLeft,
          child: Wrap(
            spacing: 12,
            crossAxisAlignment: WrapCrossAlignment.center,
            children: [
              OutlinedButton.icon(
                onPressed: catalog.pendingCount == 0 ? null : catalog.sync,
                icon: const Icon(Icons.sync),
                label: const Text('Sincronizar'),
              ),
              Text('${catalog.pendingCount} pendente(s)'),
            ],
          ),
        ),
      if (catalog.api != null) const SizedBox(height: 12),
```

```
Form(
  key: _form,
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.stretch,
    children: [
      DropdownButtonFormField<int>(
        initialValue: _productId,
        isExpanded: true,
        decoration: const InputDecoration(labelText: 'Material'),
        items: products
            .map(
              (p) => DropdownMenuItem(
                value: p.id,
                child: Text(
                  '${p.partNumber} • ${p.name}',
                  overflow: TextOverflow.ellipsis,
                ),
              ),
            )
            .toList(),
        onChanged: catalog.isSaving
            ? null
            : (value) => setState(() {
                _productId = value;
                _message = null;
              }),
        validator: (value) =>
            value == null ? 'Selecione um material.' : null,
      ),
      if (_productId != null)
        Padding(
          padding: const EdgeInsets.symmetric(vertical: 12),
          child: Text(
            'Saldo disponível: ${catalog.balance(_productId!)} ${products.firstWhere((p) =>
                p.id == _productId).unit}',
          ),
        ),
      const SizedBox(height: 12),
      Wrap(
        spacing: 12,
        children: [
          ChoiceChip(
            label: const Text('Entrada'),
            selected: _incoming,
            onSelected: catalog.isSaving
                ? null
                : (_) => setState(() => _incoming = true),
          ),
          ChoiceChip(
            label: const Text('Saída'),
            selected: !_incoming,
            onSelected: catalog.isSaving
                ? null
                : (_) => setState(() => _incoming = false),
          ),
        ],
      ),
      const SizedBox(height: 16),
      TextFormField(
        controller: _quantity,
        enabled: !catalog.isSaving,
        keyboardType: TextInputType.number,
        onChanged: (_) => setState(() => _message = null),
        decoration: const InputDecoration(labelText: 'Quantidade'),
        validator: (value) {
          final quantity = int.tryParse(value?.trim() ?? '');
          return quantity == null ||
                  quantity <= 0 ||
                  quantity > 999999999
              ? 'Informe um número inteiro positivo até 999999999.'
              : null;
        },
      ),
      const SizedBox(height: 16),
      TextFormField(
        controller: _reason,
        enabled: !catalog.isSaving,
        maxLength: 150,
        decoration: const InputDecoration(
          labelText: 'Motivo / referência',
        ),
        validator: (value) => value == null || value.trim().isEmpty
```

```
                    ? 'Informe o motivo.'
                    : null,
              ),
              const SizedBox(height: 12),
              if (_productId != null) _summary(),
              const SizedBox(height: 12),
              FilledButton.icon(
                onPressed: !catalog.canWrite || catalog.isSaving ? null : _save,
                icon: const Icon(Icons.swap_vert),
                label: Text(
                  catalog.isSaving ? 'Salvando…' : 'Registrar movimentação',
                ),
              ),
              if (_message != null)
                Padding(
                  padding: const EdgeInsets.symmetric(vertical: 12),
                  child: Semantics(liveRegion: true, child: Text(_message!)),
                ),
            ],
          ),
        ),
        const SizedBox(height: 24),
        Text(
          'Saldos por material',
          style: Theme.of(context).textTheme.titleLarge,
        ),
        FilterChip(
          label: const Text('Somente reposição'),
          selected: _onlyReplenishment,
          onSelected: (value) => setState(() => _onlyReplenishment = value),
        ),
        if (_onlyReplenishment && catalog.replenishment.isEmpty)
          const Text(
            'Nenhum material precisa de reposição com os mínimos configurados.',
          ),
        for (final product
            in (_onlyReplenishment ? catalog.replenishment : products))
          ListTile(
            contentPadding: EdgeInsets.zero,
            title: Text(product.name),
            subtitle: Text(
              '${product.partNumber} • ${product.location.isEmpty ? "Sem localização" :
                  product.location}${catalog.needsReplenishment(product) ? " • Repor" : ""}',
            ),
            trailing: Text('${catalog.balance(product.id)} ${product.unit}'),
          ),
        const SizedBox(height: 24),
        Text(
          'Últimas movimentações',
          style: Theme.of(context).textTheme.titleLarge,
        ),
        if (catalog.movements.isEmpty)
          const Padding(
            padding: EdgeInsets.symmetric(vertical: 12),
            child: Text('Nenhuma movimentação registrada.'),
          ),
        for (final movement in catalog.movements.take(30))
          ListTile(
            contentPadding: EdgeInsets.zero,
            leading: Icon(
              movement.quantity > 0 ? Icons.south_west : Icons.north_east,
            ),
            title: Text(
              '${products.firstWhere((p) => p.id == movement.productId).partNumber} •
                  ${movement.quantity > 0 ? '+' : ''}${movement.quantity} ${products.firstWhere((p)
                      => p.id == movement.productId).unit}',
            ),
            subtitle: Text(
              '${movement.reason}\n${movement.createdAt.toLocal().toString().substring(0, 16)}',
            ),
          ),
      ],
    );
  }
}
```

ANEXO A – FICHA DE CONTROLE DO PIM

FICHA DE CONTROLE DO PIM

Grupo Nº: n/a Ano: 2026 Período: 4º/3º – 2026/2 Orientador: n/a

Tema: Desenvolvimento e Implantação de uma Solução Tecnológica Corporativa Integrada para Apoiar a Transformação Digital de uma Empresa

Alunos:

RA	Nome	Curso	Visto do aluno
R661946	Hendryo Batista Sarmento	CST ADS	
H32CIA9	Lucas Alcântara Leal	CST ADS	
T319595	Matheus Terci	CST ADS	
H76DJJ6	Vinnicius Gabriel Matos dos Santos	CST ADS	

Registros:

Data do encontro	Observações